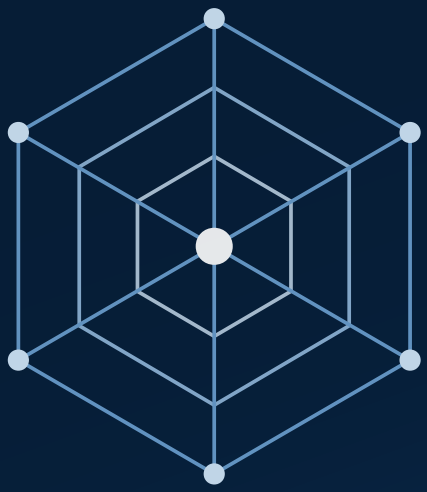
VOLUME 2 OF 6



THE COMPLETE SNOWFLAKE LEARNING BIBLE

Zero to Expert | Snowflake Core | SQL | Data Engineering | Architecture | Streaming | Governance | Cortex AI | RAG | Agents | MCP | Migration | Production Engineering

SQL, Programming, Ingestion, Streaming and ELT

Part III — Snowflake SQL and Programming (Ch. 8–11) and Part IV — Ingestion, Transformation, and Streaming (Ch. 12–16), with the updated master manifest and coverage matrix.

Edition 2026.10 · Research cutoff: 10 October 2026 · Verified against docs.snowflake.com and modelcontextprotocol.io
An independent educational textbook. Not produced, endorsed or reviewed by Snowflake Inc. Snowflake and related names are trademarks of their owners.

Contents

About Volume 2 7
What this volume covers 7
How the examples in this volume were verified 7
Errata for Volume 1 8
Master manifest (updated for Volume 2) 8
Coverage matrix (Volume 2) 10

PART III — SNOWFLAKE SQL AND PROGRAMMING 12

Chapter 8: Snowflake SQL — Complete Reference 13
8.1 How to use this chapter 13
8.2 Statement categories and their behaviour 15
8.3 DDL in depth 15
8.4 DML in depth 17
8.5 Transactions and locking 19
8.6 DCL in one page 20
8.7 Snowflake query features 20
8.8 Semi-structured data queries 25
8.9 Time Travel 27
8.10 Querying metadata 28
8.11 Function catalogue 30
8.12 Dynamic SQL 31
8.13 Guided lab 32
8.14 Production use case: a safe daily rebuild 32
8.15 Common mistakes 33
8.16 Troubleshooting exercise 33
8.17 Knowledge check 33
8.18 Hands-on assignment 34
8.19 Chapter summary 34
8.20 Key takeaways 34
8.21 Further reading 35
8.22 Solutions 35

Chapter 9: Advanced SQL and Data Modeling 37
9.0 How this chapter was tested 37
9.1 Why model data at all? 37
9.2 Star schemas, snowflake schemas and alternatives 38
9.3 Keys: natural, surrogate and hash 40
9.4 Slowly Changing Dimensions 40
9.5 Incremental loads, watermarks and idempotency 46
9.6 Reconciliation 47
9.7 Analytical patterns 48
9.8 Refactoring SQL 51
9.9 Modelling for analytics and AI 52
9.10 Common mistakes 53
9.11 Troubleshooting exercise 53

9.12 Practice exercises ... 53
9.13 Chapter summary ... 53
9.14 Key takeaways ... 54
9.15 Further reading ... 54
9.16 Solutions ... 54

Chapter 10: Snowflake Scripting, Procedures, and Functions ... 57
10.1 Choosing a programming model ... 57
10.2 Snowflake Scripting fundamentals ... 58
10.3 Error handling ... 60
10.4 Transactions in procedures ... 61
10.5 A production-style SQL procedure ... 61
10.6 Owner's rights, caller's rights and restricted caller's rights ... 63
10.7 Python stored procedures ... 64
10.8 JavaScript stored procedures ... 65
10.9 User-defined functions ... 65
10.10 Calling external services: external access integrations ... 67
10.11 Logging and tracing ... 68
10.12 Testing and deployment ... 69
10.13 Guided lab ... 69
10.14 Production use case: replacing a fragile external script ... 70
10.15 Common mistakes ... 71
10.16 Troubleshooting exercise ... 71
10.17 Knowledge check ... 71
10.18 Hands-on assignment ... 72
10.19 Chapter summary ... 72
10.20 Key takeaways ... 72
10.21 Further reading ... 72
10.22 Solutions ... 73

Chapter 11: Snowpark and Developer Tooling ... 74
11.0 What was executed ... 74
11.1 What Snowpark is ... 74
11.2 Setting up ... 76
11.3 The ByteMart Snowpark module ... 76
11.4 Testing Snowpark code ... 80
11.5 Python UDFs and procedures from Snowpark ... 81
11.6 Package and dependency management ... 82
11.7 Choosing the right tool ... 82
11.8 Developer tooling ... 83
11.9 Guided lab ... 85
11.10 Production use case: ByteMart's Python transformations ... 86
11.11 Common mistakes ... 86
11.12 Troubleshooting exercise ... 87
11.13 Knowledge check ... 87
11.14 Hands-on assignment ... 87
11.15 Chapter summary ... 87
11.16 Key takeaways ... 88
11.17 Further reading ... 88

11.18 Solutions 88

PART IV — INGESTION, TRANSFORMATION, AND STREAMING 90

Chapter 12: File Formats, Stages, and Bulk Loading 91

12.1 The loading landscape 91
12.2 File formats 92
12.3 Stages 94
12.4 COPY INTO in depth 97
12.5 Schema evolution 100
12.6 Performance and cost of loading 100
12.7 Permissions for loading 101
12.8 End-to-end bulk ingestion pipeline 102
12.9 Guided lab: load without a cloud account 104
12.10 Common mistakes 105
12.11 Troubleshooting exercise 105
12.12 Knowledge check 105
12.13 Hands-on assignment 106
12.14 Chapter summary 106
12.15 Key takeaways 106
12.16 Further reading 106
12.17 Solutions 107

Chapter 13: Snowpipe and Continuous Ingestion 108

13.1 Batch, micro-batch and streaming 108
13.2 Snowpipe 109
13.3 Snowpipe Streaming 112
13.4 Kafka, Openflow and other paths 115
13.5 Designing for real-world event streams 115
13.6 Cost and monitoring 116
13.7 Guided lab 117
13.8 Production use case: ByteMart clickstream 117
13.9 Common mistakes 118
13.10 Troubleshooting exercise 118
13.11 Knowledge check 118
13.12 Hands-on assignment 118
13.13 Chapter summary 119
13.14 Key takeaways 119
13.15 Further reading 119
13.16 Solutions 120

Chapter 14: Change Data Capture and Streams 121

14.1 Change data capture patterns 121
14.2 Streams 122
14.3 Applying stream changes with MERGE 125
14.4 The CHANGES clause 127
14.5 Staleness and recovery 127
14.6 Applying CDC events from external databases 128
14.7 Designing incremental processing 130
14.8 Guided lab 130

14.9 Production use case: ByteMart inventory replica 131
14.10 Common mistakes 131
14.11 Troubleshooting exercise 132
14.12 Knowledge check 132
14.13 Hands-on assignments 132
14.14 Chapter summary 132
14.15 Key takeaways 133
14.16 Further reading 133
14.17 Solutions 133

Chapter 15: Tasks, Dynamic Tables, and Orchestration 135

15.1 Tasks 135
15.2 Task graphs 137
15.3 Dynamic tables 140
15.4 Custom incremental dynamic tables 142
15.5 Materialized views 143
15.6 Choosing the right tool 143
15.7 Alerts 144
15.8 External orchestration 145
15.9 Backfills and reprocessing 145
15.10 Guided lab 146
15.11 Production use case: ByteMart's platform schedule 146
15.12 Common mistakes 147
15.13 Troubleshooting exercise 147
15.14 Knowledge check 147
15.15 Hands-on assignment 148
15.16 Chapter summary 148
15.17 Key takeaways 148
15.18 Further reading 148
15.19 Solutions 149

Chapter 16: Modern Data Engineering and ELT 150

16.1 Platform architecture 151
16.2 Data contracts 152
16.3 Data quality 152
16.4 dbt fundamentals 153
16.5 Incremental models 161
16.6 Snapshots 163
16.7 When a test fails 163
16.8 Packages and documentation 164
16.9 Schema evolution, lineage and observability 164
16.10 dbt Projects on Snowflake 165
16.11 CI/CD for dbt 166
16.12 Other transformation approaches 167
16.13 End-to-end: the ByteMart platform so far 168
16.14 Guided lab 168
16.15 Production use case: from scripts to a governed dbt platform 169
16.16 Common mistakes 169
16.17 Troubleshooting exercise 170

16.18 Knowledge check 170
16.19 Hands-on assignment 170
16.20 Chapter summary 170
16.21 Key takeaways 171
16.22 Further reading 171
16.23 Solutions 171

About Volume 2

What this volume covers

Volume 2 turns the foundations of Volume 1 into working engineering skill. **Part III — Snowflake SQL and Programming** (Chapters 8–11) covers Snowflake's SQL dialect in depth, advanced SQL and dimensional modelling, Snowflake Scripting, stored procedures and user-defined functions, and Snowpark with the developer tooling around it. **Part IV — Ingestion, Transformation, and Streaming** (Chapters 12–16) follows data from files and event streams into Snowflake, through change data capture and orchestration, to a tested, layered platform built with dbt.

Volume	Parts	Chapters	Theme
1	Front matter, What's New, I, II	1–7	Foundations and Snowflake Core
2 (this volume)	III, IV	8–16	SQL, programming, ingestion, streaming and ELT
3	V, VI	17–22	Performance, cost, operations, security, governance and AI guardrails
4	VII	23–31	AI fundamentals, Cortex AI, Search, Analyst, Agents, RAG, MCP and CoCo
5	VIII, IX, X	32–39	Streamlit, applications, migration, integration, enterprise architecture and recovery
6	XI, XII	40–44	Six guided projects, glossary, misconceptions, troubleshooting handbook and roadmap

Before you start. This volume assumes Volume 1: SQL from Chapter 3, the object model and lab environment from Chapter 6 (`bytemart_lab` database, `lab_role`, `lab_wh`), and the type system from Chapter 7. The conventions, status labels (GA, Preview, Private Preview, Deprecated, Announced) and research cutoff (**10 October 2026**) described in Volume 1's "How to Use This Book" apply unchanged. "What's New in Snowflake" at the front of Volume 1 summarises the 2026 changes that these chapters build on.

How the examples in this volume were verified

Honesty about testing is a rule of this book: code is described as *executed* only when it was actually run.

Chapter	Verification
8 Snowflake SQL	Syntax and behaviour checked against Snowflake documentation; expected outputs for the portable examples derived and cross-checked with equivalent queries on DuckDB 1.5.6. Snowflake-only features (Time Travel, `ASOF JOIN`, `CONNECT BY`, `FLATTEN`…) are documentation-verified, not executed
9 Advanced SQL and modelling	All result tables **executed** on DuckDB 1.5.6 and inserted automatically; Snowflake-specific differences noted in the text

Chapter	Verification
10 Scripting, procedures, functions	Documentation-verified; not executed (Snowflake Scripting and procedure runtimes exist only in Snowflake)
11 Snowpark and tooling	Snowpark module and tests **executed** with Snowpark 1.55.1 local testing mode and pytest; Snowflake-side commands documentation-verified
12 Bulk loading	Documentation-verified; lab expectations derived from documented behaviour
13 Continuous ingestion	Documentation-verified; the Python streaming client follows the documented SDK API and is not executed
14 CDC and streams	Stream semantics documentation-verified; the MERGE and CDC-apply logic **executed** on DuckDB with stream contents simulated exactly as documented
15 Tasks, dynamic tables, orchestration	Documentation-verified; not executed
16 Modern data engineering and ELT	The complete dbt project **executed** with dbt Core 1.12.5 + dbt-duckdb 1.11.0; outputs captured automatically; Snowflake deployment documentation-verified

The source package that accompanies this volume contains every chapter's Markdown, the diagrams, the Snowpark and dbt projects, and the scripts that executed the examples, so you can re-run them yourself.

Errata for Volume 1

- **What's New, item 12 (dynamic tables):** adaptive refresh mode entered Public Preview on 26 May 2026 and became **Generally Available on 30 July 2026**; Volume 1 still described it as Public Preview. Custom incremental dynamic tables became GA on 27 July 2026. Chapter 15 reflects the correct status.

Master manifest (updated for Volume 2)

#	Chapter	Vol	Written	Doc-verified	Rendered
—	How to Use This Book; What's New in Snowflake	1	Yes	Yes	Yes
1	Introduction to Data Engineering	1	Yes	n/a (stable concepts)	Yes
2	Cloud Computing Fundamentals	1	Yes	n/a (stable concepts)	Yes
3	SQL From Zero	1	Yes	Executed on DuckDB	Yes
4	Introduction to Snowflake	1	Yes	Yes	Yes
5	Snowflake Architecture in Depth	1	Yes	Yes	Yes
6	Snowflake Account and Object Fundamentals	1	Yes	Yes	Yes
7	Snowflake Data Types and Storage	1	Yes	Yes	Yes
8	Snowflake SQL — Complete Reference	2	Yes	Doc-verified; expected outputs cross-checked on DuckDB	Yes

#	Chapter	Vol	Written	Doc-verified	Rendered
9	Advanced SQL and Data Modeling	2	Yes	Executed on DuckDB	Yes
10	Snowflake Scripting, Procedures, and Functions	2	Yes	Yes	Yes
11	Snowpark and Developer Tooling	2	Yes	Executed (Snowpark local testing + pytest)	Yes
12	File Formats, Stages, and Bulk Loading	2	Yes	Yes	Yes
13	Snowpipe and Continuous Ingestion	2	Yes	Yes	Yes
14	Change Data Capture and Streams	2	Yes	Yes; MERGE/CDC logic executed on DuckDB	Yes
15	Tasks, Dynamic Tables, and Orchestration	2	Yes	Yes	Yes
16	Modern Data Engineering and ELT	2	Yes	Yes; dbt project executed on DuckDB	Yes
17	Performance Optimization	3	Planned	—	—
18	Cost Optimization and FinOps	3	Planned	—	—
19	Monitoring, Reliability, and Troubleshooting	3	Planned	—	—
20	Snowflake Security Architecture	3	Planned	—	—
21	Data Governance and Protection	3	Planned	—	—
22	AI Governance, Guardrails, and Responsible AI	3	Planned	—	—
23	AI Fundamentals for Snowflake Engineers	4	Planned	—	—
24	Snowflake Cortex AI Ecosystem	4	Planned	—	—
25	Cortex AI Functions — Complete Practical Guide	4	Planned	—	—
26	Cortex Search and Enterprise Search	4	Planned	—	—
27	Cortex Analyst and Natural-Language Analytics	4	Planned	—	—
28	Cortex Agents and Agentic Workflows	4	Planned	—	—
29	RAG From Zero to Production	4	Planned	—	—
30	MCP — Model Context Protocol and Snowflake	4	Planned	—	—
31	CoCo (formerly Cortex Code) and AI-Assisted Data Engineering	4	Planned	—	—
32	Streamlit From Zero	5	Planned	—	—
33	Streamlit in Snowflake	5	Planned	—	—
34	Full-Stack Snowflake Applications	5	Planned	—	—
35	Snowflake Migration From Zero to Expert	5	Planned	—	—

#	Chapter	Vol	Written	Doc-verified	Rendered
36	Advanced Migration Engineering	5	Planned	—	—
37	Integrations and the Modern Data Stack	5	Planned	—	—
38	Enterprise Architecture and Workload Design	5	Planned	—	—
39	Advanced Reliability and Recovery	5	Planned	—	—
40	Guided Projects From Beginner to Expert	6	Planned	—	—
41	Snowflake Glossary	6	Planned	—	—
42	Common Mistakes and Misconceptions	6	Planned	—	—
43	Practical Troubleshooting Handbook	6	Planned	—	—
44	Complete Learning Roadmap	6	Planned	—	—

Final compilation status: Volumes 1–2 complete. Volume 3 (Chapters 17–22) is next. The master PDF will be compiled after Volume 6.

Coverage matrix (Volume 2)

Subject	Chapters	Concepts covered	Practical implementation	Exercises	Current docs verified	Remaining limitations
Snowflake SQL dialect	8	DDL incl. `CREATE OR ALTER`, clone, swap, undrop; DML incl. multi-table insert, MERGE; transactions; DCL; query extensions (`QUALIFY`, `EXCLUDE`, `ILIKE ANY`, `ASOF JOIN`, `PIVOT`, recursive CTE, `CONNECT BY`, flow operator); semi-structured; Time Travel; metadata views; dynamic SQL	ByteMart sales schema, lab and swap-based rebuild use case	Yes, with solutions	Yes	Some outputs derived rather than executed (Snowflake-only features)
Advanced SQL and modelling	9	Star schema, date dimension, surrogate keys, SCD 0–6, SCD2 MERGE, fact grain, range joins, incremental watermarks, unknown members, reconciliation, sessionization, gaps and islands, hierarchies, modelling for AI	Executed dimensional model of ByteMart	Yes, 5 executed solutions	Executed on DuckDB; Snowflake notes	Performance aspects deferred to Ch 17
Scripting, procedures, UDFs	10	Snowflake Scripting, cursors, `RESULTSET`, exceptions, transactions, owner's/caller's/restricted caller's rights, Python/JavaScript procedures, SQL/Python UDFs, vectorized UDFs, UDTFs, external access, logging, testing	Production-grade idempotent load procedure with run log	Yes, with solutions	Yes	Not executed; Java/Scala handlers mentioned only

Subject	Chapters	Concepts covered	Practical implementation	Exercises	Current docs verified	Remaining limitations
Snowpark and tooling	11	DataFrames, lazy evaluation, sessions, UDFs/procedures from Python, local testing, packages, SPCS overview, Snowflake CLI, Git integration, Workspaces, CI/CD	Snowpark module with 4 passing tests	Yes, with solutions	Yes	SPCS deepened in Vol 5
Bulk loading	12	File formats, all stage types, storage integrations (S3/Azure/GCS), COPY options and transformations, metadata columns, validation, error handling, load history and metadata, schema inference/evolution, permissions, file sizing	End-to-end audited bulk pipeline with rejects and MERGE	Yes, with solutions	Yes	Cloud-console steps described, not screenshotted
Continuous ingestion	13	Batch vs micro-batch vs streaming, Snowpipe auto-ingest and REST, pipe operations and recreation, Snowpipe Streaming high-performance architecture, channels, offset tokens, delivery semantics, Kafka connector 4.x, Openflow, late/duplicate/out-of-order events, retries and recovery, cost	Snowpipe lab; streaming client; clickstream design	Yes, with solutions	Yes	Python SDK accessor for committed offsets left as a documented stub; Openflow deepened in Ch 37
CDC and streams	14	CDC patterns, stream types and metadata, offsets and transactions, canonical MERGE, `CHANGES`, staleness and recovery, external CDC apply with LSN guards, incremental design	Executed MERGE/CDC examples incl. the naive-MERGE failure	Yes, with solutions	Yes; logic executed	Stream objects themselves not executable outside Snowflake
Orchestration	15	Serverless/warehouse/triggered tasks, task graphs (config, return values, finalizers, overlap, retries), dynamic tables (lag, refresh modes incl. adaptive and custom incremental, frozen regions, backfill), materialized views, decision matrix, alerts, external orchestration, backfills, monitoring	Same pipeline as task graph and dynamic tables; alerts	Yes, with solutions	Yes	Not executed
Modern data engineering and ELT	16	Medallion layers, environments, data contracts, quality, dbt (models, sources, seeds, tests, unit tests, snapshots, macros, packages), incremental strategies, schema evolution, lineage, observability, dbt Projects on Snowflake, CI/CD, Matillion and other tools	Executed dbt project incl. late-update failure, snapshot and failing test	Yes, with solutions	Yes; dbt executed on DuckDB	Snowflake deployment not executed

PART III

Snowflake SQL and Programming

Roadmap levels 2–4. Snowflake's SQL dialect in depth, advanced SQL and dimensional modelling, Snowflake Scripting, procedures and functions, and Snowpark with modern developer tooling.

CHAPTER 8

Snowflake SQL — Complete Reference

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- classify every SQL statement as DDL, DML, DQL, DCL or TCL and predict its transactional behaviour;
- use Snowflake's DDL conveniences — `OR REPLACE`, `IF NOT EXISTS`, `CREATE OR ALTER`, `CLONE`, `LIKE`, `SWAP WITH`, `UNDROP`;
- write advanced DML: `INSERT OVERWRITE`, multi-table `INSERT`, `UPDATE … FROM`, `DELETE … USING`, `MERGE` with delete branches;
- unload data with `COPY INTO <location>`;
- control transactions and understand locking;
- query the past with Time Travel and the `CHANGES` clause;
- query metadata through `SHOW`, `INFORMATION_SCHEMA`, `ACCOUNT_USAGE` and the flow operator `->>`;
- use Snowflake-specific query features: `QUALIFY`, `GROUP BY ALL`, `ROLLUP/CUBE`, `EXCLUDE/RENAME`, `ILIKE ANY`, `SAMPLE`, `ASOF JOIN`, `LATERAL`, `FLATTEN`, `PIVOT/UNPIVOT`, recursive CTEs, `CONNECT BY`, `GENERATOR`;
- query semi-structured data fluently;
- write safe dynamic SQL with `EXECUTE IMMEDIATE`, bind variables and `IDENTIFIER()`.

Prerequisites: Chapter 3 (portable SQL), Chapters 5–7 (architecture, objects, types).

Where this leads: Chapter 9 (modelling patterns), Chapter 10 (scripting), Chapter 12 (COPY INTO in depth), Chapters 14–15 (streams, tasks, dynamic tables), Chapter 17 (performance).

8.1 How to use this chapter

Chapter 3 taught portable SQL that runs almost anywhere. This chapter covers what is **specific to Snowflake** — or behaves differently there. Treat it as both a tutorial (read it in order once) and a reference (return to sections when needed).

Execution environment for every example: a Snowsight worksheet or Workspace SQL file, role `lab_role`, warehouse `lab_wh`, database `bytemart_lab`, schema `sales` (created in Lab 4.1). The statements were checked against Snowflake's SQL reference at the research cutoff but **not executed on Snowflake**. Where an "Expected output" is shown, it was derived by running an equivalent query on the same sample data in DuckDB; Snowflake's display formatting (for example, trailing decimal zeros) may differ slightly.

Setting up the sample data in Snowflake

Run this once. It recreates Chapter 3's ByteMart tables with Snowflake types.

```
USE ROLE lab_role;
USE WAREHOUSE lab_wh;
CREATE SCHEMA IF NOT EXISTS bytemart_lab.sales;
USE SCHEMA bytemart_lab.sales;

CREATE OR REPLACE TABLE customers (
    customer_id NUMBER NOT NULL, name VARCHAR(100) NOT NULL, email VARCHAR(200) NOT NULL,
    city VARCHAR(60), country VARCHAR(2) NOT NULL, signup_date DATE NOT NULL,
    loyalty_tier VARCHAR(10), referred_by NUMBER,
    CONSTRAINT pk_customers PRIMARY KEY (customer_id));

CREATE OR REPLACE TABLE products (
    product_id NUMBER NOT NULL, sku VARCHAR(30) NOT NULL, product_name VARCHAR(100) NOT NULL,
    category VARCHAR(40) NOT NULL, unit_price NUMBER(10,2) NOT NULL,
    CONSTRAINT pk_products PRIMARY KEY (product_id));

CREATE OR REPLACE TABLE orders (
    order_id NUMBER NOT NULL, customer_id NUMBER NOT NULL, order_ts TIMESTAMP_NTZ NOT NULL,
    status VARCHAR(12) NOT NULL, channel VARCHAR(6) NOT NULL,
    CONSTRAINT pk_orders PRIMARY KEY (order_id));

CREATE OR REPLACE TABLE order_lines (
    order_id NUMBER NOT NULL, line_no NUMBER NOT NULL, product_id NUMBER NOT NULL,
    quantity NUMBER NOT NULL, unit_price NUMBER(10,2) NOT NULL,
    CONSTRAINT pk_order_lines PRIMARY KEY (order_id, line_no),
    CONSTRAINT chk_qty CHECK (quantity > 0));

INSERT INTO customers VALUES
 (1,'Aarav Sharma','aarav@example.com','Bengaluru','IN','2024-01-15','GOLD',NULL),
 (2,'Priya Nair','priya@example.com','Kochi','IN','2024-03-02','SILVER',1),
 (3,'Oliver Smith','oliver@example.co.uk','London','UK','2024-05-20',NULL,NULL),
 (4,'Amelia Jones','amelia@example.co.uk','Manchester','UK','2025-01-11','GOLD',3),
 (5,'Ethan Brown','ethan@example.com','Austin','US','2025-02-14','BRONZE',NULL),
 (6,'Diya Patel','diya@example.com',NULL,'IN','2025-06-30','BRONZE',1),
 (7,'Noah Wilson','noah@example.com','Seattle','US','2025-09-05',NULL,NULL),
 (8,'Kavya Reddy','kavya@example.com','Hyderabad','IN','2026-01-20','SILVER',2);

INSERT INTO products VALUES
 (101,'CHG-USBC-65W','USB-C Charger 65W','Electronics',899.00),
 (102,'HP-NC-700','Noise-Cancelling Headphones','Electronics',12999.00),
 (103,'KB-MECH-87','Mechanical Keyboard','Electronics',4499.00),
 (104,'MUG-STEEL','Insulated Steel Mug','Home',749.00),
 (105,'LAMP-LED','LED Desk Lamp','Home',1899.00),
 (106,'CHAIR-ERG','Ergonomic Chair','Furniture',15999.00),
 (107,'CABLE-HDMI','HDMI Cable 2m','Electronics',399.00);

INSERT INTO orders VALUES
 (1001,1,'2026-01-05 10:15:00','DELIVERED','web'), (1002,2,'2026-01-07
19:42:00','DELIVERED','app'),
 (1003,1,'2026-01-20 08:05:00','DELIVERED','app'), (1004,3,'2026-01-28
21:10:00','CANCELLED','web'),
 (1005,4,'2026-02-03 12:00:00','DELIVERED','web'), (1006,5,'2026-02-11 16:30:00','SHIPPED','w
eb'),
 (1007,2,'2026-02-14 09:20:00','DELIVERED','app'), (1008,6,'2026-02-20
23:55:00','DELIVERED','app'),
 (1009,1,'2026-03-01 13:45:00','SHIPPED','web'),   (1010,7,'2026-03-03 07:10:00','PENDING','a
pp'),
```

```
 (1011,4,'2026-03-08 18:25:00','SHIPPED','app'),   (1012,3,'2026-03-09
11:00:00','DELIVERED','web');

INSERT INTO order_lines VALUES
 (1001,1,101,2,899.00),(1001,2,104,1,749.00),(1002,1,102,1,12999.00),
 (1003,1,103,1,4499.00),(1003,2,101,1,899.00),(1004,1,106,1,15999.00),
 (1005,1,105,2,1899.00),(1005,2,104,2,749.00),(1006,1,102,1,12999.00),(1006,2,101,1,899.00),
 (1007,1,104,4,749.00),(1008,1,106,1,15999.00),(1009,1,105,1,1899.00),
 (1010,1,103,1,4499.00),(1011,1,102,1,12499.00),(1012,1,101,3,899.00),(1012,2,105,1,1899.00);

CREATE OR REPLACE VIEW order_revenue AS
SELECT o.order_id, o.customer_id, o.order_ts, o.status, o.channel,
       SUM(l.quantity * l.unit_price) AS revenue
FROM orders o JOIN order_lines l ON l.order_id = o.order_id
GROUP BY ALL;
```

Unlike DuckDB, Snowflake accepts string literals such as `'2026-01-05 10:15:00'` for date and timestamp columns through implicit conversion, and does not enforce the self-referencing foreign key, so all customers load in one statement.

8.2 Statement categories and their behaviour

Category	Meaning	Examples	Transaction behaviour in Snowflake
DDL — Data Definition	Create and change objects	`CREATE`, `ALTER`, `DROP`, `UNDROP`, `TRUNCATE`*, `COMMENT`	Each DDL statement runs in its own transaction and **commits any open transaction first**
DML — Data Manipulation	Change table data	`INSERT`, `UPDATE`, `DELETE`, `MERGE`, `COPY INTO <table>`	Autocommit by default; can be grouped in explicit transactions
DQL — Data Query	Read data	`SELECT`, `WITH … SELECT`	Read committed snapshot at statement start
DCL — Data Control	Manage access	`GRANT`, `REVOKE`	DDL-like
TCL — Transaction Control	Manage transactions	`BEGIN`, `COMMIT`, `ROLLBACK`	—
Other	Session, metadata, programming	`USE`, `SET`, `SHOW`, `DESCRIBE`, `CALL`, `EXECUTE IMMEDIATE`, `EXPLAIN`	`SHOW`/`DESCRIBE` read metadata; `CALL` depends on the procedure

* `TRUNCATE TABLE` is classed as DML in some references; in Snowflake it removes all rows but keeps the table's structure and grants, deletes load metadata (so files can be reloaded), and its effect remains recoverable through Time Travel.

8.3 DDL in depth

Creating objects safely

Form	Behaviour	Use when
`CREATE TABLE t …`	Fails if `t` exists	One-off creation

Form	Behaviour	Use when
`CREATE TABLE IF NOT EXISTS t …`	Does nothing if `t` exists — *even if its definition differs*	Idempotent bootstrap scripts
`CREATE OR REPLACE TABLE t …`	Drops and recreates atomically — **data lost**, old version kept in Time Travel	Rebuilding derived tables; never for tables holding unique data
`CREATE OR ALTER TABLE t …`	Converges the existing table to the stated definition (adds/drops columns, changes properties) while preserving data where possible	Declarative deployments
`CREATE TABLE t CLONE src`	Zero-copy clone: new table sharing the source's micro-partitions until either is modified	Dev/test copies, backups before risky changes
`CREATE TABLE t LIKE src`	Empty table with the same column definitions	Staging tables
`CREATE TABLE t AS SELECT …` (CTAS)	Table filled from a query	Materialising results
`… COPY GRANTS`	Keep privileges when replacing (for supported objects)	Replacing views and tables in production

```sql
-- Declarative deployment: re-running is safe; adding a column later just edits this text
CREATE OR ALTER TABLE bytemart_lab.sales.returns (
    return_id     NUMBER        NOT NULL,
    order_id      NUMBER        NOT NULL,
    reason_code   VARCHAR(20),
    refund_amount NUMBER(12,2),
    created_at    TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Zero-copy clone before a risky change
CREATE TABLE bytemart_lab.sales.orders_backup_20260310 CLONE bytemart_lab.sales.orders;

-- Replace a view without losing grants
CREATE OR REPLACE VIEW bytemart_lab.sales.v_delivered COPY GRANTS AS
SELECT * FROM bytemart_lab.sales.orders WHERE status = 'DELIVERED';
```

CREATE OR ALTER VERSUS CREATE OR REPLACE

`CREATE OR REPLACE TABLE` rewrites history: the new table has none of the old rows, streams on the old table break, and only Time Travel can bring data back. `CREATE OR ALTER TABLE` evolves the existing table. Check the documentation for which object types and changes `CREATE OR ALTER` supports before relying on it in pipelines.

Altering tables

```sql
ALTER TABLE orders ADD COLUMN promo_code VARCHAR(20);
ALTER TABLE orders RENAME COLUMN promo_code TO coupon_code;
ALTER TABLE orders ALTER COLUMN coupon_code SET DATA TYPE VARCHAR(40);  -- widening is allowed
ALTER TABLE orders ALTER COLUMN coupon_code COMMENT 'Applied coupon, upper-case';
ALTER TABLE orders DROP COLUMN coupon_code;
ALTER TABLE orders SET DATA_RETENTION_TIME_IN_DAYS = 7;
ALTER TABLE orders CLUSTER BY (order_ts::DATE);          -- Chapter 17
ALTER TABLE orders DROP CLUSTERING KEY;
```

Snowflake allows increasing a `VARCHAR` length or a `NUMBER` precision in place, but not decreasing them or changing the type family (for example, `VARCHAR` to `NUMBER`). For such changes, add a new column, populate it, and swap names.

Swapping tables atomically

`ALTER TABLE a SWAP WITH b` exchanges two tables' names (and therefore their contents and metadata) in one atomic metadata operation. It is the classic **blue/green rebuild** pattern:

```
CREATE OR REPLACE TABLE gold_daily_sales_new AS SELECT /* expensive rebuild */ 1 AS x;
-- validate gold_daily_sales_new here
ALTER TABLE gold_daily_sales SWAP WITH gold_daily_sales_new;   -- consumers switch instantly
DROP TABLE gold_daily_sales_new;                               -- old version, now under the
new name
```

Dropping and undropping

```
DROP TABLE orders_backup_20260310;
UNDROP TABLE orders_backup_20260310;     -- within the retention period
SHOW TABLES HISTORY LIKE 'ORDERS%';      -- includes dropped tables with dropped_on
timestamps
```

`UNDROP` works for tables, schemas and databases (and several other object types) while they are within their Time Travel retention period. If a new object with the same name exists, rename it first.

8.4 DML in depth

INSERT variants

```
-- Column list: always name columns in production code
INSERT INTO returns (return_id, order_id, reason_code, refund_amount)
VALUES (1, 1004, 'CANCELLED', 15999.00);

-- INSERT … SELECT
INSERT INTO returns (return_id, order_id, reason_code, refund_amount)
SELECT 1000 + ROW_NUMBER() OVER (ORDER BY order_id), order_id, 'AUDIT', 0
FROM orders WHERE status = 'PENDING';

-- INSERT OVERWRITE: truncate and insert in one atomic statement
INSERT OVERWRITE INTO returns (return_id, order_id, reason_code, refund_amount)
SELECT 1, 1004, 'CANCELLED', 15999.00;
```

`INSERT OVERWRITE` is an atomic replace of a table's contents — useful for small, fully recomputed tables (and the only DML that interactive tables accept).

Multi-table INSERT

One source query can feed several tables in a single pass:

```
CREATE OR REPLACE TABLE orders_web LIKE orders;
CREATE OR REPLACE TABLE orders_app LIKE orders;
```

```
CREATE OR REPLACE TABLE orders_problem LIKE orders;

INSERT FIRST
  WHEN status IN ('CANCELLED','PENDING') THEN INTO orders_problem
  WHEN channel = 'web'                   THEN INTO orders_web
  ELSE                                        INTO orders_app
SELECT * FROM orders;
```

- `INSERT FIRST` sends each row to the **first** matching `WHEN` branch.
- `INSERT ALL` sends each row to **every** matching branch.

Expected: `orders_problem` 2 rows (1004, 1010), `orders_web` 5 rows, `orders_app` 5 rows.

UPDATE … FROM and DELETE … USING

```
-- Re-price open order lines from the catalogue (join-driven update)
UPDATE order_lines l
SET    unit_price = p.unit_price
FROM   products p, orders o
WHERE  l.product_id = p.product_id
  AND  o.order_id   = l.order_id
  AND  o.status     = 'PENDING';

-- Delete lines belonging to cancelled orders
DELETE FROM order_lines l
USING orders o
WHERE o.order_id = l.order_id
  AND o.status   = 'CANCELLED';
```

NON-DETERMINISTIC JOINS IN UPDATE

If the `FROM` clause matches a target row to *several* source rows with different values, the result is non-deterministic. Snowflake's `ERROR_ON_NONDETERMINISTIC_UPDATE` parameter (default FALSE) can turn this into an error. Always ensure the join is one-to-one, typically by de-duplicating the source.

MERGE with all branches

```
MERGE INTO products t
USING (
    SELECT * FROM VALUES
      (104, 'MUG-STEEL',  'Insulated Steel Mug', 'Home',        699.00, 'U'),
      (107, 'CABLE-HDMI', 'HDMI Cable 2m',       'Electronics', 399.00, 'D'),
      (108, 'SSD-1TB',    'Portable SSD 1TB',    'Electronics', 7499.00,'U')
      AS s(product_id, sku, product_name, category, unit_price, op)
) s
ON t.product_id = s.product_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED AND t.unit_price <> s.unit_price THEN UPDATE SET unit_price = s.unit_price
WHEN NOT MATCHED AND s.op <> 'D' THEN
     INSERT (product_id, sku, product_name, category, unit_price)
     VALUES (s.product_id, s.sku, s.product_name, s.category, s.unit_price);
```

Line by line: the source uses the `VALUES … AS s(cols)` form; `op` is a change-type flag as delivered by many CDC feeds. Matched rows flagged `D` are deleted; matched rows with a price change are updated; unmatched non-delete rows are inserted. Multiple `WHEN MATCHED` clauses are evaluated in order; the first whose condition is true applies. **Expected:** product 104 price becomes 699.00, product 107 is deleted, product 108 is inserted.

The MERGE output (in Snowsight) reports `number of rows inserted`, `number of rows updated` and `number of rows deleted` — record these in pipeline logs.

Unloading with COPY INTO <location>

`COPY INTO <table>` loads files (Chapter 12); `COPY INTO <location>` **unloads** query results to a stage:

```
CREATE STAGE IF NOT EXISTS bytemart_lab.sales.exports;

COPY INTO @bytemart_lab.sales.exports/orders/orders_
FROM (SELECT * FROM orders WHERE order_ts >= '2026-03-01')
FILE_FORMAT = (TYPE = PARQUET)
HEADER = TRUE
OVERWRITE = TRUE
MAX_FILE_SIZE = 104857600;     -- ~100 MB per file

LIST @bytemart_lab.sales.exports/orders/;
```

Snowflake writes one or more files in parallel (file names get suffixes). `SINGLE = TRUE` forces one file (slower; size-limited). `PARTITION BY <expression>` writes files into sub-paths by value — useful for lake layouts. To download from an internal stage to your machine, use `GET` from the Snowflake CLI or SnowSQL (Chapter 12).

8.5 Transactions and locking

```
BEGIN TRANSACTION;
  UPDATE orders SET status = 'DELIVERED' WHERE order_id = 1009;
  INSERT INTO returns (return_id, order_id, reason_code, refund_amount)
       VALUES (2, 1009, 'DAMAGED', 1899.00);
COMMIT;
```

Topic	Snowflake behaviour
Default mode	`AUTOCOMMIT = TRUE`: each DML statement commits on success
Explicit transactions	`BEGIN [TRANSACTION]` … `COMMIT` / `ROLLBACK`
Isolation	READ COMMITTED for tables; each statement sees data committed before it began
DDL inside a transaction	Commits the open transaction first, then runs in its own transaction — keep DDL out of transactions you might roll back
Locks	`UPDATE`, `DELETE`, `MERGE` take locks that generally serialise with each other on the same table; `INSERT`/`COPY` usually run concurrently
Waiting	Blocked statements wait up to `LOCK_TIMEOUT` (seconds); deadlocks are detected and one transaction is aborted
Inspecting	`SHOW TRANSACTIONS`, `SHOW LOCKS`; abort with `SELECT SYSTEM$ABORT_TRANSACTION(<id>)`
Hybrid tables	Row-level locking

Topic	Snowflake behaviour
Failed statement in a transaction	The failed statement is rolled back; the transaction stays open until you `COMMIT` or `ROLLBACK`

ORPHANED TRANSACTIONS

A session that runs `BEGIN`, then disconnects without committing, can leave a transaction holding locks until it times out. Pipelines should use explicit `COMMIT` / `ROLLBACK` in error handlers (Chapter 10) and monitoring for long-running transactions.

8.6 DCL in one page

```
GRANT USAGE ON DATABASE bytemart_lab TO ROLE analyst_lab;
GRANT USAGE ON SCHEMA bytemart_lab.sales TO ROLE analyst_lab;
GRANT SELECT ON ALL TABLES IN SCHEMA bytemart_lab.sales TO ROLE analyst_lab;
GRANT SELECT ON FUTURE TABLES IN SCHEMA bytemart_lab.sales TO ROLE analyst_lab;
REVOKE SELECT ON TABLE bytemart_lab.sales.customers FROM ROLE analyst_lab;
SHOW GRANTS TO ROLE analyst_lab;
SHOW GRANTS ON TABLE bytemart_lab.sales.orders;
```

Chapter 20 covers role design, ownership, managed access and future grants in depth.

8.7 Snowflake query features

SELECT * with EXCLUDE and RENAME

```
SELECT * EXCLUDE (email, referred_by) FROM customers;
SELECT * RENAME (loyalty_tier AS tier) FROM customers;
SELECT * EXCLUDE (email) RENAME (loyalty_tier AS tier) FROM customers;
```

Useful in exploration and when exposing most of a table's columns while hiding a few — but remember that `*` still changes when columns are added.

Pattern matching against lists

```
SELECT name, email FROM customers WHERE email ILIKE ANY ('%.co.uk', '%@example.org');
SELECT sku FROM products WHERE sku LIKE ALL ('%-%', '%E%');
SELECT name FROM customers WHERE RLIKE(email, '^[a-z]+@example\\.com$');
```

GROUP BY ALL, ROLLUP, CUBE and GROUPING SETS

`GROUP BY ALL` groups by every non-aggregate expression in the `SELECT` list. `ROLLUP` adds subtotal rows; `CUBE` adds all combinations; `GROUPING SETS` lists the exact combinations.

```
SELECT c.country,
       o.channel,
       SUM(r.revenue)            AS revenue,
       GROUPING(c.country)       AS is_country_total,
```

```
       GROUPING(o.channel)       AS is_channel_total
FROM order_revenue r
JOIN customers c ON c.customer_id = r.customer_id
JOIN orders    o ON o.order_id    = r.order_id
WHERE r.status <> 'CANCELLED'
GROUP BY ROLLUP (c.country, o.channel)
ORDER BY c.country NULLS LAST, o.channel NULLS LAST;
```

Expected output (derived from the sample data):

country	channel	revenue
IN	app	37392.00
IN	web	4446.00
IN	NULL	41838.00
UK	app	12499.00
UK	web	9892.00
UK	NULL	22391.00
US	app	4499.00
US	web	13898.00
US	NULL	18397.00
NULL	NULL	82626.00

Rows with NULL in a grouping column are subtotals; `GROUPING()` returns 1 for them, which distinguishes a subtotal NULL from a genuine NULL value in the data.

QUALIFY, TOP and LIMIT ... OFFSET

```
SELECT TOP 3 order_id, revenue FROM order_revenue ORDER BY revenue DESC;
SELECT order_id, revenue FROM order_revenue ORDER BY revenue DESC LIMIT 3 OFFSET 3;  -- page
2
SELECT * FROM order_revenue
QUALIFY RANK() OVER (PARTITION BY channel ORDER BY revenue DESC) <= 2;
```

Sampling

```
SELECT * FROM snowflake_sample_data.tpch_sf1.lineitem SAMPLE (1);          -- ~1% of rows
SELECT * FROM snowflake_sample_data.tpch_sf1.lineitem SAMPLE (1000 ROWS);  -- fixed count
SELECT * FROM snowflake_sample_data.tpch_sf1.lineitem SAMPLE SYSTEM (1) SEED (42);  -- block
sampling, repeatable
```

`SAMPLE` / `TABLESAMPLE` supports row (Bernoulli) and block (`SYSTEM`) sampling; `SEED` makes block sampling repeatable. Sampling is ideal for exploring huge tables and for building development datasets.

LATERAL joins

A **lateral** subquery or table function can reference columns from tables earlier in the `FROM` clause — it is evaluated once per row of the preceding table.

```
SELECT c.name, recent.order_id, recent.order_ts
FROM customers c,
     LATERAL (SELECT o.order_id, o.order_ts
              FROM orders o
              WHERE o.customer_id = c.customer_id
              ORDER BY o.order_ts DESC
              LIMIT 1) recent;
```

ASOF JOIN

An **ASOF join** matches each row on the left with the *closest preceding (or following)* row on the right — essential for time-series data such as "the exchange rate in force when each order was placed".

```
CREATE OR REPLACE TABLE fx_rates (currency VARCHAR(3), rate_ts TIMESTAMP_NTZ, inr_per_unit
NUMBER(10,4));
INSERT INTO fx_rates VALUES
 ('GBP','2026-01-01 00:00:00',105.2000), ('GBP','2026-02-01 00:00:00',106.1000),
 ('GBP','2026-03-01 00:00:00',104.8000), ('USD','2026-01-01 00:00:00',83.1000),
 ('USD','2026-02-15 00:00:00',83.6000);

SELECT o.order_id, o.order_ts, c.country, f.rate_ts, f.inr_per_unit
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
ASOF JOIN fx_rates f
     MATCH_CONDITION (o.order_ts >= f.rate_ts)
     ON f.currency = DECODE(c.country, 'UK', 'GBP', 'US', 'USD')
WHERE c.country IN ('UK', 'US')
ORDER BY o.order_id;
```

`MATCH_CONDITION (o.order_ts >= f.rate_ts)` picks, for each order, the rate with the latest `rate_ts` not after the order; the `ON` clause restricts matches to the same currency. Without ASOF, this requires a correlated subquery or a window over a range join — far more expensive. **Expected:** order 1006 (US, 11 Feb) matches the USD rate from 1 Jan (83.1); order 1010 (US, 3 Mar) matches the rate from 15 Feb (83.6).

Recursive CTEs and CONNECT BY

Hierarchies — referral trees, org charts, bills of materials — need recursion.

```
WITH RECURSIVE referral_tree (customer_id, name, referred_by, lvl, path) AS (
    SELECT customer_id, name, referred_by, 0, name::VARCHAR
    FROM customers
    WHERE referred_by IS NULL                          -- anchor: roots
    UNION ALL
    SELECT c.customer_id, c.name, c.referred_by, t.lvl + 1, t.path || ' > ' || c.name
    FROM customers c
    JOIN referral_tree t ON c.referred_by = t.customer_id   -- recursive step
)
SELECT lvl, name, path FROM referral_tree ORDER BY path;
```

Expected output:

lvl	name	path
0	Aarav Sharma	Aarav Sharma

lvl	name	path
1	Diya Patel	Aarav Sharma > Diya Patel
1	Priya Nair	Aarav Sharma > Priya Nair
2	Kavya Reddy	Aarav Sharma > Priya Nair > Kavya Reddy
0	Ethan Brown	Ethan Brown
0	Noah Wilson	Noah Wilson
0	Oliver Smith	Oliver Smith
1	Amelia Jones	Oliver Smith > Amelia Jones

A recursive CTE has an **anchor** member (the starting rows) and a **recursive** member that joins back to the CTE; recursion stops when the recursive member returns no rows. Guard against cycles in real data (for example, by tracking the path and stopping when a node repeats); Snowflake also enforces a recursion limit, described in the WITH clause reference. Snowflake additionally supports Oracle-style `START WITH … CONNECT BY PRIOR …`, convenient when migrating Oracle code (Chapter 35):

```
SELECT LEVEL, name, SYS_CONNECT_BY_PATH(name, ' > ') AS path
FROM customers
START WITH referred_by IS NULL
CONNECT BY referred_by = PRIOR customer_id;
```

Generating rows: GENERATOR and calendars

```
-- A 2026 calendar: one row per day
SELECT DATEADD(day, SEQ4(), '2026-01-01'::DATE) AS cal_date
FROM TABLE(GENERATOR(ROWCOUNT => 365));
```

`GENERATOR` produces rows; `SEQ4()` numbers them (gap-free here because the generator is not distributed across joins, but `SEQ` functions in general do not guarantee gap-free sequences — use `ROW_NUMBER()` when you need that). Calendar tables like this solve the "missing months" problem from Chapter 3.

PIVOT and UNPIVOT

```
SELECT *
FROM (SELECT TO_CHAR(order_ts, 'YYYY-MM') AS month, channel, revenue
      FROM order_revenue WHERE status <> 'CANCELLED')
PIVOT (SUM(revenue) FOR channel IN ('app', 'web')) AS p
ORDER BY month;
```

Expected output:

MONTH	'app'	'web'
2026-01	18397.00	2547.00
2026-02	18995.00	19194.00
2026-03	16998.00	6495.00

Note the pivoted column names include quotes (`'app'`); alias them with `AS p (month, app_revenue, web_revenue)` for clean names. Snowflake also supports **dynamic pivots** with `IN (ANY ORDER BY channel)` or a subquery, so new channel values become columns automatically — convenient for exploration, risky for downstream code that expects fixed columns.

`UNPIVOT` turns columns back into rows:

```
SELECT month, channel, revenue
FROM monthly_channel_revenue           -- a table shaped like the pivot output
UNPIVOT (revenue FOR channel IN (app_revenue, web_revenue));
```

By default UNPIVOT drops rows where the value is NULL; `UNPIVOT INCLUDE NULLS` keeps them.

LISTAGG, ARRAY_AGG and OBJECT_AGG

```
SELECT c.name,
       LISTAGG(p.sku, ', ') WITHIN GROUP (ORDER BY p.sku)          AS skus,
       ARRAY_AGG(DISTINCT p.category) WITHIN GROUP (ORDER BY p.category) AS categories
FROM customers c
JOIN orders o      ON o.customer_id = c.customer_id
JOIN order_lines l ON l.order_id    = o.order_id
JOIN products p    ON p.product_id  = l.product_id
GROUP BY c.name
ORDER BY c.name;
```

`LISTAGG` concatenates strings; `ARRAY_AGG` builds an ARRAY; `OBJECT_AGG(key, value)` builds an OBJECT. For Aarav, the expected `skus` value is `CHG-USBC-65W, CHG-USBC-65W, KB-MECH-87, LAMP-LED, MUG-STEEL` (duplicates kept; use `LISTAGG(DISTINCT …)` to remove them).

The flow (pipe) operator ->>

The **flow operator** chains statements so each can consume the previous result as `$1` in its `FROM` clause — a clean replacement for `RESULT_SCAN(LAST_QUERY_ID())`:

```
SHOW WAREHOUSES
  ->> SELECT "name", "state", "size", "auto_suspend" FROM $1 WHERE "state" = 'STARTED';
```

The chain runs as one unit; only the last statement's result is returned; `$n` is valid only in a `FROM` clause and counts back from the current statement. Column names from `SHOW` are lowercase and must be double-quoted. Client drivers do not support multi-statement execution with this operator.

MATCH_RECOGNIZE (preview of a powerful tool)

`MATCH_RECOGNIZE` finds sequences of rows that match a pattern — "a page view, then add-to-cart, then no checkout within 30 minutes". It is used for funnel analysis, fraud patterns and sessionization. Chapter 9 solves sessionization with window functions; the `MATCH_RECOGNIZE` reference in Further reading shows the pattern syntax.

8.8 Semi-structured data queries

Chapter 7 introduced `VARIANT`. This section covers the operators you will use daily.

Reference table

Task	Syntax
Parse text to VARIANT	`PARSE_JSON(s)`, `TRY_PARSE_JSON(s)` (NULL on invalid JSON)
Access a key	`v:key`, `v['key']`, `GET(v, 'key')`
Nested path	`v:a.b.c`, `GET_PATH(v, 'a.b.c')`
Array element	`v:items[0]`
Key with special characters	`v:"order-id"`
Cast	`v:qty::NUMBER`, `TO_NUMBER(v:qty)`
Type of a value	`TYPEOF(v:x)`
Explode an array/object into rows	`LATERAL FLATTEN(input => v:items)`
Build an object / array	`OBJECT_CONSTRUCT('k', v, …)`, `ARRAY_CONSTRUCT(…)`, `OBJECT_CONSTRUCT(*)` (whole row)
Aggregate into an array / object	`ARRAY_AGG(x)`, `OBJECT_AGG(k, v)`
Size	`ARRAY_SIZE(a)`
Test membership	`ARRAY_CONTAINS(value::VARIANT, array)`
Keys of an object	`OBJECT_KEYS(o)`

FLATTEN

`FLATTEN` is a table function that turns each element of an array (or each key of an object) into a row. It returns columns `SEQ`, `KEY`, `PATH`, `INDEX`, `VALUE` and `THIS`.

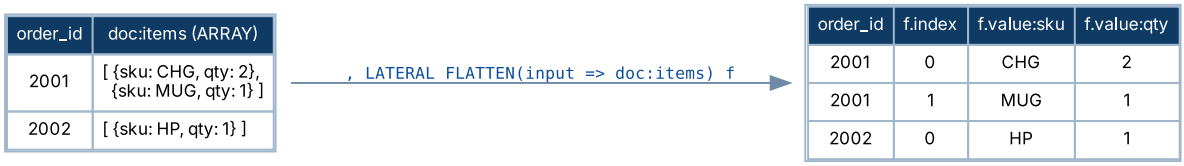


Figure 8.1 — LATERAL FLATTEN turns each array element into its own row, alongside the columns of the parent row.

```
CREATE OR REPLACE TABLE order_docs (order_id NUMBER, doc VARIANT);
INSERT INTO order_docs
SELECT column1, PARSE_JSON(column2) FROM VALUES
 (2001, '{"customer":{"id":1,"tier":"GOLD"},"items":[{"sku":"CHG-USBC-65W","qty":2},
{"sku":"MUG-STEEL","qty":1}],"tags":["gift","express"]}'),
 (2002, '{"customer":{"id":5},"items":[{"sku":"HP-NC-700","qty":1}],"tags":[]}');
```

```sql
SELECT d.order_id,
       d.doc:customer.id::NUMBER   AS customer_id,
       f.index                     AS line_index,
       f.value:sku::STRING         AS sku,
       f.value:qty::NUMBER         AS qty
FROM order_docs d,
     LATERAL FLATTEN(input => d.doc:items) f
ORDER BY d.order_id, f.index;
```

Expected output: three rows — (2001, 1, 0, CHG-USBC-65W, 2), (2001, 1, 1, MUG-STEEL, 1), (2002, 5, 0, HP-NC-700, 1).

Important `FLATTEN` arguments:

- `OUTER => TRUE` keeps parent rows whose array is empty or missing (one row with NULLs) — without it, order 2002 would vanish if you flattened `tags`, because its tags array is empty.
- `RECURSIVE => TRUE` descends into nested structures (useful for discovering all paths in unknown JSON).
- `PATH => 'a.b'` flattens a nested element.
- `MODE => 'OBJECT' | 'ARRAY' | 'BOTH'`.

```sql
-- Discover every path present in the documents (schema discovery)
SELECT DISTINCT f.path, TYPEOF(f.value) AS value_type
FROM order_docs d,
     LATERAL FLATTEN(input => d.doc, RECURSIVE => TRUE) f
ORDER BY f.path;
```

Building JSON

```sql
SELECT OBJECT_CONSTRUCT(
         'order_id', o.order_id,
         'status',   o.status,
         'lines',    ARRAY_AGG(OBJECT_CONSTRUCT('sku', p.sku, 'qty', l.quantity))
                       WITHIN GROUP (ORDER BY l.line_no)
       ) AS order_json
FROM orders o
JOIN order_lines l ON l.order_id = o.order_id
JOIN products p    ON p.product_id = l.product_id
WHERE o.order_id = 1005
GROUP BY o.order_id, o.status;
```

Expected: `{"lines":[{"qty":2,"sku":"LAMP-LED"},{"qty":2,"sku":"MUG-STEEL"}],"order_id":1005,"status":"DELIVERED"}` (key order in the displayed JSON may differ; do not rely on it) — note that `OBJECT_CONSTRUCT` omits keys whose value is SQL NULL (use `OBJECT_CONSTRUCT_KEEP_NULL` to keep them).

8.9 Time Travel

Concept

Time Travel lets you query, clone or restore data **as it existed at an earlier point** within the object's retention period (`DATA_RETENTION_TIME_IN_DAYS`: up to 1 day on Standard Edition, up to 90 days on Enterprise and higher for permanent objects; 0 or 1 day for transient and temporary tables). It works because modified micro-partitions are retained (Chapter 5).

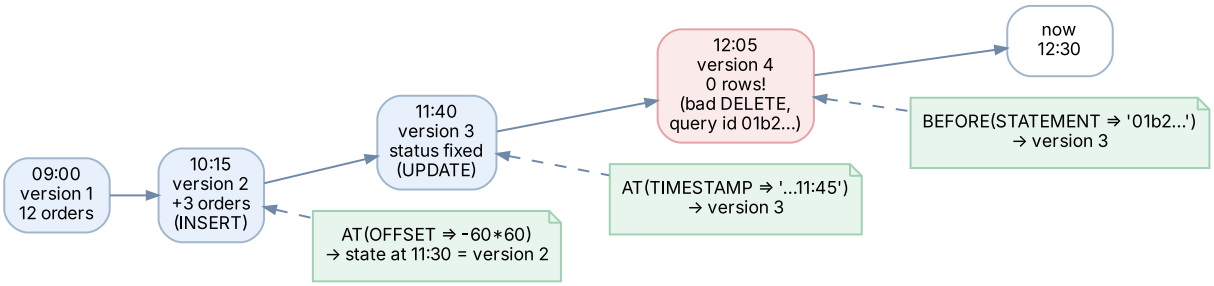


Figure 8.2 — Each DML commit creates a new table version. Time Travel clauses select an earlier version by timestamp, offset or statement ID.

Syntax

```
-- At a timestamp
SELECT * FROM orders AT(TIMESTAMP => '2026-03-10 11:45:00'::TIMESTAMP_LTZ);

-- Relative offset in seconds (one hour ago)
SELECT * FROM orders AT(OFFSET => -60*60);

-- Immediately before a specific statement (e.g. the bad DELETE)
SELECT * FROM orders BEFORE(STATEMENT => '<query_id_of_bad_delete>');

-- Restore by cloning the earlier version, then swapping
CREATE TABLE orders_restored CLONE orders BEFORE(STATEMENT => '<query_id_of_bad_delete>');
ALTER TABLE orders SWAP WITH orders_restored;

-- Restore dropped objects
UNDROP TABLE orders;
UNDROP SCHEMA sales;
UNDROP DATABASE bytemart_lab;
```

Finding the query ID: in Snowsight query history, or with `SELECT query_id, query_text, start_time FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY()) WHERE query_text ILIKE 'delete%orders%' ORDER BY start_time DESC;`.

The CHANGES clause

For tables with change tracking enabled (`ALTER TABLE orders SET CHANGE_TRACKING = TRUE`, or implicitly when a stream exists), the `CHANGES` clause returns row-level changes between two points **without consuming anything** — a read-only alternative to streams (Chapter 14):

```sql
ALTER TABLE orders SET CHANGE_TRACKING = TRUE;
-- … some DML happens …
SELECT *
FROM orders
  CHANGES (INFORMATION => DEFAULT)
  AT (OFFSET => -60*60)
  END (TIMESTAMP => CURRENT_TIMESTAMP());
```

`INFORMATION => APPEND_ONLY` returns only inserts. The output includes `METADATA$ACTION`, `METADATA$ISUPDATE` and `METADATA$ROW_ID`, exactly like a stream.

Retention, Fail-safe and cost

Setting	Where	Notes
`DATA_RETENTION_TIME_IN_DAYS`	Account, database, schema, table	Child objects inherit unless overridden
`MIN_DATA_RETENTION_TIME_IN_DAYS`	Account	Floor that object-level settings cannot go below — protects against someone setting 0
Fail-safe	Automatic, permanent tables	7 days after Time Travel ends; recovery only via Snowflake Support; not user-queryable

Longer retention means more historical storage — significant for tables with heavy update/delete churn. Typical choice: 1 day for staging (transient), 7–30 days for business tables, longer for critical audit data.

> **TIME TRAVEL IS NOT A BACKUP STRATEGY BY ITSELF**
>
> It protects against mistakes *within the retention period* in the *same account*. It does not protect against a region outage, a malicious administrator setting retention to zero and dropping objects, or problems noticed after the period ends. Combine it with `MIN_DATA_RETENTION_TIME_IN_DAYS`, replication, backups/snapshots and access controls (Chapter 39).

8.10 Querying metadata

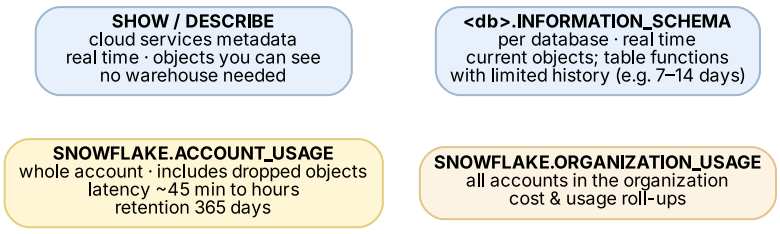


Figure 8.3 — Four sources of metadata, with different scope, latency and history.

SHOW and DESCRIBE

```sql
SHOW TABLES IN SCHEMA bytemart_lab.sales;
SHOW COLUMNS IN TABLE bytemart_lab.sales.orders;
SHOW WAREHOUSES LIKE 'LAB%';
DESCRIBE TABLE bytemart_lab.sales.orders;
```

```
SHOW PARAMETERS LIKE 'TIMEZONE' IN SESSION;
SHOW GRANTS TO ROLE lab_role;
```

`SHOW` reads cloud-services metadata, needs no warehouse, and lists only objects your role can see. Post-process results with `->>` or `RESULT_SCAN`.

INFORMATION_SCHEMA

Every database contains `INFORMATION_SCHEMA`, a set of read-only views and table functions describing that database's objects in real time.

```
-- Tables and row counts in a schema
SELECT table_name, table_type, row_count, bytes, created, last_altered
FROM bytemart_lab.information_schema.tables
WHERE table_schema = 'SALES'
ORDER BY table_name;

-- Columns and types
SELECT column_name, data_type, is_nullable
FROM bytemart_lab.information_schema.columns
WHERE table_schema = 'SALES' AND table_name = 'ORDERS'
ORDER BY ordinal_position;

-- Recent query history for your session's queries (table function, limited history)
SELECT query_id, query_text, total_elapsed_time, rows_produced
FROM TABLE(bytemart_lab.information_schema.query_history_by_session())
ORDER BY start_time DESC LIMIT 10;
```

Note that identifiers are stored in uppercase (`'SALES'`, `'ORDERS'`) in metadata views.

ACCOUNT_USAGE

The shared `SNOWFLAKE` database's `ACCOUNT_USAGE` schema holds account-wide history: queries, logins, warehouse metering, storage, access history, task history, copy history and much more — **including dropped objects**, with **up to about a year of history** and **latency** from roughly 45 minutes to a few hours depending on the view. Access requires privileges granted by an administrator (for example, `IMPORTED PRIVILEGES` on the `SNOWFLAKE` database or specific `SNOWFLAKE` database roles such as `GOVERNANCE_VIEWER` or `USAGE_VIEWER`).

```
-- Most expensive queries by elapsed time in the last 7 days
SELECT query_id, user_name, warehouse_name,
       total_elapsed_time/1000 AS seconds, bytes_scanned, partitions_scanned, partitions_tota
l
FROM snowflake.account_usage.query_history
WHERE start_time >= DATEADD(day, -7, CURRENT_TIMESTAMP())
  AND execution_status = 'SUCCESS'
ORDER BY total_elapsed_time DESC
LIMIT 20;
```

Need	Use
"What exists right now in this database?"	`SHOW`, `INFORMATION_SCHEMA`
"What happened in the last hour, exactly now?"	`INFORMATION_SCHEMA` table functions (`QUERY_HISTORY`, `TASK_HISTORY`, `COPY_HISTORY`…)

Need	Use
"Trends, costs, audits across the account, including dropped objects"	`ACCOUNT_USAGE`
"Across all accounts in the organization"	`ORGANIZATION_USAGE`

Chapters 18 and 19 build dashboards on these views.

8.11 Function catalogue

Snowflake has hundreds of functions. Learn the categories and the most useful members; look up the rest when needed.

Category	Most used	Notes
Conditional	`CASE`, `IFF`, `DECODE`, `COALESCE`, `NVL`, `NVL2`, `NULLIF`, `ZEROIFNULL`, `EQUAL_NULL`, `GREATEST`, `LEAST`	`DECODE(x, a, r1, b, r2, default)` is compact for value mapping
Aggregate	`COUNT`, `COUNT_IF`, `SUM`, `AVG`, `MIN`, `MAX`, `MEDIAN`, `MODE`, `PERCENTILE_CONT`, `STDDEV`, `APPROX_COUNT_DISTINCT`, `APPROX_PERCENTILE`, `ANY_VALUE`, `MIN_BY`, `MAX_BY`, `LISTAGG`, `ARRAY_AGG`, `OBJECT_AGG`	`APPROX_*` functions trade tiny error for large speedups on big data
Window	`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE`, `RATIO_TO_REPORT`, `CONDITIONAL_CHANGE_EVENT`, `CONDITIONAL_TRUE_EVENT`	`LAG/LEAD … IGNORE NULLS`; the last two are perfect for sessionization
String	`CONCAT`, `||`, `SUBSTR`, `SPLIT`, `SPLIT_PART`, `SPLIT_TO_TABLE`, `TRIM`, `REPLACE`, `TRANSLATE`, `INITCAP`, `LPAD`, `REGEXP_SUBSTR`, `REGEXP_REPLACE`, `REGEXP_COUNT`, `RLIKE`, `EDITDISTANCE`, `JAROWINKLER_SIMILARITY`, `SOUNDEX`	Fuzzy-matching functions help with customer de-duplication
Date/time	`CURRENT_DATE`, `CURRENT_TIMESTAMP`, `DATEADD`, `DATEDIFF`, `DATE_TRUNC`, `LAST_DAY`, `DAYNAME`, `TO_CHAR`, `CONVERT_TIMEZONE`, `TIME_SLICE`, `TIMESTAMP_FROM_PARTS`	`DATEDIFF` counts boundaries crossed, not full periods
Conversion	`CAST`, `::`, `TO_NUMBER`, `TO_DATE`, `TO_TIMESTAMP_*`, `TO_VARCHAR`, `TRY_*`	See Chapter 7
Semi-structured	`PARSE_JSON`, `OBJECT_CONSTRUCT`, `ARRAY_*`, `FLATTEN`, `GET_PATH`, `TYPEOF`, `CHECK_JSON`, `STRIP_NULL_VALUE`	
Hash & IDs	`HASH`, `MD5`, `SHA2`, `UUID_STRING`, `SEQ4`	`SHA2` for stable surrogate keys; `HASH` is fast but 64-bit and not cryptographic
Context	`CURRENT_USER`, `CURRENT_ROLE`, `IS_ROLE_IN_SESSION`, `CURRENT_WAREHOUSE`, `CURRENT_DATABASE`, `CURRENT_ACCOUNT`, `LAST_QUERY_ID`, `INVOKER_ROLE`	Used in policies and auditing
System	`SYSTEM$CLUSTERING_INFORMATION`, `SYSTEM$PIPE_STATUS`, `SYSTEM$STREAM_HAS_DATA`, `SYSTEM$TYPEOF`, `SYSTEM$ABORT_TRANSACTION`	
Table functions	`FLATTEN`, `GENERATOR`, `SPLIT_TO_TABLE`, `RESULT_SCAN`, `INFER_SCHEMA`, `INFORMATION_SCHEMA.*_HISTORY`	Used in `FROM TABLE(...)`
Geospatial	`ST_DISTANCE`, `ST_MAKEPOINT`, `ST_CONTAINS`, `H3_*`	
AI	`AI_COMPLETE`, `AI_CLASSIFY`, `AI_EXTRACT`, …	Part VII

Window function defaults worth memorising

- With `ORDER BY` and **no explicit frame**, aggregate window functions use a cumulative frame from the start of the partition to the current row (a **RANGE** frame, so rows with *equal* order keys are included together). For running totals with ties, specify `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` explicitly.
- `LAST_VALUE` with the default frame returns the *current* row's value — almost never what you want. Use `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

```
SELECT order_id, customer_id, order_ts, revenue,
       FIRST_VALUE(order_id) OVER (PARTITION BY customer_id ORDER BY order_ts)       AS first
_order,
       LAST_VALUE(order_id)  OVER (PARTITION BY customer_id ORDER BY order_ts
                                   ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
AS latest_order,
       RATIO_TO_REPORT(revenue) OVER (PARTITION BY customer_id)                     AS share
_of_customer,
       CONDITIONAL_CHANGE_EVENT(channel) OVER (PARTITION BY customer_id ORDER BY order_ts)
AS channel_switches
FROM order_revenue
ORDER BY customer_id, order_ts;
```

`CONDITIONAL_CHANGE_EVENT(channel)` increments each time `channel` differs from the previous row's value — for Aarav (web, app, web) it yields 0, 1, 2.

8.12 Dynamic SQL

Sometimes the SQL text itself must be built at run time — table names from a configuration table, columns discovered from metadata.

Session variables and IDENTIFIER()

```
SET target_table = 'bytemart_lab.sales.orders';
SET min_date = '2026-03-01';

SELECT COUNT(*) FROM IDENTIFIER($target_table) WHERE order_ts >= $min_date;
```

`IDENTIFIER()` treats a string as an object name, safely: it accepts only a valid identifier, not arbitrary SQL. `$var` references a session variable as a **value**.

EXECUTE IMMEDIATE with bind variables

```
EXECUTE IMMEDIATE
  'SELECT COUNT(*) FROM bytemart_lab.sales.orders WHERE status = ? AND channel = ?'
  USING ('DELIVERED', 'app');
```

Inside Snowflake Scripting (Chapter 10), bind variables are written `:name` and `EXECUTE IMMEDIATE … USING (var1, var2)` passes them. **Bind values, never concatenate them**:

SQL INJECTION IN DYNAMIC SQL

Building SQL by concatenating user-provided text — `'… WHERE status = ''' || p_status || ''''` — allows an attacker (or an AI agent following injected instructions, Chapter 22) to change the statement. Use bind variables for values and `IDENTIFIER()` for object names; validate identifiers against an allow-list (for example, `INFORMATION_SCHEMA.TABLES`) before use.

EXECUTE IMMEDIATE FROM a file

```
EXECUTE IMMEDIATE FROM @my_git_repo/branches/main/deploy/create_tables.sql;
```

Runs a SQL script stored on a stage or in a Snowflake Git repository clone — the backbone of SQL-based deployments (Chapter 11).

8.13 Guided lab

LAB 8.1 — SNOWFLAKE SQL WORKOUT

Environment: Snowsight, `lab_role`, `lab_wh`, schema `bytemart_lab.sales` with Section 8.1's data. Time: 60–90 minutes.

Step 1 — Clone for safety. `CREATE SCHEMA bytemart_lab.sales_lab CLONE bytemart_lab.sales;` and work in `sales_lab` so you can always recompare with the original.

Step 2 — DDL. Use `CREATE OR ALTER TABLE` to add a `currency` column (`VARCHAR(3)`, default `'INR'`) to `orders`. Run the statement twice and confirm the second run changes nothing.

Step 3 — Multi-table insert. Run the `INSERT FIRST` example and check the row counts against the expected values.

Step 4 — MERGE with deletes. Run the MERGE example on `products` and note the inserted/updated/deleted counts.

Step 5 — Time Travel recovery drill. Run `DELETE FROM orders;` (in `sales_lab`!). Find its query ID in history, then restore with `CREATE TABLE … CLONE … BEFORE(STATEMENT => …)` and `SWAP WITH`. Verify 12 rows again.

Step 6 — Semi-structured. Create `order_docs`, flatten items, then run the recursive path-discovery query.

Step 7 — Metadata. Use `SHOW TABLES ->> SELECT …` to list tables with more than 0 rows, sorted by bytes.

Step 8 — Dynamic SQL. Store a table name in a session variable and count its rows with `IDENTIFIER()`.

8.14 Production use case: a safe daily rebuild

ByteMart's `gold_daily_sales` table is rebuilt every night from silver tables. The first implementation used `CREATE OR REPLACE TABLE gold_daily_sales AS SELECT …`. Twice, a failure mid-rebuild left dashboards pointing at an empty table, and once a buggy change shipped wrong numbers with no easy rollback.

The redesign uses this chapter's features:

1. Build into `gold_daily_sales_next` with CTAS.
2. Run quality checks against `_next` (row counts within tolerance, totals reconciling with silver, no NULL keys).
3. On success, `ALTER TABLE gold_daily_sales SWAP WITH gold_daily_sales_next;` — consumers switch atomically.
4. Keep the previous version as `gold_daily_sales_next` until the next run (instant rollback = swap again), with 7-day Time Travel as a second line of defence.
5. All statements fully qualified; executed by a task under a deployment role (Chapter 15).

(Chapter 15 shows that dynamic tables can replace this pattern for many cases; Chapter 9 shows incremental alternatives when full rebuilds become too expensive.)

8.15 Common mistakes

SNOWFLAKE SQL PITFALLS

1. `CREATE OR REPLACE TABLE` on tables with unique data.
2. DDL inside a transaction you expect to roll back.
3. `UPDATE … FROM` with one-to-many joins (non-deterministic results).
4. MERGE with duplicate keys in the source.
5. Forgetting `OUTER => TRUE` in `FLATTEN`, silently losing parent rows.
6. Comparing `VARIANT` values without casting (string values carry quotes).
7. `LAST_VALUE` with the default window frame.
8. Using `ACCOUNT_USAGE` for real-time monitoring (it lags).
9. Concatenating user input into dynamic SQL.
10. Assuming `SEQ4()` or sequences are gap-free.
11. Relying on Time Travel without setting a minimum retention.

8.16 Troubleshooting exercise

EXERCISE — THE MERGE THAT FAILED AT 2 A.M.

The nightly job fails with an error stating that a row in the target table was matched by multiple rows in the source (`Duplicate row detected during DML action`). The source is `raw.customer_updates`, which receives a full extract plus intraday corrections. Explain the cause, how to confirm it, and two fixes. (Solution at the end of the chapter.)

8.17 Knowledge check

1. What happens to an open transaction when you run `CREATE TABLE` inside it?
2. When would you use `CREATE OR ALTER` instead of `CREATE OR REPLACE`?

3. Explain `INSERT FIRST` versus `INSERT ALL`.
4. Write the Time Travel clause to read a table as it was immediately before query `01b2-…`.
5. What does `OUTER => TRUE` change in `FLATTEN`?
6. Why does `PIVOT` produce column names like `'app'`, and how do you fix it?
7. Which metadata source would you use to find tables dropped three weeks ago?
8. How do `IDENTIFIER()` and bind variables together prevent SQL injection?
9. What does `GROUPING()` return, and why is it needed with `ROLLUP`?
10. Give one use of an ASOF join.

8.18 Hands-on assignment

1. Build a calendar table for 2024–2027 with `GENERATOR`, including year, month, ISO week, day name and an `is_weekend` flag.
2. Using the calendar, produce daily revenue for March 2026 including zero-revenue days.
3. Write a MERGE that applies a CDC feed with `I`/`U`/`D` operation codes, including de-duplication of the feed with `QUALIFY`.
4. Write a query using `SHOW TABLES ->>` that reports, for every table in your lab database, its size in MB and retention period.

8.19 Chapter summary

Snowflake SQL extends standard SQL with conveniences for safe deployment (`CREATE OR ALTER`, `CLONE`, `SWAP WITH`, `UNDROP`), powerful DML (`INSERT OVERWRITE`, multi-table inserts, multi-branch `MERGE`), unloading with `COPY INTO <location>`, and query features such as `QUALIFY`, `GROUP BY ALL`, `EXCLUDE`/`RENAME`, `ASOF JOIN`, `LATERAL`, `FLATTEN`, `PIVOT`/`UNPIVOT`, recursive CTEs, `CONNECT BY`, `GENERATOR` and the flow operator. DDL commits open transactions; DML autocommits unless grouped. Time Travel and the `CHANGES` clause expose earlier table versions within the retention period, which must be configured deliberately and complemented by other protections. Metadata comes from `SHOW`, `INFORMATION_SCHEMA` (real time, per database) and `ACCOUNT_USAGE` (account-wide, historical, delayed). Dynamic SQL must use bind variables and `IDENTIFIER()`.

8.20 Key takeaways

- Prefer `CREATE OR ALTER`, `CLONE` and `SWAP WITH` over destructive replaces.
- De-duplicate MERGE and UPDATE sources.
- `FLATTEN` + `LATERAL` + casts are the semi-structured toolkit.
- Use Time Travel for recovery drills — before you need them.
- Choose the metadata source by scope, freshness and history.
- Bind values; never concatenate them.

8.21 Further reading

- SQL command reference: https://docs.snowflake.com/en/sql-reference-commands
- Query syntax: https://docs.snowflake.com/en/sql-reference/constructs
- MERGE: https://docs.snowflake.com/en/sql-reference/sql/merge
- INSERT (multi-table): https://docs.snowflake.com/en/sql-reference/sql/insert-multi-table
- COPY INTO <location>: https://docs.snowflake.com/en/sql-reference/sql/copy-into-location
- Transactions: https://docs.snowflake.com/en/sql-reference/transactions
- Understanding & using Time Travel: https://docs.snowflake.com/en/user-guide/data-time-travel
- CHANGES clause: https://docs.snowflake.com/en/sql-reference/constructs/changes
- FLATTEN: https://docs.snowflake.com/en/sql-reference/functions/flatten
- PIVOT: https://docs.snowflake.com/en/sql-reference/constructs/pivot
- ASOF JOIN: https://docs.snowflake.com/en/sql-reference/constructs/asof-join
- MATCH_RECOGNIZE: https://docs.snowflake.com/en/sql-reference/constructs/match_recognize
- Flow operator: https://docs.snowflake.com/en/sql-reference/operators-flow
- Account Usage: https://docs.snowflake.com/en/sql-reference/account-usage
- EXECUTE IMMEDIATE: https://docs.snowflake.com/en/sql-reference/sql/execute-immediate

8.22 Solutions

Troubleshooting — MERGE failure. The source contains **more than one row per `customer_id`** (the full extract plus corrections), so a target row matches several source rows and Snowflake cannot choose deterministically (governed by `ERROR_ON_NONDETERMINISTIC_MERGE`, default TRUE). *Confirm:* `SELECT customer_id, COUNT(*) FROM raw.customer_updates GROUP BY 1 HAVING COUNT(*) > 1;`. *Fixes:* (1) de-duplicate in the `USING` clause, keeping the latest record per key: `USING (SELECT * FROM raw.customer_updates QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY loaded_at DESC, source_seq DESC) = 1)`; (2) fix the feed contract so corrections replace rather than append, or stage each batch separately and process batches in order. Setting `ERROR_ON_NONDETERMINISTIC_MERGE = FALSE` would hide the problem and produce arbitrary results — do not do that.

Knowledge check answers.

1. The open transaction is committed first; the DDL then runs in its own transaction.
2. When an object holds data or grants you must preserve and you want to converge it to a definition declaratively (typical in deployments).
3. `INSERT FIRST` sends each row to the first matching branch only; `INSERT ALL` to every matching branch.
4. `SELECT * FROM t BEFORE(STATEMENT => '01b2-…');`
5. Parent rows whose input is empty or NULL produce one row (with NULL element columns) instead of disappearing.

6. Pivoted columns are named after the `IN` list literals, including quotes; alias the pivot result with a column list, e.g. `AS p (month, app_revenue, web_revenue)`.
7. `SNOWFLAKE.ACCOUNT_USAGE.TABLES` (includes dropped tables with `DELETED` timestamps) or `SHOW TABLES HISTORY` within Time Travel.
8. Bind variables pass values separately from the SQL text, so they cannot alter the statement; `IDENTIFIER()` accepts only a valid object name for dynamic table/column references.
9. 1 for rows where the column is aggregated away (a subtotal), 0 otherwise — needed to tell subtotal NULLs from real NULL values.
10. Attaching the exchange rate (or price, or status) in effect at each event's time; joining sensor readings to the latest calibration.

CHAPTER 9

Advanced SQL and Data Modeling

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- explain the grain of a table and design facts and dimensions around it;
- build a star schema for ByteMart and compare it with snowflake schemas, wide tables and Data Vault;
- choose between natural, surrogate and hash keys;
- implement Slowly Changing Dimensions Type 1 and Type 2 with idempotent MERGE logic, and describe Types 0, 3, 4 and 6;
- design incremental, idempotent loads with watermarks and handle late-arriving data;
- reconcile data between layers;
- solve classic analytical problems: sessionization, running totals, gaps and islands, hierarchies, range joins;
- refactor slow or fragile SQL;
- model data for analytics, semantic layers and AI.

Prerequisites: Chapters 3 and 8.

Where this leads: Chapter 14 (CDC streams feed these patterns), Chapter 15 (dynamic tables automate them), Chapter 16 (dbt implements them as models, snapshots and tests), Chapter 27 (semantic views sit on top of the model).

9.0 How this chapter was tested

Like Chapter 3, every query here was **executed on DuckDB** against the ByteMart sample data, and the result tables are the real output. The SQL deliberately uses syntax that Snowflake also accepts — `MERGE`, `QUALIFY`, window functions, recursive CTEs, `MD5`, interval arithmetic. Where Snowflake offers a better native alternative (`GENERATOR`, `CONDITIONAL_TRUE_EVENT`, `SHA2_BINARY`, `DAYOFWEEKISO`), a note says so.

The chapter starts from Chapter 3's tables (`customers`, `products`, `orders`, `order_lines`). In Snowflake, create them with the script in Section 8.1.

9.1 Why model data at all?

Concept

An operational schema is designed to **record transactions correctly**: normalised tables, no redundancy, fast single-row writes. An analytical model is designed to **answer business questions correctly and simply**: predictable joins, consistent definitions, history, and fast aggregation.

Without a model, every analyst re-derives "revenue" from raw tables — and each derivation differs (Chapter 1's "two revenue numbers" problem). Modelling puts the business logic in one tested place.

Grain: the first decision

The **grain** of a table is what one row represents: "one order line", "one customer per day", "one product price version". Declare the grain before choosing columns, and never mix grains in one table. Most modelling bugs — double counting, missing rows — are grain bugs.

Dimensional modelling vocabulary

Term	Meaning	ByteMart example
Fact table	Measurements of a business process at a declared grain	`fct_order_lines`
Measure	Numeric value you aggregate	`quantity`, `line_revenue`
Dimension table	Descriptive context used to filter and group	`dim_customer`, `dim_product`, `dim_date`
Attribute	A descriptive column of a dimension	`country`, `category`, `month`
Degenerate dimension	An identifier kept on the fact with no dimension table	`order_id`
Conformed dimension	A dimension shared consistently by several fact tables	`dim_date`, `dim_customer`
Additive / semi-additive / non-additive	Can be summed across all / some / no dimensions	revenue / account balance (not over time) / ratios

9.2 Star schemas, snowflake schemas and alternatives

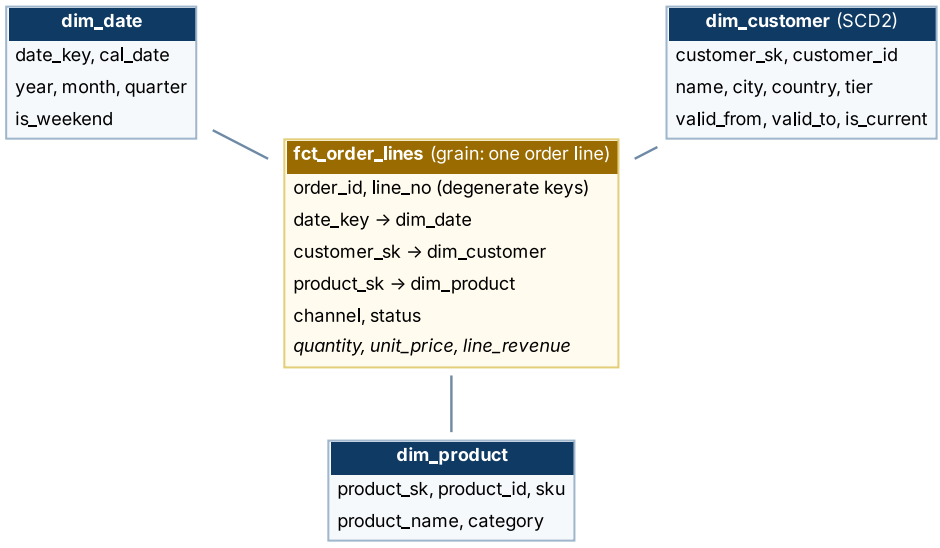


Figure 9.1 — ByteMart's star schema: one fact table at order-line grain surrounded by dimensions.

Approach	Structure	Strengths	Weaknesses
Star schema	Fact + denormalised dimensions	Simple joins, BI-friendly, fast	Some redundancy in dimensions
Snowflake schema	Dimensions normalised into sub-dimensions (product → category)	Less redundancy	More joins; harder for users
One Big Table (OBT)	Fact and dimension attributes pre-joined	Simplest to query; good for some BI and ML feature use	Expensive to maintain history; wide; duplication
Data Vault	Hubs (keys), links (relationships), satellites (history)	Auditable, flexible integration of many sources	Requires a presentation layer on top; more objects

In Snowflake, columnar storage makes wide tables cheap to query, so many teams use **star schemas for the governed core** and **OBT-style marts or semantic views** for specific consumers. The "snowflake schema" is a modelling term and has nothing to do with the Snowflake product.

Building ByteMart's dimensions

`dim_date` — generated, not loaded. The portable recursive version:

```
CREATE TABLE dim_date AS
WITH RECURSIVE d(cal_date) AS (
    SELECT DATE '2025-12-01'
    UNION ALL
    SELECT CAST(cal_date + INTERVAL '1 day' AS DATE) FROM d WHERE cal_date < DATE '2026-03-31
'
)
SELECT CAST(strftime(cal_date, '%Y%m%d') AS INTEGER) AS date_key,
       cal_date,
       EXTRACT(YEAR    FROM cal_date) AS year,
       EXTRACT(QUARTER FROM cal_date) AS quarter,
       EXTRACT(MONTH   FROM cal_date) AS month,
       EXTRACT(ISODOW  FROM cal_date) IN (6, 7) AS is_weekend
FROM d;

SELECT COUNT(*) AS days, MIN(cal_date) AS first_day, MAX(cal_date) AS last_day,
       SUM(CASE WHEN is_weekend THEN 1 ELSE 0 END) AS weekend_days
FROM dim_date;
```

Result (executed on DuckDB):

days	first_day	last_day	weekend_days
121	2025-12-01	2026-03-31	34

1 row.

> **SNOWFLAKE EQUIVALENTS**
>
> In Snowflake, generate dates with `TABLE(GENERATOR(ROWCOUNT => n))` and `DATEADD(day, SEQ4(), start)` (Section 8.7), format the key with `TO_NUMBER(TO_CHAR(cal_date, 'YYYYMMDD'))`, and compute weekends with `DAYOFWEEKISO(cal_date) IN (6, 7)`. DuckDB's `strftime` and `ISODOW` are used here only so the example can be executed.

`dim_product` — Type 1 (current values only), with a hash surrogate key:

```
CREATE TABLE dim_product AS
SELECT MD5(CAST(product_id AS VARCHAR)) AS product_sk,
       product_id, sku, product_name, category
FROM products;

SELECT product_sk, product_id, sku, category FROM dim_product ORDER BY product_id LIMIT 3;
```

Result (executed on DuckDB):

product_sk	product_id	sku	category
38b3eff8baf56627478ec76a704e9b52	101	CHG-USBC-65W	Electronics
ec8956637a99787bd197eacd77acce5e	102	HP-NC-700	Electronics
6974ce5ac660610b44d9b9fed0ff9548	103	KB-MECH-87	Electronics

3 rows.

9.3 Keys: natural, surrogate and hash

Key type	Example	Pros	Cons
Natural (business) key	`sku`, `email`, source `customer_id`	Meaningful; available from source	Can change; can collide across sources; may be sensitive (email)
Sequence surrogate	1, 2, 3 from a sequence/IDENTITY	Small, fast joins	Requires lookup during loads; order-dependent; differs between environments
Hash surrogate	`MD5(source || '|' || natural_key)`	Deterministic — same input gives same key in every environment and every run; parallel loads need no lookups	Wider (32 hex chars); tiny collision risk; must normalise inputs (trim, case) consistently

ByteMart uses **hash keys** for dimensions so that facts can compute the foreign key directly from source data, without looking it up — which makes loads parallel and idempotent. For SCD2 dimensions, the key hashes the natural key **plus the version start** so each version gets its own key.

> **HASH KEYS IN SNOWFLAKE**
>
> `MD5(text)` returns 32 hexadecimal characters in both engines. In Snowflake, `SHA2(text, 256)` reduces collision risk further, and `MD5_BINARY` / `SHA2_BINARY` store keys as compact `BINARY`. Always separate concatenated parts with a delimiter (`'|'`) and replace NULLs (`COALESCE(x, '')`) — otherwise `('ab','c')` and `('a','bc')` hash identically, and any NULL makes the whole key NULL.

9.4 Slowly Changing Dimensions

Concept

Dimension attributes change: a customer moves city, a product changes category. **How** you record that change determines how history is reported. The classic catalogue:

Type	Behaviour	Use when
0	Never change the attribute (keep original)	Original signup date, birth date
1	Overwrite; no history	Corrections (typos), attributes where only the current value matters
2	Add a new row per change, with validity dates and a current flag	Reporting must reflect the value *at the time of the fact* (region, tier)
3	Keep current and previous value in two columns	Limited "before/after" comparison (one prior value)
4	Current values in the dimension; full history in a separate history table	Very frequently changing attributes
6	Hybrid of 1 + 2 + 3: SCD2 rows that also carry the current value	Report both "as was" and "as is" without extra joins

SCD Type 1 with MERGE

```
CREATE TABLE product_updates AS
SELECT * FROM (VALUES
    (104, 'MUG-STEEL', 'Insulated Steel Mug 500ml', 'Home'),       -- name corrected
    (108, 'SSD-1TB',   'Portable SSD 1TB',          'Electronics')  -- new product
) AS v(product_id, sku, product_name, category);
```

```
MERGE INTO dim_product t
USING product_updates s
   ON t.product_id = s.product_id
WHEN MATCHED AND (t.product_name <> s.product_name OR t.category <> s.category) THEN
     UPDATE SET product_name = s.product_name, category = s.category
WHEN NOT MATCHED THEN
     INSERT (product_sk, product_id, sku, product_name, category)
     VALUES (MD5(CAST(s.product_id AS VARCHAR)), s.product_id, s.sku, s.product_name, s.categ
ory);

SELECT product_id, product_name, category FROM dim_product WHERE product_id IN (104, 108)
ORDER BY product_id;
```

Result (executed on DuckDB):

product_id	product_name	category
104	Insulated Steel Mug 500ml	Home
108	Portable SSD 1TB	Electronics

2 rows.

Running the MERGE a second time changes nothing — the update condition is false and the insert finds a match. That is **idempotency**.

SCD Type 2: the full pattern

customer_sk	customer_id	city	tier	valid_from	valid_to	is_current
a1…	2	Kochi	SILVER	2024-03-02	2026-02-01	FALSE
b7…	2	Bengaluru	GOLD	2026-02-01	9999-12-31	TRUE

Priya moved and was upgraded on 2026-02-01: the old row is closed (valid_to set, is_current = FALSE) and a new version is inserted. Orders before February join to version 1; later orders to version 2.

Figure 9.2 — An SCD2 change: the current row is closed and a new version inserted.

Step 1 — Initial load. Every customer gets one current version.

```
CREATE TABLE dim_customer AS
SELECT MD5(CAST(customer_id AS VARCHAR) || '|' || CAST(signup_date AS VARCHAR)) AS customer_s
k,
       customer_id, name, city, country, loyalty_tier,
       MD5(COALESCE(city,'') || '|' || COALESCE(country,'') || '|' || COALESCE(loyalty_tier,'
'))
                                                                    AS attr_hash,
       signup_date              AS valid_from,
       DATE '9999-12-31'        AS valid_to,
       TRUE                     AS is_current
FROM customers;

SELECT customer_id, name, city, loyalty_tier, valid_from, valid_to, is_current
FROM dim_customer ORDER BY customer_id LIMIT 4;
```

Result (executed on DuckDB):

customer_id	name	city	loyalty_tier	valid_from	valid_to	is_current
1	Aarav Sharma	Bengaluru	GOLD	2024-01-15	9999-12-31	TRUE
2	Priya Nair	Kochi	SILVER	2024-03-02	9999-12-31	TRUE
3	Oliver Smith	London	NULL	2024-05-20	9999-12-31	TRUE
4	Amelia Jones	Manchester	GOLD	2025-01-11	9999-12-31	TRUE

4 rows.

`attr_hash` fingerprints the **tracked attributes** (city, country, tier). Comparing one hash is simpler and less error-prone than comparing many nullable columns.

Step 2 — A change feed arrives. Priya moves to Bengaluru and becomes GOLD on 1 February 2026; Oliver gets a tier; Kavya's record arrives unchanged (it must not create a new version).

```
CREATE TABLE customer_changes AS
SELECT * FROM (VALUES
    (2, 'Priya Nair',   'Bengaluru', 'IN', 'GOLD',   DATE '2026-02-01'),
    (3, 'Oliver Smith', 'London',    'UK', 'SILVER', DATE '2026-02-10'),
    (8, 'Kavya Reddy',  'Hyderabad', 'IN', 'SILVER', DATE '2026-02-15')
) AS v(customer_id, name, city, country, loyalty_tier, effective_date);
```

Step 3 — Apply SCD2 in one MERGE. The well-known trick: the source contains each changed record **twice** — once with its real key (to *close* the current row) and once with a NULL merge key (which never matches, so it is *inserted* as the new version).

```
MERGE INTO dim_customer t
USING (
    WITH src AS (
        SELECT c.*,
               MD5(COALESCE(city,'') || '|' || COALESCE(country,'') || '|' || COALESCE(loyalt
y_tier,''))
                   AS attr_hash
        FROM customer_changes c
    ),
    changed AS (
        SELECT s.*
        FROM src s
        JOIN dim_customer d
          ON d.customer_id = s.customer_id AND d.is_current
        WHERE d.attr_hash <> s.attr_hash
    )
    SELECT customer_id AS merge_key, * FROM changed      -- closes the old version
    UNION ALL
    SELECT NULL        AS merge_key, * FROM changed      -- inserts the new version
) s
ON t.customer_id = s.merge_key AND t.is_current
WHEN MATCHED THEN UPDATE SET
     valid_to   = s.effective_date,
     is_current = FALSE
WHEN NOT MATCHED THEN INSERT
     (customer_sk, customer_id, name, city, country, loyalty_tier, attr_hash,
      valid_from, valid_to, is_current)
     VALUES (MD5(CAST(s.customer_id AS VARCHAR) || '|' || CAST(s.effective_date AS VARCHAR)),
             s.customer_id, s.name, s.city, s.country, s.loyalty_tier, s.attr_hash,
             s.effective_date, DATE '9999-12-31', TRUE);

SELECT customer_id, city, loyalty_tier, valid_from, valid_to, is_current
FROM dim_customer
WHERE customer_id IN (2, 3, 8)
ORDER BY customer_id, valid_from;
```

Result (executed on DuckDB):

customer_id	city	loyalty_tier	valid_from	valid_to	is_current
2	Kochi	SILVER	2024-03-02	2026-02-01	FALSE
2	Bengaluru	GOLD	2026-02-01	9999-12-31	TRUE
3	London	NULL	2024-05-20	2026-02-10	FALSE
3	London	SILVER	2026-02-10	9999-12-31	TRUE
8	Hyderabad	SILVER	2026-01-20	9999-12-31	TRUE

5 rows.

Line by line:

- `src` computes the hash of incoming tracked attributes.
- `changed` keeps only records whose hash differs from the **current** dimension row — Kavya's unchanged record is filtered out, so it creates no version.
- The `UNION ALL` emits each changed record twice: with `merge_key = customer_id` (matches the current row → `UPDATE` closes it) and with `merge_key = NULL` (never matches → `INSERT` the new version).

- `valid_to` of the old row equals `valid_from` of the new row; queries use the half-open interval `valid_from <= d < valid_to`.

Step 4 — Prove idempotency. Running the same MERGE again must change nothing, because the new current rows now match the incoming hashes:

```
MERGE INTO dim_customer t
USING (
    WITH src AS (
        SELECT c.*, MD5(COALESCE(city,'') || '|' || COALESCE(country,'') || '|' || COALESCE(l
oyalty_tier,'')) AS attr_hash
        FROM customer_changes c
    ),
    changed AS (
        SELECT s.* FROM src s JOIN dim_customer d ON d.customer_id = s.customer_id AND d.is_c
urrent
        WHERE d.attr_hash <> s.attr_hash
    )
    SELECT customer_id AS merge_key, * FROM changed
    UNION ALL
    SELECT NULL AS merge_key, * FROM changed
) s
ON t.customer_id = s.merge_key AND t.is_current
WHEN MATCHED THEN UPDATE SET valid_to = s.effective_date, is_current = FALSE
WHEN NOT MATCHED THEN INSERT (customer_sk, customer_id, name, city, country, loyalty_tier, at
tr_hash, valid_from, valid_to, is_current)
     VALUES (MD5(CAST(s.customer_id AS VARCHAR) || '|' || CAST(s.effective_date AS
VARCHAR)), s.customer_id, s.name, s.city, s.country, s.loyalty_tier, s.attr_hash, s.effective
_date, DATE '9999-12-31', TRUE);

SELECT COUNT(*) AS versions, SUM(CASE WHEN is_current THEN 1 ELSE 0 END) AS current_rows
FROM dim_customer;
```

Result (executed on DuckDB):

versions	current_rows
10	8

1 row.

Still 10 versions (8 customers + 2 changes) and 8 current rows.

SCD2 EDGE CASES TO HANDLE IN PRODUCTION

1. **Several changes for the same key in one batch** — de-duplicate or process them in effective-date order; otherwise two "new current" rows appear.
2. **Out-of-order (late) changes** — a change effective *before* the current version's `valid_from` requires splitting history; most teams reject or quarantine such records, or rebuild the affected key's history.
3. **Deletes in the source** — decide whether to close the current row (`is_current = FALSE`, `is_deleted = TRUE`) or keep it.
4. **Hash inputs** — normalise case and whitespace before hashing, or cosmetic changes create versions.

dbt **snapshots** (Chapter 16) and Snowflake **streams** (Chapter 14) automate much of this.

Building the fact table with point-in-time dimension keys

```
CREATE TABLE fct_order_lines AS
SELECT l.order_id,
       l.line_no,
       CAST(strftime(o.order_ts, '%Y%m%d') AS INTEGER) AS date_key,
       dc.customer_sk,
       MD5(CAST(l.product_id AS VARCHAR))              AS product_sk,
       o.channel,
       o.status,
       l.quantity,
       l.unit_price,
       l.quantity * l.unit_price                       AS line_revenue
FROM order_lines l
JOIN orders o        ON o.order_id = l.order_id
JOIN dim_customer dc ON dc.customer_id = o.customer_id
                    AND CAST(o.order_ts AS DATE) >= dc.valid_from
                    AND CAST(o.order_ts AS DATE) <  dc.valid_to;

SELECT COUNT(*) AS fact_rows, SUM(line_revenue) AS total_revenue FROM fct_order_lines;
```

Result (executed on DuckDB):

fact_rows	total_revenue
17	98625.00

1 row.

The `valid_from <= order_date < valid_to` join attaches the customer version **in force when the order was placed**. Seventeen fact rows — the same as `order_lines` — confirm that the range join neither dropped nor duplicated lines. **Always check row counts after range joins.**

```
SELECT d.month, dc.city, SUM(f.line_revenue) AS revenue
FROM fct_order_lines f
JOIN dim_date d      ON d.date_key = f.date_key
JOIN dim_customer dc ON dc.customer_sk = f.customer_sk
WHERE dc.customer_id = 2
GROUP BY d.month, dc.city
ORDER BY d.month;
```

Result (executed on DuckDB):

month	city	revenue
1	Kochi	12999.00
2	Bengaluru	2996.00

2 rows.

Priya's January order is attributed to Kochi and her February order to Bengaluru — the "as was" view that SCD2 exists to provide. Joining instead on `dc.customer_id` with `dc.is_current` gives the "as is" view (all her revenue under Bengaluru); Type 6 stores both.

9.5 Incremental loads, watermarks and idempotency

Full versus incremental

A **full load** recomputes everything each run: simple and self-healing, but cost grows with history. An **incremental load** processes only new or changed data: cheaper and faster, but harder to get right.

Watermarks

A **watermark** records how far the last successful load got — typically the maximum `updated_at` (or a CDC sequence number) processed.

```
CREATE TABLE etl_watermarks (pipeline VARCHAR, high_watermark TIMESTAMP);
INSERT INTO etl_watermarks VALUES ('orders_to_silver', TIMESTAMP '2026-02-28 23:59:59');

CREATE TABLE silver_orders AS
SELECT * FROM orders WHERE order_ts <= TIMESTAMP '2026-02-28 23:59:59';
```

```
-- 1. Select new rows beyond the watermark (with a safety overlap for late commits)
CREATE TEMP TABLE batch AS
SELECT *
FROM orders
WHERE order_ts > (SELECT high_watermark - INTERVAL '1 hour'
                  FROM etl_watermarks WHERE pipeline = 'orders_to_silver');

-- 2. Apply idempotently: MERGE by key, so the overlap does not duplicate rows
MERGE INTO silver_orders t
USING batch s ON t.order_id = s.order_id
WHEN MATCHED AND t.status <> s.status THEN UPDATE SET status = s.status
WHEN NOT MATCHED THEN INSERT VALUES (s.order_id, s.customer_id, s.order_ts, s.status, s.chann
el);

-- 3. Advance the watermark only after success
UPDATE etl_watermarks
SET high_watermark = (SELECT MAX(order_ts) FROM batch)
WHERE pipeline = 'orders_to_silver';

SELECT (SELECT COUNT(*) FROM batch)          AS batch_rows,
       (SELECT COUNT(*) FROM silver_orders)  AS silver_rows,
       (SELECT high_watermark FROM etl_watermarks) AS new_watermark;
```

Result (executed on DuckDB):

batch_rows	silver_rows	new_watermark
4	12	2026-03-09 11:00:00

1 row.

Key rules:

1. **Overlap and merge.** Re-reading a small window before the watermark catches rows committed late; MERGE makes the overlap harmless.
2. **Advance the watermark last**, in the same transaction as the data change where possible.

3. **Watermark on a reliable column.** `updated_at` columns that applications forget to update cause silent data loss; CDC (Chapter 14) is more reliable.
4. **Delete-insert by partition** is an alternative idempotent pattern for append-mostly data: delete the target day and re-insert it in one transaction.

Late-arriving data

Situation	Problem	Pattern
Late-arriving fact (an order from yesterday arrives today)	Yesterday's aggregates already published	Reprocess a rolling window (e.g. last 3 days) every run; dynamic tables do this automatically (Chapter 15)
Early-arriving fact / late dimension (order references a customer not yet loaded)	Fact cannot find its dimension key	Insert an **inferred member** (a placeholder dimension row) or map to an "Unknown" member (`customer_sk = '-1'`), and fix up when the dimension arrives
Late correction (status changes days later)	Fact rows must change	MERGE by key; for snapshot facts, keep an `as_of` date

```
-- An "unknown member" row so facts never lose rows to inner joins
INSERT INTO dim_customer VALUES ('-1', -1, 'Unknown', NULL, 'XX', NULL, NULL,
                                 DATE '1900-01-01', DATE '9999-12-31', TRUE);

SELECT customer_sk, name, country FROM dim_customer WHERE customer_id = -1;
```

Result (executed on DuckDB):

customer_sk	name	country
-1	Unknown	XX

1 row.

9.6 Reconciliation

Reconciliation proves that data moved between layers without loss or duplication. Compare counts and sums, overall and per partition:

```
WITH src AS (
    SELECT CAST(o.order_ts AS DATE) AS d, COUNT(*) AS lines, SUM(l.quantity * l.unit_price) A
S amt
    FROM order_lines l JOIN orders o ON o.order_id = l.order_id
    GROUP BY 1
),
tgt AS (
    SELECT dd.cal_date AS d, COUNT(*) AS lines, SUM(f.line_revenue) AS amt
    FROM fct_order_lines f JOIN dim_date dd ON dd.date_key = f.date_key
    GROUP BY 1
)
SELECT COALESCE(s.d, t.d)                    AS d,
       s.lines AS src_lines, t.lines AS tgt_lines,
       s.amt   AS src_amt,   t.amt   AS tgt_amt,
       CASE WHEN s.lines = t.lines AND s.amt = t.amt THEN 'OK' ELSE 'MISMATCH' END AS status
FROM src s
FULL OUTER JOIN tgt t ON t.d = s.d
```

```
WHERE s.lines IS DISTINCT FROM t.lines OR s.amt IS DISTINCT FROM t.amt
   OR s.d IS NULL OR t.d IS NULL;
```

Result (executed on DuckDB):

d	src_lines	tgt_lines	src_amt	tgt_amt	status

0 rows.

No rows returned means every day reconciles. The **full outer join** catches days present on one side only, and `IS DISTINCT FROM` treats NULLs as comparable values. In production, store reconciliation results in an audit table and alert on mismatches (Chapter 19).

9.7 Analytical patterns

Sessionization

A **session** groups a user's events separated by gaps shorter than a threshold — commonly 30 minutes of inactivity.

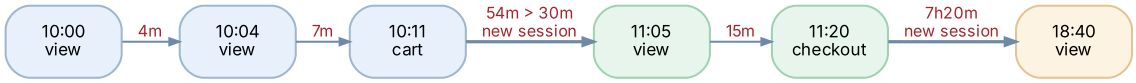


Figure 9.3 — Sessionization: a gap longer than 30 minutes starts a new session.

```
CREATE TABLE click_events AS
SELECT * FROM (VALUES
 (4211, TIMESTAMP '2026-03-02 10:00:00', 'page_view'),
 (4211, TIMESTAMP '2026-03-02 10:04:00', 'page_view'),
 (4211, TIMESTAMP '2026-03-02 10:11:00', 'add_to_cart'),
 (4211, TIMESTAMP '2026-03-02 11:05:00', 'page_view'),
 (4211, TIMESTAMP '2026-03-02 11:20:00', 'checkout'),
 (4211, TIMESTAMP '2026-03-02 18:40:00', 'page_view'),
 (5530, TIMESTAMP '2026-03-02 10:02:00', 'page_view'),
 (5530, TIMESTAMP '2026-03-02 10:20:00', 'page_view')
) AS v(user_id, event_ts, event_type);
```

```
WITH flagged AS (
    SELECT *,
           CASE WHEN LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts) IS NULL
                  OR event_ts - LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts)
                     > INTERVAL '30 minutes'
                THEN 1 ELSE 0 END AS is_new_session
    FROM click_events
),
numbered AS (
    SELECT *,
           SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY event_ts
                                     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS ses
sion_no
    FROM flagged
)
SELECT user_id, session_no,
       MIN(event_ts) AS session_start,
       MAX(event_ts) AS session_end,
```

```
       COUNT(*)      AS events,
       MAX(CASE WHEN event_type = 'checkout' THEN 1 ELSE 0 END) AS converted
FROM numbered
GROUP BY user_id, session_no
ORDER BY user_id, session_no;
```

Result (executed on DuckDB):

user_id	session_no	session_start	session_end	events	converted
4211	1	2026-03-02 10:00:00	2026-03-02 10:11:00	3	0
4211	2	2026-03-02 11:05:00	2026-03-02 11:20:00	2	1
4211	3	2026-03-02 18:40:00	2026-03-02 18:40:00	1	0
5530	1	2026-03-02 10:02:00	2026-03-02 10:20:00	2	0

4 rows.

The technique: flag rows that start a session (first event, or gap above the threshold), then take a **running sum** of the flags — each start increments the session number. In Snowflake, `CONDITIONAL_TRUE_EVENT(gap > threshold) OVER (PARTITION BY user_id ORDER BY event_ts)` computes the session number in one function.

Running totals and moving windows

```
SELECT CAST(order_ts AS DATE) AS d,
       SUM(l.quantity * l.unit_price) AS daily_revenue,
       SUM(SUM(l.quantity * l.unit_price)) OVER (ORDER BY CAST(order_ts AS DATE)
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM orders o JOIN order_lines l ON l.order_id = o.order_id
WHERE o.status <> 'CANCELLED' AND o.order_ts < TIMESTAMP '2026-02-01'
GROUP BY 1
ORDER BY 1;
```

Result (executed on DuckDB):

d	daily_revenue	running_total
2026-01-05	2547.00	2547.00
2026-01-07	12999.00	15546.00
2026-01-20	5398.00	20944.00

3 rows.

Gaps and islands

Islands are runs of consecutive values; **gaps** are the breaks between them. Classic questions: consecutive days a customer was active, periods a product was out of stock, streaks of failed jobs. The trick: for consecutive dates, `date - ROW_NUMBER()` is constant within an island.

```
CREATE TABLE daily_activity AS
SELECT * FROM (VALUES
 (1, DATE '2026-03-01'), (1, DATE '2026-03-02'), (1, DATE '2026-03-03'),
 (1, DATE '2026-03-05'), (1, DATE '2026-03-06'),
 (2, DATE '2026-03-02'), (2, DATE '2026-03-04')
) AS v(customer_id, active_date);
```

```sql
WITH grp AS (
    SELECT customer_id, active_date,
           active_date - CAST(ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY active_dat
e) AS INTEGER)
               AS island_id
    FROM daily_activity
)
SELECT customer_id,
       MIN(active_date) AS streak_start,
       MAX(active_date) AS streak_end,
       COUNT(*)         AS streak_days
FROM grp
GROUP BY customer_id, island_id
ORDER BY customer_id, streak_start;
```

Result (executed on DuckDB):

customer_id	streak_start	streak_end	streak_days
1	2026-03-01	2026-03-03	3
1	2026-03-05	2026-03-06	2
2	2026-03-02	2026-03-02	1
2	2026-03-04	2026-03-04	1

4 rows.

Within an island, both the date and the row number increase by one per row, so their difference is constant and identifies the island. (In Snowflake, write `DATEADD(day, -ROW_NUMBER() OVER (…), active_date)`.)

Hierarchies

Hierarchies are solved with recursive CTEs (Section 8.7 showed the referral tree). A common modelling output is a **bridge** or **closure table** listing every ancestor–descendant pair, so that "total revenue of everyone Aarav referred, directly or indirectly" becomes a simple join:

```sql
CREATE TABLE referral_closure AS
WITH RECURSIVE t(ancestor_id, descendant_id, depth) AS (
    SELECT customer_id, customer_id, 0 FROM customers
    UNION ALL
    SELECT t.ancestor_id, c.customer_id, t.depth + 1
    FROM t JOIN customers c ON c.referred_by = t.descendant_id
)
SELECT * FROM t;

SELECT a.name AS referrer,
       COUNT(DISTINCT rc.descendant_id) - 1                    AS network_size,
       COALESCE(SUM(CASE WHEN f.status <> 'CANCELLED'
                         THEN f.line_revenue END), 0)          AS network_revenue_incl_self
FROM referral_closure rc
JOIN customers a            ON a.customer_id = rc.ancestor_id
LEFT JOIN dim_customer dc   ON dc.customer_id = rc.descendant_id
LEFT JOIN fct_order_lines f ON f.customer_sk = dc.customer_sk
GROUP BY a.name
HAVING COUNT(DISTINCT rc.descendant_id) > 1
ORDER BY network_revenue_incl_self DESC;
```

Result (executed on DuckDB):

referrer	network_size	network_revenue_incl_self
Aarav Sharma	3	41838.00
Oliver Smith	1	22391.00
Priya Nair	1	15995.00

3 rows.

The closure table has one row per (ancestor, descendant) pair, including each customer as their own ancestor at depth 0 — hence the `- 1` in the network size. Aarav's network contains Priya, Diya and Kavya (Kavya indirectly, through Priya). The **left** joins matter: Kavya has no orders, and inner joins to the fact table would have silently removed her from the network count.

Range joins and validity periods

Joining an event to the record valid at its time (prices, tiers, exchange rates) is a **range join**. You saw it in the fact build (`valid_from <= date < valid_to`). Snowflake's `ASOF JOIN` (Section 8.7) is the efficient alternative when only "the latest record at or before" is needed. Range joins on large tables can be expensive; Chapter 17 shows how to keep them fast.

9.8 Refactoring SQL

Production SQL degrades as requirements accumulate. Common refactors:

Smell	Refactor
Correlated subquery in `SELECT` for each row ("latest order per customer")	Window function + `QUALIFY`, or pre-aggregate and join
Same subquery repeated three times	CTE, or a view/model
`SELECT DISTINCT` to "fix" duplicates	Find the join that fans out; aggregate to the right grain first
Huge `CASE` mapping embedded in many queries	Mapping table (reference data) joined once
`OR` across different columns in joins	Split into `UNION ALL` of simpler joins
Filtering on a function of a column (`TO_CHAR(date) = …`)	Range predicate on the raw column (pruning, Chapter 5)
Business logic duplicated across dashboards	Move into a modelled table or semantic view

```sql
-- Before: correlated subquery per customer
SELECT c.name,
       (SELECT MAX(o.order_ts) FROM orders o WHERE o.customer_id = c.customer_id) AS last_ord
er
FROM customers c
ORDER BY c.name
LIMIT 3;
```

Result (executed on DuckDB):

name	last_order
Aarav Sharma	2026-03-01 13:45:00
Amelia Jones	2026-03-08 18:25:00

name	last_order
Diya Patel	2026-02-20 23:55:00

3 rows.

```sql
-- After: one aggregation and a join (same result, scales better and is easier to extend)
WITH last_orders AS (
    SELECT customer_id, MAX(order_ts) AS last_order FROM orders GROUP BY customer_id
)
SELECT c.name, lo.last_order
FROM customers c
LEFT JOIN last_orders lo ON lo.customer_id = c.customer_id
ORDER BY c.name
LIMIT 3;
```

Result (executed on DuckDB):

name	last_order
Aarav Sharma	2026-03-01 13:45:00
Amelia Jones	2026-03-08 18:25:00
Diya Patel	2026-02-20 23:55:00

3 rows.

Modern optimizers often rewrite simple correlated subqueries automatically, but explicit aggregation is clearer, testable, and robust when logic grows. Always **compare results before and after** a refactor (`EXCEPT` in both directions should return no rows).

9.9 Modelling for analytics and AI

Modern consumers include BI tools, semantic layers, ML pipelines and LLM-based agents. Each benefits from different modelling choices:

Consumer	Modelling needs
BI dashboards	Star schemas or OBT marts; conformed dimensions; pre-aggregated tables for heavy dashboards
Semantic layer (semantic views, Chapter 27)	Clear facts/dimensions with **declared relationships and primary keys**, business-friendly names, column descriptions, documented metrics
ML feature engineering	Point-in-time correct features (no future leakage) — SCD2 and as-of joins; one row per entity per snapshot date
Text-to-SQL agents (Cortex Analyst)	Fewer, well-named tables; synonyms; verified queries; avoid ambiguous column names like `amount` in many tables
RAG (Chapter 29)	Document and chunk tables with stable IDs, metadata columns for filtering (product, region, date, access level), source URLs for citations

DOCUMENTATION IS PART OF THE MODEL

Column comments (`COMMENT ON COLUMN ...` or `COMMENT` in DDL), table comments, tags and declared keys are read by people, catalogs, semantic layers and AI assistants alike. An undocumented model is only half built.

9.10 Common mistakes

MODELLING MISTAKES

1. Not declaring the grain; mixing grains in one table.
2. Joining facts to SCD2 dimensions on the natural key without the date range (duplicates!).
3. Hash keys without delimiters or NULL handling.
4. Advancing the watermark before the load succeeds.
5. Using `SELECT DISTINCT` to hide fan-out.
6. Treating semi-additive measures (balances, inventory) as additive over time.
7. Inner-joining facts to dimensions that may be missing members.
8. SCD2 on attributes that change constantly (use Type 4 or a separate fact).
9. No reconciliation between layers.

9.11 Troubleshooting exercise

EXERCISE — REVENUE DOUBLED FOR SOME CUSTOMERS

After introducing SCD2 on `dim_customer`, a dashboard's revenue for a few customers doubles. The query is `SELECT dc.country, SUM(f.line_revenue) FROM fct_order_lines f JOIN dim_customer dc ON dc.customer_id = <customer_id from fact> GROUP BY 1`. Explain and fix. (Solution at the end of the chapter.)

9.12 Practice exercises

1. Write the SCD Type 3 variant for `loyalty_tier`: add `previous_tier` and `tier_changed_on`, and update them from `customer_changes`.
2. Using `fct_order_lines` and `dim_product`, compute each category's share of non-cancelled revenue per month.
3. Find customers whose consecutive orders are more than 30 days apart.
4. Write a reconciliation query comparing `orders` with `silver_orders` by status.
5. Compute, for each order, the number of days since that customer's previous order and flag "returning within 30 days".

9.13 Chapter summary

Data modelling turns operational data into structures that answer questions correctly: start with the grain, build facts and dimensions (usually a star schema), and choose keys deliberately — deterministic hash keys make loads parallel and idempotent. Slowly Changing Dimensions record attribute changes; Type 2 adds versions with validity dates so facts join to the version in force at the time, and a two-row MERGE implements it

idempotently. Incremental loads use watermarks, overlap windows and key-based MERGE; late-arriving data needs reprocessing windows and unknown members. Reconciliation proves correctness between layers. Window functions solve sessionization, running totals and gaps-and-islands; recursive CTEs and closure tables handle hierarchies. Refactoring keeps SQL scalable and readable, and modelling for semantic layers and AI requires declared relationships, documentation and point-in-time correctness.

9.14 Key takeaways

- Grain first, always.
- SCD2 joins need the date range; check row counts after every range join.
- Make every load idempotent: MERGE by key, delete-insert by partition, watermark last.
- Reconcile layers automatically.
- Document models — people and AI both depend on it.

9.15 Further reading

- Snowflake — MERGE: https://docs.snowflake.com/en/sql-reference/sql/merge
- Snowflake — Window functions: https://docs.snowflake.com/en/sql-reference/functions-window
- Snowflake — CONDITIONAL_TRUE_EVENT: https://docs.snowflake.com/en/sql-reference/functions/conditional_true_event
- Snowflake — Working with CTEs (recursive): https://docs.snowflake.com/en/user-guide/queries-cte
- Snowflake — Hierarchical data: https://docs.snowflake.com/en/user-guide/queries-hierarchical
- Kimball Group — Dimensional modeling techniques: https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/

9.16 Solutions

Troubleshooting — doubled revenue. After SCD2, `dim_customer` has **several rows per `customer_id`** (one per version). Joining on `customer_id` alone matches every version, duplicating fact rows for customers with history. Fix: join on the surrogate key stored in the fact (`f.customer_sk = dc.customer_sk`) for "as was" reporting, or on `customer_id` **and** `dc.is_current` for "as is" reporting.

Exercise 1 — SCD Type 3:

```
ALTER TABLE customers ADD COLUMN previous_tier VARCHAR;
ALTER TABLE customers ADD COLUMN tier_changed_on DATE;

UPDATE customers c
SET previous_tier   = c.loyalty_tier,
    loyalty_tier    = ch.loyalty_tier,
    tier_changed_on = ch.effective_date
FROM customer_changes ch
WHERE ch.customer_id = c.customer_id
  AND ch.loyalty_tier IS DISTINCT FROM c.loyalty_tier;
```

```
SELECT customer_id, name, previous_tier, loyalty_tier, tier_changed_on
FROM customers WHERE tier_changed_on IS NOT NULL ORDER BY customer_id;
```

Result (executed on DuckDB):

customer_id	name	previous_tier	loyalty_tier	tier_changed_on
2	Priya Nair	SILVER	GOLD	2026-02-01
3	Oliver Smith	NULL	SILVER	2026-02-10

2 rows.

Exercise 2 — category share per month:

```
SELECT d.year, d.month, p.category,
       SUM(f.line_revenue) AS revenue,
       ROUND(100.0 * SUM(f.line_revenue)
             / SUM(SUM(f.line_revenue)) OVER (PARTITION BY d.year, d.month), 1) AS pct_of_mon
th
FROM fct_order_lines f
JOIN dim_date d    ON d.date_key = f.date_key
JOIN dim_product p ON p.product_sk = f.product_sk
WHERE f.status <> 'CANCELLED'
GROUP BY d.year, d.month, p.category
ORDER BY d.year, d.month, revenue DESC;
```

Result (executed on DuckDB):

year	month	category	revenue	pct_of_month
2026	1	Electronics	20195.00	96.4
2026	1	Home	749.00	3.6
2026	2	Furniture	15999.00	41.9
2026	2	Electronics	13898.00	36.4
2026	2	Home	8292.00	21.7
2026	3	Electronics	19695.00	83.8
2026	3	Home	3798.00	16.2

7 rows.

Exercise 3 — gaps over 30 days:

```
SELECT customer_id, order_id, CAST(order_ts AS DATE) AS order_date,
       CAST(order_ts AS DATE)
         - CAST(LAG(order_ts) OVER (PARTITION BY customer_id ORDER BY order_ts) AS DATE) AS g
ap_days
FROM orders
QUALIFY gap_days > 30
ORDER BY customer_id;
```

Result (executed on DuckDB):

customer_id	order_id	order_date	gap_days
1	1009	2026-03-01	40
2	1007	2026-02-14	38
3	1012	2026-03-09	40
4	1011	2026-03-08	33

4 rows.

Exercise 4 — reconciliation by status:

```
SELECT COALESCE(a.status, b.status) AS status, a.n AS source_rows, b.n AS silver_rows
FROM (SELECT status, COUNT(*) n FROM orders GROUP BY status) a
FULL OUTER JOIN (SELECT status, COUNT(*) n FROM silver_orders GROUP BY status) b
  ON a.status = b.status
ORDER BY 1;
```

Result (executed on DuckDB):

status	source_rows	silver_rows
CANCELLED	1	1
DELIVERED	7	7
PENDING	1	1
SHIPPED	3	3

4 rows.

Exercise 5 — returning within 30 days:

```
SELECT customer_id, order_id, CAST(order_ts AS DATE) AS order_date,
       CAST(order_ts AS DATE)
         - CAST(LAG(order_ts) OVER (PARTITION BY customer_id ORDER BY order_ts) AS DATE) AS d
ays_since_prev,
       CASE WHEN CAST(order_ts AS DATE)
                 - CAST(LAG(order_ts) OVER (PARTITION BY customer_id ORDER BY order_ts) AS DA
TE) <= 30
            THEN TRUE ELSE FALSE END AS returning_30d
FROM orders
ORDER BY customer_id, order_ts;
```

Result (executed on DuckDB):

customer_id	order_id	order_date	days_since_prev	returning_30d
1	1001	2026-01-05	NULL	FALSE
1	1003	2026-01-20	15	TRUE
1	1009	2026-03-01	40	FALSE
2	1002	2026-01-07	NULL	FALSE
2	1007	2026-02-14	38	FALSE
3	1004	2026-01-28	NULL	FALSE
3	1012	2026-03-09	40	FALSE
4	1005	2026-02-03	NULL	FALSE
4	1011	2026-03-08	33	FALSE
5	1006	2026-02-11	NULL	FALSE
6	1008	2026-02-20	NULL	FALSE
7	1010	2026-03-03	NULL	FALSE

12 rows.

CHAPTER 10

Snowflake Scripting, Procedures, and Functions

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- choose between SQL statements, UDFs, UDTFs, stored procedures and external code for a task;
- write Snowflake Scripting blocks with variables, control flow, cursors, result sets and dynamic SQL;
- handle errors with declared exceptions, built-in exceptions, `EXIT` and `CONTINUE` handlers;
- manage transactions inside procedures;
- write stored procedures in SQL, Python and JavaScript;
- write scalar and table functions in SQL and Python, including vectorized Python UDFs;
- explain owner's, caller's and restricted caller's rights;
- call external APIs safely with external access integrations and secrets;
- log, trace, test and deploy procedural code.

Prerequisites: Chapters 6 (objects, roles) and 8 (Snowflake SQL, dynamic SQL).

Where this leads: Chapter 11 (Snowpark builds procedures and UDFs from Python projects), Chapter 15 (tasks call procedures), Chapter 19 (logging and alerting), Chapter 22 (procedures as agent tools need guardrails), Chapter 30 (procedures exposed as MCP tools).

Execution environment: Snowsight, role `lab_role`, warehouse `lab_wh`, the `bytemart_lab.sales` data from Section 8.1. Examples were checked against Snowflake's documentation at the research cutoff but not executed on Snowflake.

10.1 Choosing a programming model

Before writing procedural code, ask whether plain SQL — perhaps in a view, dynamic table or task — already solves the problem. Set-based SQL is usually faster, simpler and easier to optimise than row-by-row logic.

Need	Best fit	Why
Compute a value per row inside a query (tax, normalisation, parsing)	**Scalar UDF** (SQL, Python, Java, JavaScript, Scala)	Used like a built-in function
Return several rows per input row, or a table from parameters	**UDTF** (table function)	Used in `FROM TABLE(...)`
Heavy per-row Python over many rows (ML scoring, text processing)	**Vectorized Python UDF**	Processes batches as pandas DataFrames

Need	Best fit	Why
Multi-step workflow: DDL, DML, branching, loops, error handling, transactions	**Stored procedure** (SQL/Snowflake Scripting, Python, Java, JavaScript, Scala)	Called with `CALL`; can run any statement
DataFrame-style transformations in Python	**Snowpark** (Chapter 11), deployed as procedures or run from a client	Code pushed down as SQL
Call an external HTTP API	UDF/procedure with an **external access integration**	Controlled egress with secrets
Long-running services, custom containers, GPUs	**Snowpark Container Services** (Chapter 11)	Full container runtime

Language choice: use **Snowflake Scripting (SQL)** for orchestration of SQL statements; **Python** when you need libraries (regex, ML, parsing, HTTP) or DataFrames; **Java/Scala** when reusing JVM code; **JavaScript** mainly for existing legacy procedures.

10.2 Snowflake Scripting fundamentals

Blocks

Snowflake Scripting adds procedural constructs to SQL. Code is organised in **blocks**:

```
DECLARE
    -- variables, cursors, result sets, exceptions
BEGIN
    -- statements
EXCEPTION
    -- handlers
END;
```

You can run a block directly in Snowsight as an **anonymous block**, or make it the body of a stored procedure. In some clients (classic SnowSQL, older drivers) wrap anonymous blocks in `EXECUTE IMMEDIATE $$ … $$;` so the client does not split the code at semicolons.

```
EXECUTE IMMEDIATE $$
DECLARE
    n_orders  NUMBER;
    threshold NUMBER DEFAULT 10;
BEGIN
    SELECT COUNT(*) INTO :n_orders FROM bytemart_lab.sales.orders;
    IF (n_orders > threshold) THEN
        RETURN 'Busy: ' || n_orders || ' orders';
    ELSE
        RETURN 'Quiet: ' || n_orders || ' orders';
    END IF;
END;
$$;
```

Expected output: `Busy: 12 orders`.

Variables

Construct	Example	Notes
Declare in `DECLARE`	`total NUMBER(12,2) DEFAULT 0;`	Type optional if a default is given
Declare inline	`LET rate FLOAT := 1.18;`	`LET` inside `BEGIN … END`
Assign	`total := total + 1;`	`:=` operator
Use in a SQL statement	`WHERE status = :p_status`	**Colon prefix** inside SQL statements
Use in expressions	`IF (total > 0) THEN`	**No colon** outside SQL statements
Assign query result	`SELECT COUNT(*) INTO :n FROM t;`	Single-row queries
Object name in a variable	`FROM IDENTIFIER(:tbl)`	Safe dynamic identifiers

The colon rule is the most common beginner error: inside an embedded SQL statement a variable needs `:`; in Scripting expressions it does not.

Built-in status variables

After each DML statement, Snowflake Scripting sets `SQLROWCOUNT` (rows affected), `SQLFOUND` (TRUE if at least one row was affected) and `SQLNOTFOUND`. After any statement, `SQLID` holds the query ID — invaluable for logging and Time Travel recovery.

Control flow

```
-- IF / ELSEIF / ELSE
IF (x > 100) THEN ... ELSEIF (x > 10) THEN ... ELSE ... END IF;

-- CASE statement
CASE (channel)
    WHEN 'web' THEN ...
    WHEN 'app' THEN ...
    ELSE ...
END CASE;

-- Loops
FOR i IN 1 TO 10 DO ... END FOR;
WHILE (attempt < 3) DO ... END WHILE;
REPEAT ... UNTIL (done) END REPEAT;
LOOP ... IF (done) THEN BREAK; END IF; END LOOP;
-- BREAK exits a loop; CONTINUE skips to the next iteration
```

Cursors and result sets

A **cursor** iterates over query rows; a **RESULTSET** holds a query result that you can loop over or return as a table.

```
EXECUTE IMMEDIATE $$
DECLARE
    c_tables CURSOR FOR
        SELECT table_name
        FROM bytemart_lab.information_schema.tables
        WHERE table_schema = 'SALES' AND table_type = 'BASE TABLE';
```

```
    report  VARCHAR DEFAULT '';
    n       NUMBER;
BEGIN
    FOR rec IN c_tables DO
        LET tbl VARCHAR := 'bytemart_lab.sales.' || rec.table_name;
        SELECT COUNT(*) INTO :n FROM IDENTIFIER(:tbl);
        report := report || rec.table_name || '=' || n || '; ';
    END FOR;
    RETURN report;
END;
$$;
```

Line by line: the cursor lists base tables in the schema; `FOR rec IN c_tables` loops over its rows, exposing columns as `rec.table_name`; `IDENTIFIER(:tbl)` safely turns the string into a table reference; the result accumulates in `report`.

> **PREFER SET-BASED SQL TO CURSOR LOOPS**
> Looping over rows and issuing one statement per row is slow and costly. Cursors are appropriate for iterating over *objects* (tables, partitions, configuration rows), not over business data.

Returning tables

```
CREATE OR REPLACE PROCEDURE bytemart_lab.sales.top_orders(p_n NUMBER)
RETURNS TABLE (order_id NUMBER, customer_id NUMBER, revenue NUMBER(12,2))
LANGUAGE SQL
AS
$$
DECLARE
    res RESULTSET DEFAULT (
        SELECT order_id, customer_id, revenue
        FROM bytemart_lab.sales.order_revenue
        ORDER BY revenue DESC
        LIMIT :p_n);
BEGIN
    RETURN TABLE(res);
END;
$$;

CALL bytemart_lab.sales.top_orders(3);
```

`CALL` output can be queried afterwards with `RESULT_SCAN(LAST_QUERY_ID())` or the flow operator: `CALL top_orders(3) ->> SELECT * FROM $1 WHERE revenue > 13000;`.

10.3 Error handling

Declared and built-in exceptions

```
DECLARE
    future_date EXCEPTION (-20001, 'Run date cannot be in the future');
BEGIN
    IF (:p_date > CURRENT_DATE()) THEN
        RAISE future_date;
    END IF;
    ...
```

```
EXCEPTION
    WHEN future_date THEN
        RETURN 'Rejected: ' || SQLERRM;
    WHEN STATEMENT_ERROR THEN
        RETURN 'SQL failed: ' || SQLCODE || ' ' || SQLERRM;
    WHEN EXPRESSION_ERROR THEN
        RETURN 'Expression failed: ' || SQLERRM;
    WHEN OTHER THEN
        RAISE;          -- re-raise anything unexpected
END;
```

Element	Meaning
`name EXCEPTION (number, 'message')`	Custom exception; numbers are negative in the range used for user errors (examples use −20000 to −20999)
`RAISE name;`	Raise a declared exception
`RAISE;`	Inside a handler, re-raise the current exception
`STATEMENT_ERROR`	A SQL statement failed
`EXPRESSION_ERROR`	An expression failed (for example, assigning text to a number variable)
`OTHER`	Anything else
`SQLCODE`, `SQLERRM`, `SQLSTATE`	Code, message and state of the current exception
`WHEN … THEN` (EXIT, default)	Run the handler, then leave the block
`WHEN … CONTINUE THEN`	Run the handler, then continue with the statement after the failing one

SWALLOWED ERRORS

A handler that returns a message instead of re-raising makes the *caller* (a task, an orchestrator) believe the procedure succeeded. For pipeline procedures, log the error and then `RAISE;` so the task run is marked FAILED and alerts fire (Chapter 19).

10.4 Transactions in procedures

- A procedure can start and end its own transactions with `BEGIN TRANSACTION` … `COMMIT` / `ROLLBACK`.
- DDL inside a procedure commits any open transaction, exactly as outside.
- If an exception propagates out of a block, an open transaction started in that procedure is not automatically committed; your handler should `ROLLBACK` explicitly before re-raising.
- A procedure called inside a caller's transaction participates in it unless it manages its own; nesting rules are documented under "Transactions in stored procedures".
- Since May 2026, stored procedure bodies can create **scoped temporary tables** (the `SCOPED TEMP TABLE` syntax in `CREATE PROCEDURE`) whose lifetime is limited to the procedure — preventing scratch tables from leaking into the caller's session. Check the CREATE PROCEDURE reference for the exact syntax before use.

10.5 A production-style SQL procedure

ByteMart needs an idempotent daily load of a sales summary, with an audit log. First the supporting tables:

```
CREATE SCHEMA IF NOT EXISTS bytemart_lab.ops;

CREATE TABLE IF NOT EXISTS bytemart_lab.ops.daily_sales (
    sales_date DATE, channel VARCHAR(6), orders NUMBER, revenue NUMBER(14,2),
    loaded_at TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP());

CREATE TABLE IF NOT EXISTS bytemart_lab.ops.run_log (
    run_id VARCHAR DEFAULT UUID_STRING(), proc_name VARCHAR, run_date DATE,
    status VARCHAR, rows_written NUMBER, error_code NUMBER, error_message VARCHAR,
    query_id VARCHAR, started_at TIMESTAMP_LTZ, finished_at TIMESTAMP_LTZ);
```

The procedure:

```
CREATE OR REPLACE PROCEDURE bytemart_lab.ops.load_daily_sales(p_run_date DATE)
RETURNS VARIANT
LANGUAGE SQL
EXECUTE AS OWNER
COMMENT = 'Idempotently rebuilds one day of ops.daily_sales and logs the run'
AS
$$
DECLARE
    v_started   TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP();
    v_rows      NUMBER DEFAULT 0;
    v_query_id  VARCHAR;
    future_date EXCEPTION (-20001, 'Run date cannot be in the future');
BEGIN
    IF (p_run_date > CURRENT_DATE()) THEN
        RAISE future_date;
    END IF;

    BEGIN TRANSACTION;
        DELETE FROM bytemart_lab.ops.daily_sales WHERE sales_date = :p_run_date;

        INSERT INTO bytemart_lab.ops.daily_sales (sales_date, channel, orders, revenue)
        SELECT order_ts::DATE, channel, COUNT(DISTINCT order_id), SUM(revenue)
        FROM bytemart_lab.sales.order_revenue
        WHERE order_ts::DATE = :p_run_date
          AND status <> 'CANCELLED'
        GROUP BY 1, 2;

        v_rows     := SQLROWCOUNT;
        v_query_id := SQLID;
    COMMIT;

    INSERT INTO bytemart_lab.ops.run_log
        (proc_name, run_date, status, rows_written, query_id, started_at, finished_at)
    VALUES ('LOAD_DAILY_SALES', :p_run_date, 'SUCCESS', :v_rows, :v_query_id,
            :v_started, CURRENT_TIMESTAMP());

    RETURN OBJECT_CONSTRUCT('status', 'SUCCESS', 'run_date', p_run_date, 'rows', v_rows);

EXCEPTION
    WHEN future_date THEN
        RETURN OBJECT_CONSTRUCT('status', 'REJECTED', 'reason', SQLERRM);
    WHEN OTHER THEN
        ROLLBACK;
        LET v_code NUMBER  := SQLCODE;
        LET v_msg  VARCHAR := SQLERRM;
        INSERT INTO bytemart_lab.ops.run_log
            (proc_name, run_date, status, error_code, error_message, started_at, finished_at)
        VALUES ('LOAD_DAILY_SALES', :p_run_date, 'FAILED', :v_code, :v_msg,
```

```
                :v_started, CURRENT_TIMESTAMP());
        RAISE;
END;
$$;

CALL bytemart_lab.ops.load_daily_sales('2026-02-11');
CALL bytemart_lab.ops.load_daily_sales('2026-02-11');   -- idempotent: same result
SELECT * FROM bytemart_lab.ops.daily_sales ORDER BY sales_date, channel;
SELECT proc_name, run_date, status, rows_written, error_message FROM bytemart_lab.ops.run_log
ORDER BY started_at DESC;
```

Design points:

1. **Idempotent:** delete-then-insert of one day inside one transaction; calling twice gives the same result.
2. **Atomic:** consumers never see the day half-loaded.
3. **Audited:** every run is logged with rows written and the query ID.
4. **Fails loudly:** unexpected errors are rolled back, logged and re-raised.
5. **Validates inputs** before doing work.

Expected after the calls: `daily_sales` contains one row for 2026-02-11 (`web`, 1 order, revenue 13898.00); `run_log` contains two `SUCCESS` rows.

10.6 Owner's rights, caller's rights and restricted caller's rights

A stored procedure runs with one of three privilege contexts, chosen by `EXECUTE AS`:

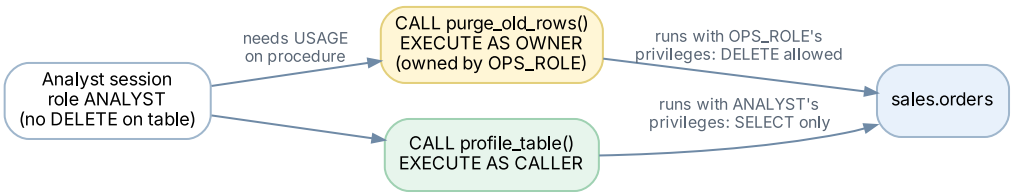


Figure 10.1 — The same caller gets different powers depending on the procedure's rights model.

Model	Runs with privileges of	Session state visible	Typical use
`EXECUTE AS OWNER` (**default**)	The procedure's owner role	Limited — cannot read or change the caller's session variables or most session parameters	Controlled operations: let analysts trigger a purge or refresh without granting them DELETE
`EXECUTE AS CALLER`	The calling session's role(s)	Full caller session	Utilities that should only do what the caller could do anyway (profiling, admin helpers)
`EXECUTE AS RESTRICTED CALLER` (Preview)	The caller, but only privileges explicitly granted for use by the procedure	Caller session, restricted	Reducing the blast radius of caller's-rights code (for example, apps)

OWNER'S-RIGHTS PROCEDURES ARE PRIVILEGE ESCALATION BY DESIGN

Anyone with `USAGE` on an owner's-rights procedure can do whatever the procedure does with the owner's privileges. Treat such procedures like `sudo`: validate every parameter, never build SQL by concatenating parameters, keep the owner role narrowly privileged, and grant `USAGE` deliberately. This matters even more when procedures become **AI agent tools** (Chapters 22, 28, 30).

10.7 Python stored procedures

Python procedures receive a Snowpark `Session` as their first argument and can use Snowpark DataFrames (Chapter 11) and supported packages.

```
CREATE OR REPLACE PROCEDURE bytemart_lab.ops.profile_table(table_name VARCHAR)
RETURNS TABLE (column_name VARCHAR, null_count NUMBER, distinct_count NUMBER)
LANGUAGE PYTHON
RUNTIME_VERSION = '3.12'
PACKAGES = ('snowflake-snowpark-python')
HANDLER = 'run'
EXECUTE AS CALLER
AS
$$
from snowflake.snowpark import functions as F

def run(session, table_name):
    df = session.table(table_name)
    rows = []
    for col in df.columns:
        stats = df.select(
            F.sum(F.iff(F.col(col).is_null(), 1, 0)).alias("NULLS"),
            F.count_distinct(F.col(col)).alias("DISTINCTS"),
        ).collect()[0]
        rows.append((col, stats["NULLS"], stats["DISTINCTS"]))
    return session.create_dataframe(rows, schema=["COLUMN_NAME", "NULL_COUNT", "DISTINCT_COUN
T"])
$$;

CALL bytemart_lab.ops.profile_table('BYTEMART_LAB.SALES.CUSTOMERS');
```

Line by line:

- `RUNTIME_VERSION` selects the Python version. At the research cutoff, Snowflake lists 3.10–3.14 as generally available, with 3.9 deprecated.
- `PACKAGES` must include `snowflake-snowpark-python` for procedures; other packages from Snowflake's channel (or PyPI via an artifact repository, where enabled) can be added.
- `HANDLER = 'run'` names the entry function; Snowflake passes the session first, then the SQL arguments.
- `RETURNS TABLE (…)` + returning a DataFrame produces a tabular result.
- `EXECUTE AS CALLER` ensures the procedure can only profile tables the caller can read.
- The loop issues one aggregation per column — acceptable for a utility, but for wide tables build all aggregations into one `select` to scan the table once.

Expected (abridged): one row per column of `customers`; for example `CITY` with `null_count = 1`, `LOYALTY_TIER` with `null_count = 2`, `CUSTOMER_ID` with `distinct_count = 8`.

10.8 JavaScript stored procedures

JavaScript procedures remain supported and appear in many existing codebases. They use the `snowflake` object to run statements; arguments are referenced in **uppercase**, and a JavaScript procedure returns a single value.

```
CREATE OR REPLACE PROCEDURE bytemart_lab.ops.count_rows_js(TABLE_NAME VARCHAR)
RETURNS FLOAT
LANGUAGE JAVASCRIPT
EXECUTE AS CALLER
AS
$$
  var stmt = snowflake.createStatement({
      sqlText: "SELECT COUNT(*) FROM IDENTIFIER(?)",
      binds: [TABLE_NAME]
  });
  var rs = stmt.execute();
  rs.next();
  return rs.getColumnValue(1);
$$;

CALL bytemart_lab.ops.count_rows_js('bytemart_lab.sales.orders');
```

For new development, prefer Snowflake Scripting (SQL orchestration) or Python (libraries and DataFrames). Migrating JavaScript procedures is a common modernisation task (Chapter 35).

10.9 User-defined functions

SQL scalar UDF

```
CREATE OR REPLACE FUNCTION bytemart_lab.sales.gst_amount(net NUMBER(12,2), rate NUMBER(5,4))
RETURNS NUMBER(12,2)
AS 'ROUND(net * rate, 2)';

SELECT order_id, revenue, bytemart_lab.sales.gst_amount(revenue, 0.18) AS gst
FROM bytemart_lab.sales.order_revenue LIMIT 3;
```

SQL UDFs are inlined into the calling query and are usually the fastest UDF type. Mark a UDF `IMMUTABLE` when it always returns the same output for the same input; `MEMOIZABLE` caches results of scalar UDFs with no or constant arguments (useful for lookups such as "current FX table").

SQL table function (UDTF)

```
CREATE OR REPLACE FUNCTION bytemart_lab.sales.orders_for_customer(p_customer NUMBER)
RETURNS TABLE (order_id NUMBER, order_ts TIMESTAMP_NTZ, revenue NUMBER(12,2))
AS
$$
    SELECT order_id, order_ts, revenue
    FROM bytemart_lab.sales.order_revenue
    WHERE customer_id = p_customer
$$;
```

```
SELECT * FROM TABLE(bytemart_lab.sales.orders_for_customer(1));
```

A UDTF is a **parameterised view**. Combined with `LATERAL` it can be applied per row of another table.

Python scalar UDF

```
CREATE OR REPLACE FUNCTION bytemart_lab.sales.normalize_phone(phone VARCHAR, country VARCHAR)
RETURNS VARCHAR
LANGUAGE PYTHON
RUNTIME_VERSION = '3.12'
HANDLER = 'normalize'
AS
$$
import re

PREFIX = {"IN": "91", "UK": "44", "US": "1"}

def normalize(phone, country):
    if phone is None:
        return None
    digits = re.sub(r"\D", "", phone)
    prefix = PREFIX.get(country, "")
    if prefix and not digits.startswith(prefix):
        digits = prefix + digits.lstrip("0")
    return "+" + digits
$$;

SELECT bytemart_lab.sales.normalize_phone('098450 12345', 'IN');   -- +919845012345
```

Module-level code (like `PREFIX`) runs once per Python process and is reused across rows — load expensive resources (models, lookup files) there, not inside the handler.

Vectorized Python UDF

A vectorized UDF receives a **batch of rows as a pandas DataFrame** and returns a pandas Series, which is much faster for numeric work than row-at-a-time calls.

```
CREATE OR REPLACE FUNCTION bytemart_lab.sales.price_incl_tax_vec(price FLOAT, rate FLOAT)
RETURNS FLOAT
LANGUAGE PYTHON
RUNTIME_VERSION = '3.12'
PACKAGES = ('pandas')
HANDLER = 'compute'
AS
$$
import pandas
from _snowflake import vectorized

@vectorized(input=pandas.DataFrame)
def compute(df):
    # columns are positional: 0 = price, 1 = rate
    return (df[0] * (1 + df[1])).round(2)
$$;
```

Python UDTF

```
CREATE OR REPLACE FUNCTION bytemart_lab.sales.split_sku(sku VARCHAR)
RETURNS TABLE (part_no NUMBER, part VARCHAR)
LANGUAGE PYTHON
RUNTIME_VERSION = '3.12'
HANDLER = 'SkuSplitter'
AS
$$
class SkuSplitter:
    def process(self, sku):
        for i, part in enumerate(sku.split("-")):
            yield (i + 1, part)
$$;

SELECT p.sku, s.part_no, s.part
FROM bytemart_lab.sales.products p,
     TABLE(bytemart_lab.sales.split_sku(p.sku)) s
WHERE p.product_id = 101;
```

Expected: three rows for `CHG-USBC-65W`: (1, CHG), (2, USBC), (3, 65W). A Python UDTF handler class implements `process` (called per input row, yielding output rows) and optionally `end_partition` (called once per partition — use with `OVER (PARTITION BY …)` for per-group calculations).

Secure UDFs

`CREATE SECURE FUNCTION` hides the function body from users without ownership and prevents optimisations that could leak data — required when sharing functions and advisable for functions that implement access logic.

10.10 Calling external services: external access integrations

UDFs and procedures cannot reach the internet by default. To call an external API — exchange rates, address validation, a model endpoint — an administrator creates a controlled path:

```
-- 1. Which hosts may be reached
CREATE OR REPLACE NETWORK RULE bytemart_lab.ops.fx_api_rule
  MODE = EGRESS TYPE = HOST_PORT VALUE_LIST = ('api.fx-provider.example');

-- 2. The credential, stored as a secret (never in code)
CREATE OR REPLACE SECRET bytemart_lab.ops.fx_api_key
  TYPE = GENERIC_STRING SECRET_STRING = '<api-token>';

-- 3. The integration tying them together (account-level; administrator)
CREATE OR REPLACE EXTERNAL ACCESS INTEGRATION fx_api_eai
  ALLOWED_NETWORK_RULES = (bytemart_lab.ops.fx_api_rule)
  ALLOWED_AUTHENTICATION_SECRETS = (bytemart_lab.ops.fx_api_key)
  ENABLED = TRUE;

-- 4. A function that uses them
CREATE OR REPLACE FUNCTION bytemart_lab.ops.get_fx_rate(currency VARCHAR)
RETURNS FLOAT
LANGUAGE PYTHON
RUNTIME_VERSION = '3.12'
PACKAGES = ('requests')
```

```
HANDLER = 'get_rate'
EXTERNAL_ACCESS_INTEGRATIONS = (fx_api_eai)
SECRETS = ('api_key' = bytemart_lab.ops.fx_api_key)
AS
$$
import _snowflake
import requests

def get_rate(currency):
    token = _snowflake.get_generic_secret_string("api_key")
    r = requests.get(f"https://api.fx-provider.example/v1/rates/{currency}",
                     headers={"Authorization": f"Bearer {token}"}, timeout=10)
    r.raise_for_status()
    return float(r.json()["inr_per_unit"])
$$;
```

The host name and API response format are **illustrative**; adapt them to a real provider. Do not call external APIs once per row over millions of rows — fetch reference data in bulk into a table on a schedule instead, and treat API responses as **untrusted input**.

10.11 Logging and tracing

Snowflake captures log messages, trace events and metrics from procedures and UDFs into an **event table**. Configure the level with `LOG_LEVEL` and `TRACE_LEVEL` (at account, database, schema or object level), then emit messages:

```
-- Snowflake Scripting
SYSTEM$LOG_INFO('daily_sales loaded: ' || v_rows || ' rows');
SYSTEM$LOG('warn', 'No orders found for ' || p_run_date);
```

```
# Python handlers use the standard logging module
import logging
logger = logging.getLogger("bytemart.profile")
logger.info("profiling %s", table_name)
```

```
ALTER PROCEDURE bytemart_lab.ops.load_daily_sales(DATE) SET LOG_LEVEL = 'INFO';
-- Optional: automatic begin/end logging of blocks and child jobs
ALTER PROCEDURE bytemart_lab.ops.load_daily_sales(DATE) SET AUTO_EVENT_LOGGING = 'LOGGING';

SELECT timestamp,
       record['severity_text']::STRING AS level,
       value::STRING                   AS message
FROM <your_event_table>
WHERE record_type = 'LOG'
  AND resource_attributes['snow.executable.name']::STRING ILIKE 'LOAD_DAILY_SALES%'
ORDER BY timestamp DESC
LIMIT 50;
```

Replace `<your_event_table>` with the event table active for your account or database (Chapter 19 covers setup, default event tables and costs). Logging volume is billable; use INFO for milestones, DEBUG only while investigating, and never log secrets or personal data.

10.12 Testing and deployment

Testing procedural code

1. **Isolate:** test in a cloned schema (`CREATE SCHEMA test_x CLONE bytemart_lab.sales`).
2. **Arrange fixtures:** insert known inputs, including edge cases (NULLs, duplicates, empty days, future dates).
3. **Act:** `CALL` the procedure or query the UDF.
4. **Assert:** compare actual with expected using `EXCEPT` in both directions; raise an exception on mismatch so CI fails.

```
EXECUTE IMMEDIATE $$
DECLARE
    diff NUMBER;
    test_failed EXCEPTION (-20100, 'gst_amount test failed');
BEGIN
    SELECT COUNT(*) INTO :diff FROM (
        SELECT bytemart_lab.sales.gst_amount(column1, 0.18) AS got, column2 AS expected
        FROM VALUES (100.00, 18.00), (899.00, 161.82), (0.00, 0.00)
    ) WHERE got <> expected;
    IF (diff > 0) THEN RAISE test_failed; END IF;
    RETURN 'PASS';
END;
$$;
```

Deployment

- Keep all procedure, function and view definitions in **Git**, one object per file.
- Deploy with the **Snowflake CLI** (`snow sql -f …`, `snow snowpark deploy`), Snowflake **Git repository** objects with `EXECUTE IMMEDIATE FROM`, **DCM Projects** (declarative change management, GA August 2026), or Terraform/third-party tools (Chapter 11).
- Use `CREATE OR REPLACE … COPY GRANTS` (or `CREATE OR ALTER`) so redeployments do not drop privileges.
- Version behaviour, not just code: record the deployed Git commit in a comment or tag.
- Procedures are identified by **name and argument types** (overloading): `ALTER PROCEDURE load_daily_sales(DATE) …` — remember to include signatures in `GRANT`, `ALTER` and `DROP`.

10.13 Guided lab

LAB 10.1 — BUILD, BREAK AND FIX A PIPELINE PROCEDURE

Environment: Snowsight, `lab_role`, `lab_wh`, Section 8.1 data.

Step 1. Create `ops` schema, `daily_sales`, `run_log`, and `load_daily_sales` (Section 10.5).

Step 2. Load every day in February 2026 with a script:

```
EXECUTE IMMEDIATE $$
DECLARE
    d DATE DEFAULT '2026-02-01';
BEGIN
    WHILE (d <= '2026-02-28') DO
        CALL bytemart_lab.ops.load_daily_sales(:d);
        d := DATEADD(day, 1, d);
    END WHILE;
    RETURN 'done';
END;
$$;
```

Step 3. Verify: February revenue in `daily_sales` should equal 38,189.00 (the sum of non-cancelled February orders: 5,296 + 13,898 + 2,996 + 15,999).

Step 4. Break it: `CALL bytemart_lab.ops.load_daily_sales('2099-01-01');` — confirm a `REJECTED` result. Then rename `order_revenue` temporarily and call again — confirm a `FAILED` log row and an error raised to the caller. Rename it back.

Step 5. Create `profile_table` and run it against `customers` and `products`.

Step 6. Create `split_sku` and `normalize_phone`; write a unit-test block for `normalize_phone` with three cases.

10.14 Production use case: replacing a fragile external script

ByteMart ran a nightly Python script on a virtual machine that connected to Snowflake with a password stored in a configuration file, computed daily summaries row by row, and emailed a CSV. Failures were noticed only when the email did not arrive.

The replacement:

Old	New
Script on a VM with a stored password	Owner's-rights SQL procedure run by a serverless task under a service role
Row-by-row computation in Python	Set-based `DELETE` + `INSERT … SELECT` in one transaction
No logging	`run_log` audit table + event-table logs
Silent failure	Exception re-raised → task marked FAILED → alert (Chapter 19)
Emailed CSV	Dashboard on `daily_sales`; optional `COPY INTO @stage` export
Manual deployment	Git + Snowflake CLI in CI

10.15 Common mistakes

PROCEDURAL CODE PITFALLS

1. Missing or extra colon before variables in SQL vs expressions.
2. Row-by-row loops over business data.
3. Concatenating parameters into dynamic SQL in owner's-rights procedures.
4. Swallowing exceptions so tasks report success.
5. Forgetting `ROLLBACK` in handlers.
6. DDL inside a transaction that you expect to roll back.
7. Forgetting argument types in `GRANT USAGE ON PROCEDURE name(DATE)`.
8. Calling external APIs per row.
9. Hard-coding secrets in code instead of secrets objects.
10. Logging personal data.

10.16 Troubleshooting exercise

EXERCISE — 'INSUFFICIENT PRIVILEGES' ONLY WHEN THE TASK RUNS

`CALL bytemart_lab.ops.load_daily_sales('2026-02-11')` works in a worksheet, but the nightly task that calls it fails with an insufficient-privileges error on `bytemart_lab.sales.order_revenue`. The procedure is `EXECUTE AS OWNER`, owned by `lab_role`; the task is owned by `etl_role`. What are the likely causes, and how do you diagnose them? (Solution at the end of the chapter.)

10.17 Knowledge check

1. When should you choose a UDF over a stored procedure, and vice versa?
2. Where do you need a colon before a variable in Snowflake Scripting?
3. What do `SQLROWCOUNT` and `SQLID` contain?
4. What is the difference between `EXIT` and `CONTINUE` handlers?
5. What is the default `EXECUTE AS` mode, and what risk does it carry?
6. Why load expensive resources at module level in a Python UDF?
7. What does a vectorized UDF receive and return?
8. Which three objects are needed for a function to call an external API, and why?
9. Why must procedure grants include argument types?
10. Name two ways to deploy procedures from Git.

10.18 Hands-on assignment

1. Write a stored procedure `archive_old_orders(p_before DATE)` that copies orders before the date into an archive table and deletes them from the source, atomically, with logging and idempotency.
2. Write a Python UDTF that splits a full address string into components (street, city, postcode) and returns one row.
3. Write a test block for your procedure that creates fixtures in a cloned schema and asserts row counts.

10.19 Chapter summary

Snowflake offers several programming models: scalar and table UDFs for logic inside queries, vectorized Python UDFs for fast batch processing, and stored procedures for multi-step workflows — in SQL (Snowflake Scripting), Python, Java, Scala and JavaScript. Snowflake Scripting provides variables, control flow, cursors, result sets, dynamic SQL and structured exception handling with `SQLCODE` / `SQLERRM` / `SQLSTATE`. Procedures can manage their own transactions and should be idempotent, atomic, audited and fail loudly. Owner's rights (the default) enable controlled privilege escalation, caller's rights limit procedures to the caller's powers, and restricted caller's rights (Preview) narrow them further. External access integrations, network rules and secrets allow controlled API calls. Logs and traces flow into event tables; code is tested with fixtures and assertions and deployed from Git.

10.20 Key takeaways

- Prefer set-based SQL; use procedures for orchestration, UDFs for per-row logic.
- Idempotent, atomic, logged, loud: the four properties of a pipeline procedure.
- Owner's-rights procedures need the same scrutiny as admin access.
- Secrets in secrets objects; egress through integrations.
- Test procedural code like application code.

10.21 Further reading

- Snowflake Scripting developer guide: https://docs.snowflake.com/en/developer-guide/snowflake-scripting/index
- Handling exceptions: https://docs.snowflake.com/en/developer-guide/snowflake-scripting/exceptions
- CREATE PROCEDURE: https://docs.snowflake.com/en/sql-reference/sql/create-procedure
- Understanding caller's rights and owner's rights stored procedures: https://docs.snowflake.com/en/developer-guide/stored-procedure/stored-procedures-rights
- Python UDFs: https://docs.snowflake.com/en/developer-guide/udf/python/udf-python-introduction
- Vectorized Python UDFs: https://docs.snowflake.com/en/developer-guide/udf/python/udf-python-batch

- External network access: https://docs.snowflake.com/en/developer-guide/external-network-access/external-network-access-overview
- Logging from Snowflake Scripting: https://docs.snowflake.com/en/developer-guide/logging-tracing/logging-snowflake-scripting

10.22 Solutions

Troubleshooting — privileges only fail in the task. An owner's-rights procedure uses the **owner's** privileges for the statements inside it, so the task's role needs only `USAGE` on the procedure (and on its database and schema) plus whatever the *task itself* requires. Likely causes: (1) `etl_role` lacks `USAGE` on the procedure **with the correct signature** (`GRANT USAGE ON PROCEDURE bytemart_lab.ops.load_daily_sales(DATE) TO ROLE etl_role;`), or lacks `USAGE` on the `ops` schema or database; (2) the procedure was recreated with `CREATE OR REPLACE` without `COPY GRANTS`, dropping the grant; (3) the procedure is actually `EXECUTE AS CALLER` in production (a different deployed version), so `etl_role` itself needs `SELECT` on `order_revenue` and its base tables; (4) ownership of `order_revenue` changed and the owner role `lab_role` lost access. Diagnose with `SHOW GRANTS ON PROCEDURE …(DATE)`, `DESCRIBE PROCEDURE …(DATE)` (shows `execute as`), `SHOW GRANTS TO ROLE etl_role`, and the task's `TASK_HISTORY` error message and query ID.

Knowledge check answers.

1. UDF: per-row computation used inside queries. Procedure: multi-statement workflows with DDL/DML, control flow, transactions, error handling.
2. Before variables used inside embedded SQL statements (`WHERE x = :v`, `INTO :v`, `IDENTIFIER(:t)`); not in Scripting expressions such as `IF (v > 0)`.
3. `SQLROWCOUNT`: rows affected by the last DML statement. `SQLID`: query ID of the last statement.
4. `EXIT` (default) runs the handler and leaves the block; `CONTINUE` runs the handler and resumes after the failing statement.
5. Owner's rights. Callers gain the owner's privileges for the procedure's actions — privilege escalation if parameters are not validated.
6. Module-level code runs once per process and is reused for many rows, avoiding repeated expensive initialisation.
7. A pandas DataFrame of a batch of input rows; it returns a pandas Series (or array) of results.
8. A network rule (allowed hosts), a secret (credentials) and an external access integration (binding them, enabled by an administrator). They make egress explicit, auditable and credential-safe.
9. Procedures can be overloaded; Snowflake identifies them by name plus argument types.
10. Snowflake CLI in CI (`snow sql -f`, `snow snowpark deploy`); Git repository objects with `EXECUTE IMMEDIATE FROM`; DCM Projects; Terraform.

CHAPTER 11

Snowpark and Developer Tooling

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- explain what Snowpark is and how DataFrame code is pushed down to Snowflake as SQL;
- create sessions securely from a configuration file;
- write Snowpark Python transformations with filters, joins, aggregations, windows and UDFs;
- explain lazy evaluation and when work actually executes;
- unit-test Snowpark code locally with the local testing framework and pytest;
- manage Python packages and dependencies for UDFs and procedures;
- choose between Snowpark, SQL, pandas on Snowflake and external Python;
- describe Snowpark Container Services and when to use it;
- use the Snowflake CLI, Git repositories, Workspaces, notebooks, VS Code and CI/CD to build and deploy Snowflake code.

Prerequisites: basic Python (functions, lists, dictionaries); Chapters 8 and 10.

Where this leads: Chapter 16 (dbt and ELT tooling), Chapter 33 (Streamlit in Snowflake uses Snowpark sessions), Chapter 31 (CoCo assists with this tooling), Chapter 38 (environment and deployment architecture).

11.0 What was executed

The Python module and test suite in this chapter were **executed with Snowpark Python 1.55.1 in local testing mode** — Snowflake's official emulator that runs DataFrame operations without a Snowflake account — and the printed outputs are real. Code that requires a live account (session creation from a connection, SQL calls, deployment commands, Container Services) was checked against the documentation but not run.

11.1 What Snowpark is

Concept

Snowpark is a set of libraries and runtimes that let you write data-processing code in **Python, Java or Scala** and run it **inside Snowflake**, next to the data. It has two halves:

1. **The client API** — a DataFrame library. You build transformations in Python; Snowpark translates them into SQL and sends them to a warehouse.

2. **The server-side runtimes** — secure Python/Java/Scala sandboxes inside warehouses, which execute UDFs, UDTFs and stored procedures (Chapter 10).

Why it matters

Before Snowpark, Python users typically pulled data out of the warehouse into a laptop or a separate cluster, processed it, and pushed results back. That moves large volumes of data across networks, duplicates security controls and creates copies nobody governs. Snowpark keeps the computation where the data is: the same roles, masking policies and audit trail apply.

Lazy evaluation and pushdown

A Snowpark DataFrame is a **description of a query**, not data. Transformations (`filter`, `join`, `group_by`, `with_column`) only extend the plan. Nothing runs until an **action** — `collect()`, `show()`, `count()`, `to_pandas()`, `save_as_table()` — at which point Snowpark generates SQL and executes it in Snowflake.

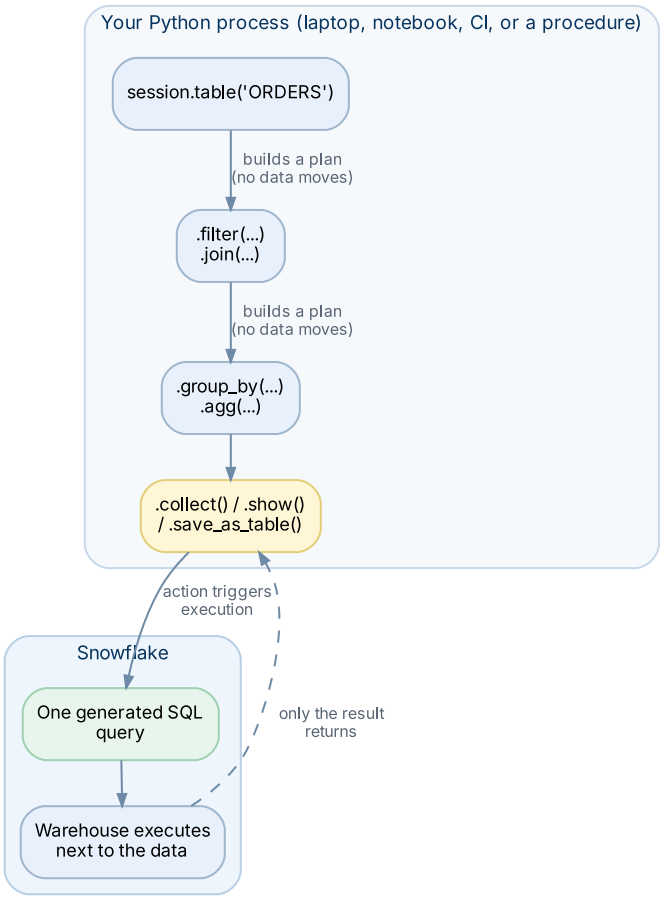


Figure 11.1 — Lazy evaluation: Python builds a plan; an action sends one SQL query to Snowflake; only results travel back.

Consequences:

- **Chained transformations become one query**, optimised by Snowflake.
- **`collect()` on a large DataFrame pulls all rows into Python memory** — use it only for small results.
- Python-side loops over rows defeat pushdown; express logic as DataFrame operations or UDFs.

- You can inspect the generated SQL with `df.queries` (live sessions) and `df.explain()`.

11.2 Setting up

Installing

```
python -m venv .venv && source .venv/bin/activate
pip install "snowflake-snowpark-python[pandas]"          # client library
pip install "snowflake-snowpark-python[localtest]"       # adds the local testing emulator
pip install snowflake-cli                                # the 'snow' command (Section 11.8)
```

Use a Python version supported by both the Snowpark client and the server runtime you deploy to; matching your local version to the procedure's `RUNTIME_VERSION` avoids surprises.

Connecting securely

Never put passwords in code. Define a named connection in `connections.toml` (in the Snowflake configuration directory, typically `~/.snowflake/` on macOS/Linux), preferably with **key-pair authentication**:

```
[connections.bytemart]
account          = "<orgname>-<account_name>"
user             = "SVC_BYTEMART_DEV"
authenticator    = "SNOWFLAKE_JWT"
private_key_file = "~/.snowflake/keys/svc_bytemart_dev.p8"
role             = "LAB_ROLE"
warehouse        = "LAB_WH"
database         = "BYTEMART_LAB"
schema           = "SALES"
```

Restrict the file's permissions (`chmod 0600`). If the key is passphrase-protected, provide the passphrase through the `PRIVATE_KEY_PASSPHRASE` environment variable. The same file is used by the Snowflake CLI, the Python connector and Snowpark.

```
from snowflake.snowpark import Session
session = Session.builder.config("connection_name", "bytemart").create()
print(session.get_current_role(), session.get_current_warehouse())
```

Inside a **stored procedure** or **notebook in Snowflake**, a session already exists: it is passed to the handler, or available via `get_active_session()`.

11.3 The ByteMart Snowpark module

The module below implements two transformations from Chapter 8 — revenue by country and the top order per customer — plus a Python UDF, and writes a result table. It runs against Snowflake (using the `bytemart` connection and the tables from Section 8.1) or, with `--local`, against the local emulator using an in-memory copy of the sample data.

```
"""ByteMart Snowpark examples (Chapter 11).
```

```
Runs against Snowflake when a connection is configured, or locally with the
Snowpark local testing emulator:  python bytemart_snowpark.py --local
"""
import sys
from datetime import datetime

from snowflake.snowpark import Session, Window
from snowflake.snowpark import functions as F
from snowflake.snowpark.types import DecimalType, IntegerType, StringType, StructField, Struc
tType, TimestampType


def get_session(local: bool) -> Session:
    if local:
        return Session.builder.config("local_testing", True).create()
    # Uses the [connections.bytemart] entry in ~/.snowflake/connections.toml
    return Session.builder.config("connection_name", "bytemart").create()


def load_sample_data(session: Session) -> None:
    """Create the sample tables (only needed in local mode; in Snowflake they exist from
Section 8.1)."""
    customers = session.create_dataframe(
        [(1, "Aarav Sharma", "IN"), (2, "Priya Nair", "IN"), (3, "Oliver Smith", "UK"),
         (4, "Amelia Jones", "UK"), (5, "Ethan Brown", "US"), (6, "Diya Patel", "IN"),
         (7, "Noah Wilson", "US"), (8, "Kavya Reddy", "IN")],
        schema=["CUSTOMER_ID", "NAME", "COUNTRY"])
    orders = session.create_dataframe(
        [(1001, 1, datetime(2026, 1, 5, 10, 15), "DELIVERED", "web"),
         (1002, 2, datetime(2026, 1, 7, 19, 42), "DELIVERED", "app"),
         (1003, 1, datetime(2026, 1, 20, 8, 5), "DELIVERED", "app"),
         (1004, 3, datetime(2026, 1, 28, 21, 10), "CANCELLED", "web"),
         (1005, 4, datetime(2026, 2, 3, 12, 0), "DELIVERED", "web"),
         (1006, 5, datetime(2026, 2, 11, 16, 30), "SHIPPED", "web"),
         (1007, 2, datetime(2026, 2, 14, 9, 20), "DELIVERED", "app"),
         (1008, 6, datetime(2026, 2, 20, 23, 55), "DELIVERED", "app"),
         (1009, 1, datetime(2026, 3, 1, 13, 45), "SHIPPED", "web"),
         (1010, 7, datetime(2026, 3, 3, 7, 10), "PENDING", "app"),
         (1011, 4, datetime(2026, 3, 8, 18, 25), "SHIPPED", "app"),
         (1012, 3, datetime(2026, 3, 9, 11, 0), "DELIVERED", "web")],
        schema=StructType([StructField("ORDER_ID", IntegerType()),
StructField("CUSTOMER_ID", IntegerType()),
                           StructField("ORDER_TS", TimestampType()), StructField("STATUS", St
ringType()),
                           StructField("CHANNEL", StringType())]))
    lines = session.create_dataframe(
        [(1001, 1, 101, 2, 899.0), (1001, 2, 104, 1, 749.0), (1002, 1, 102, 1, 12999.0),
         (1003, 1, 103, 1, 4499.0), (1003, 2, 101, 1, 899.0), (1004, 1, 106, 1, 15999.0),
         (1005, 1, 105, 2, 1899.0), (1005, 2, 104, 2, 749.0), (1006, 1, 102, 1, 12999.0),
         (1006, 2, 101, 1, 899.0), (1007, 1, 104, 4, 749.0), (1008, 1, 106, 1, 15999.0),
         (1009, 1, 105, 1, 1899.0), (1010, 1, 103, 1, 4499.0), (1011, 1, 102, 1, 12499.0),
         (1012, 1, 101, 3, 899.0), (1012, 2, 105, 1, 1899.0)],
        schema=["ORDER_ID", "LINE_NO", "PRODUCT_ID", "QUANTITY", "UNIT_PRICE"])
    customers.write.save_as_table("CUSTOMERS", mode="overwrite")
    orders.write.save_as_table("ORDERS", mode="overwrite")
    lines.write.save_as_table("ORDER_LINES", mode="overwrite")


def revenue_by_country(session: Session):
    orders = session.table("ORDERS").filter(F.col("STATUS") != "CANCELLED")
    lines = session.table("ORDER_LINES").with_column(
        "LINE_REVENUE", F.col("QUANTITY") * F.col("UNIT_PRICE"))
    customers = session.table("CUSTOMERS")
```

```
    return (lines.join(orders, "ORDER_ID")
                 .join(customers, "CUSTOMER_ID")
                 .group_by("COUNTRY")
                 .agg(F.count_distinct("ORDER_ID").alias("ORDERS"),
                      F.sum("LINE_REVENUE").alias("REVENUE"))
                 .sort(F.col("REVENUE").desc()))


def top_order_per_customer(session: Session):
    order_rev = (session.table("ORDER_LINES")
                 .group_by("ORDER_ID")
                 .agg(F.sum(F.col("QUANTITY") * F.col("UNIT_PRICE")).alias("REVENUE"))
                 .join(session.table("ORDERS"), "ORDER_ID"))
    w = Window.partition_by("CUSTOMER_ID").order_by(F.col("REVENUE").desc(),
F.col("ORDER_ID"))
    return (order_rev.with_column("RN", F.row_number().over(w))
                     .filter(F.col("RN") == 1)
                     .select("CUSTOMER_ID", "ORDER_ID", "REVENUE")
                     .sort("CUSTOMER_ID"))


def main() -> None:
    local = "--local" in sys.argv
    session = get_session(local)
    if local:
        load_sample_data(session)

    print("Revenue by country (non-cancelled):")
    revenue_by_country(session).show()

    print("Highest-revenue order per customer:")
    top_order_per_customer(session).show()

    # A Python UDF registered from the client and used inside a DataFrame expression
    @F.udf(name="price_band", replace=True, input_types=[DecimalType(12, 2)], return_type=Str
ingType())
    def price_band(revenue):
        if revenue is None:
            return None
        return "Premium" if revenue >= 10000 else ("Mid" if revenue >= 3000 else "Budget")

    banded = (session.table("ORDER_LINES")
              .group_by("ORDER_ID")
              .agg(F.sum(F.col("QUANTITY") * F.col("UNIT_PRICE")).alias("REVENUE"))
              .with_column("REVENUE", F.col("REVENUE").cast(DecimalType(12, 2)))
              .with_column("BAND", price_band(F.col("REVENUE")))
              .group_by("BAND").agg(F.count("ORDER_ID").alias("ORDERS"))
              .sort("BAND"))
    print("Orders by revenue band (Python UDF):")
    banded.show()

    # Persist a result as a table
    revenue_by_country(session).write.save_as_table("GOLD_REVENUE_BY_COUNTRY", mode="overwrit
e")
    print("Rows in GOLD_REVENUE_BY_COUNTRY:", session.table("GOLD_REVENUE_BY_COUNTRY").count(
))
    session.close()


if __name__ == "__main__":
    main()
```

Walkthrough of the key lines:

- `session.table("ORDERS")` creates a DataFrame referencing a table; no data is read yet.
- `.filter(F.col("STATUS") != "CANCELLED")` adds a `WHERE` clause to the plan.
- `.with_column("LINE_REVENUE", …)` adds a computed column.
- `.join(orders, "ORDER_ID")` joins on a column of the same name in both DataFrames (Snowpark de-duplicates the join column).
- `.group_by("COUNTRY").agg(F.count_distinct(...), F.sum(...))` builds the aggregation; `.alias()` names results. Unquoted aliases are uppercase identifiers, so rows are accessed as `r["REVENUE"]`.
- `Window.partition_by(...).order_by(...)` + `F.row_number().over(w)` is the DataFrame form of `ROW_NUMBER() OVER (…)`; filtering on it is the DataFrame form of `QUALIFY`.
- `@F.udf(...)` registers a Python function as a **temporary UDF** in Snowflake (uploaded with its dependencies) and returns a callable usable in column expressions. Add `is_permanent=True, stage_location="@stage"` to create a permanent UDF.
- `.write.save_as_table(..., mode="overwrite")` executes the plan and stores the result.

Output (local testing run):

```
Revenue by country (non-cancelled):
-----------------------------------
|"COUNTRY"  |"ORDERS"  |"REVENUE"  |
-----------------------------------
|IN         |6         |41838.0    |
|UK         |3         |22391.0    |
|US         |2         |18397.0    |
-----------------------------------

Highest-revenue order per customer:
-----------------------------------------
|"CUSTOMER_ID"  |"ORDER_ID"  |"REVENUE"  |
-----------------------------------------
|1              |1003        |5398.0     |
|2              |1002        |12999.0    |
|3              |1004        |15999.0    |
|4              |1011        |12499.0    |
|5              |1006        |13898.0    |
|6              |1008        |15999.0    |
|7              |1010        |4499.0     |
-----------------------------------------

Orders by revenue band (Python UDF):
----------------------
|"BAND"   |"ORDERS"  |
----------------------
|Budget   |3         |
|Mid      |4         |
|Premium  |5         |
----------------------

Rows in GOLD_REVENUE_BY_COUNTRY: 3
```

These numbers match the SQL results in Chapters 3 and 8 — the same logic expressed in a different API. Note that local mode prints floats (`41838.0`); against Snowflake the `NUMBER(10,2)` columns return decimals (`41838.00`).

11.4 Testing Snowpark code

The **local testing framework** emulates a large part of the DataFrame API in-process, so unit tests run in seconds without credentials or warehouse cost. It does not support `session.sql(...)`, UDTFs, vectorized UDFs, built-in table functions and some functions; objects do not persist between sessions; and only a subset of Snowflake functions is implemented — so keep a small **live** test suite too.

A common pattern is a pytest option that switches between local and live sessions:

```
# conftest.py
def pytest_addoption(parser):
    parser.addoption("--snowflake-session", action="store", default="live",
                     help="'local' for the Snowpark local testing emulator, 'live' for
Snowflake")
```

```
# test_bytemart_snowpark.py
"""Unit tests for the Chapter 11 transformations.

Local:  pytest -q test_bytemart_snowpark.py --snowflake-session local
Live:   pytest -q test_bytemart_snowpark.py          (uses the 'bytemart' connection)
"""
import pytest
from snowflake.snowpark import Session
from snowflake.snowpark import functions as F

import bytemart_snowpark as bm



@pytest.fixture(scope="module")
def session(request):
    mode = request.config.getoption("--snowflake-session", default="local")
    s = bm.get_session(local=(mode == "local"))
    if mode == "local":
        bm.load_sample_data(s)
    yield s
    s.close()


def test_revenue_by_country_totals(session):
    rows = {r["COUNTRY"]: (r["ORDERS"], float(r["REVENUE"])) for r in bm.revenue_by_country(s
ession).collect()}
    assert rows["IN"] == (6, 41838.0)
    assert rows["UK"] == (3, 22391.0)
    assert rows["US"] == (2, 18397.0)


def test_cancelled_orders_excluded(session):
    total = sum(float(r["REVENUE"]) for r in bm.revenue_by_country(session).collect())
    all_lines = session.table("ORDER_LINES").select(
        F.sum(F.col("QUANTITY") * F.col("UNIT_PRICE")).alias("T")).collect()[0]["T"]
    assert total == float(all_lines) - 15999.0   # order 1004 was cancelled


def test_one_top_order_per_customer(session):
    rows = bm.top_order_per_customer(session).collect()
    assert len(rows) == len({r["CUSTOMER_ID"] for r in rows}) == 7
```

```
def test_profile_logic(session):
    """Same logic as the profile_table procedure in Chapter 10."""
    df = session.table("CUSTOMERS")
    stats = df.select(F.sum(F.iff(F.col("COUNTRY").is_null(), 1, 0)).alias("NULLS"),
                      F.count_distinct(F.col("COUNTRY")).alias("DISTINCTS")).collect()[0]
    assert stats["NULLS"] == 0 and stats["DISTINCTS"] == 3
```

Run:

```
pytest -q test_bytemart_snowpark.py --snowflake-session local
....                                                                     [100%]
4 passed in 1.88s
```

The last test reuses the profiling logic from the `profile_table` procedure in Chapter 10 — a practical way to test procedure logic before deploying it.

> **DESIGN FOR TESTABILITY**
> Put transformations in **pure functions that take a session (or DataFrames) and return DataFrames**, as in the module above. Keep I/O (reading tables, writing results) at the edges. Then the same function runs in a test, a notebook, a procedure and a scheduled job.

11.5 Python UDFs and procedures from Snowpark

Snowpark can create server-side objects directly from Python:

```
from snowflake.snowpark.functions import sproc, udf
from snowflake.snowpark.types import FloatType, StringType

# Permanent UDF: code is uploaded to a stage and registered
@udf(name="bytemart_lab.sales.gst", is_permanent=True, stage_location="@bytemart_lab.sales.co
de",
     replace=True, input_types=[FloatType()], return_type=FloatType(),
     packages=[])
def gst(amount: float) -> float:
    return round(amount * 0.18, 2)

# Permanent stored procedure
@sproc(name="bytemart_lab.sales.rebuild_gold", is_permanent=True,
       stage_location="@bytemart_lab.sales.code", replace=True,
       packages=["snowflake-snowpark-python"])
def rebuild_gold(session) -> str:
    import bytemart_snowpark as bm   # module uploaded via session.add_import
    bm.revenue_by_country(session).write.save_as_table("GOLD_REVENUE_BY_COUNTRY", mode="overw
rite")
    return "ok"
```

`session.add_import("bytemart_snowpark.py")` uploads your own modules so procedures can import them; `session.add_packages("pandas")` or the `packages=` argument declares third-party packages. For larger projects, the **Snowflake CLI** builds and deploys code from a project definition file (Section 11.8) — preferable to registering objects from ad-hoc scripts.

11.6 Package and dependency management

Source	How	Notes
Snowflake's curated channel (Anaconda-based)	`PACKAGES = ('pandas', 'scikit-learn==1.5.2')` in DDL, or `packages=` in Snowpark	Pin versions for reproducibility; using third-party packages from this channel requires accepting the Anaconda terms in the account
PyPI via an **artifact repository**	Configure the artifact repository and reference packages	Expands available packages; Snowflake also added a `DEFAULT_PYTHON_ARTIFACT_REPOSITORY` parameter (May 2026)
Your own code	Upload `.py` or `.zip` files to a stage and use `IMPORTS = ('@stage/mylib.zip')`	Ideal for shared internal libraries
Pure-Python wheels not otherwise available	Upload to a stage and import	Native (compiled) dependencies must match the server platform

Rules of thumb: pin versions; keep dependency lists short (each package adds environment build time); prefer packages available in the curated channel for production; record the environment (`requirements.txt` or `environment.yml`) in Git alongside the code.

11.7 Choosing the right tool

Situation	Recommended	Why
Set-based transformations a SQL user can read	**SQL** (views, dynamic tables, dbt)	Most transparent; widest skill base
Complex logic better expressed in Python; reusable functions; unit tests	**Snowpark DataFrames**	Pushdown to SQL with Python ergonomics
Existing pandas code on large data	**pandas on Snowflake** (the Snowpark pandas API, a modin plugin)	pandas syntax, executed in Snowflake
Per-row Python libraries (regex, NLP, ML scoring)	**Python UDF / vectorized UDF**	Runs in parallel in the warehouse
Model training on a single large node	**Snowpark-optimized warehouse** or ML Jobs	More memory per node
Long-running service, custom runtime, GPU, web API	**Snowpark Container Services**	Full containers next to the data
Small data, interactive exploration on a laptop	Python connector → pandas (`to_pandas()`)	Fine for small extracts; governed copies matter

> **THE 'PULL EVERYTHING INTO PANDAS' ANTI-PATTERN**
>
> `session.table("ORDERS").to_pandas()` on a billion-row table transfers every row to your machine. Aggregate, filter and sample in Snowflake first; bring back only what the laptop needs.

Snowpark Container Services in one page

Snowpark Container Services (SPCS) runs your own OCI container images on Snowflake-managed compute:

Object	Purpose
Image repository	Stores container images (`CREATE IMAGE REPOSITORY`)
Compute pool	A set of VM nodes of a chosen instance family (CPU or GPU), with min/max nodes (`CREATE COMPUTE POOL`)
Service	A long-running containerised application, restarted if it exits, optionally with endpoints (`CREATE SERVICE … FROM SPECIFICATION`)
Job service	A finite batch job (`EXECUTE JOB SERVICE`)
Service function	A SQL function that sends batches of rows to a service endpoint

```
CREATE COMPUTE POOL bytemart_pool
  MIN_NODES = 1 MAX_NODES = 2
  INSTANCE_FAMILY = CPU_X64_XS
  AUTO_SUSPEND_SECS = 600;

CREATE IMAGE REPOSITORY bytemart_lab.apps.images;

CREATE SERVICE bytemart_lab.apps.recommender
  IN COMPUTE POOL bytemart_pool
  FROM SPECIFICATION $$
spec:
  containers:
  - name: api
    image: /bytemart_lab/apps/images/recommender:1.0
  endpoints:
  - name: http
    port: 8080
$$;
```

Use SPCS for workloads that do not fit warehouses: custom ML inference servers, GPU workloads, third-party applications, APIs over Snowflake data. Compute pools bill while nodes are running — configure auto-suspend and minimum nodes carefully. Openflow Snowflake deployments, Streamlit's container runtime and Native Apps with containers are built on SPCS.

11.8 Developer tooling

The Snowflake CLI (`snow`)

The **Snowflake CLI** is an open-source command-line tool for developing, deploying and managing Snowflake applications and objects. Representative commands:

Command	Purpose
`snow connection add` / `snow connection test -c bytemart` / `snow connection set-default bytemart`	Manage connections (same `connections.toml`)

Command	Purpose
`snow sql -q "SELECT 42"` / `snow sql -f deploy/001_tables.sql`	Run SQL text or files
`snow object list table --in schema bytemart_lab.sales`	Inspect objects
`snow stage copy ./data @bytemart_lab.raw.landing`	Upload/download files
`snow snowpark build` / `snow snowpark deploy`	Package and deploy UDFs/procedures defined in `snowflake.yml`
`snow streamlit deploy`	Deploy Streamlit in Snowflake apps (Chapter 33)
`snow git …`	Work with Snowflake Git repository objects
`snow dbt deploy` / `snow dbt execute`	dbt Projects on Snowflake (Chapter 16)
`snow dcm …`	DCM Projects (declarative change management)

In CI, use a **temporary connection** built from environment variables (`snow … -x` reads `SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER`, `SNOWFLAKE_PRIVATE_KEY_FILE` or `…_RAW`, etc.) rather than a file.

Git integration

Snowflake can clone a remote repository (GitHub, GitLab, Bitbucket, Azure DevOps, AWS CodeCommit) into a **Git repository object**, read its files like a stage, and execute them:

```
-- 1. Credentials (only for private repositories), stored as a secret
CREATE OR REPLACE SECRET bytemart_lab.ops.git_pat
  TYPE = PASSWORD USERNAME = '<git_user>' PASSWORD = '<personal_access_token>';

-- 2. API integration allowing access to your organisation's repositories (administrator)
CREATE OR REPLACE API INTEGRATION bytemart_git_api
  API_PROVIDER = git_https_api
  API_ALLOWED_PREFIXES = ('https://github.com/bytemart-data')
  ALLOWED_AUTHENTICATION_SECRETS = (bytemart_lab.ops.git_pat)
  ENABLED = TRUE;

-- 3. The repository clone
CREATE OR REPLACE GIT REPOSITORY bytemart_lab.ops.platform_repo
  API_INTEGRATION = bytemart_git_api
  GIT_CREDENTIALS = bytemart_lab.ops.git_pat
  ORIGIN = 'https://github.com/bytemart-data/snowflake-platform.git';

ALTER GIT REPOSITORY bytemart_lab.ops.platform_repo FETCH;
LS @bytemart_lab.ops.platform_repo/branches/main/;
EXECUTE IMMEDIATE FROM @bytemart_lab.ops.platform_repo/branches/main/deploy/run_all.sql;
```

Paths follow `@repo/branches/<branch>/…`, `@repo/tags/<tag>/…` or `@repo/commits/<sha>/…`. Repository names and URLs above are illustrative. Workspaces and dbt Projects on Snowflake can also be backed by Git.

Workspaces, notebooks and editors

Tool	What it is	Use for
Workspaces (Snowsight)	File-based editor for SQL, Python, notebooks and dbt projects; shareable; Git-connected	Day-to-day development in the browser

Tool	What it is	Use for
Notebooks in Workspaces (GA Feb 2026)	Jupyter-style notebooks running in Snowflake (the older notebooks are now "Legacy Snowflake Notebooks")	Exploration, data science, documentation of analyses
Snowflake Extension for VS Code	Run SQL, browse objects, remote development (Preview), CoCo assistant (GA Aug 2026)	Local IDE workflow
CoCo (formerly Cortex Code)	AI coding agent in Snowsight, CLI, desktop app and VS Code	Generating, explaining and refactoring code (Chapter 31)
DCM Projects (GA Aug 2026)	Declarative management of Snowflake object definitions with plan/deploy	Infrastructure-as-code for Snowflake objects
Terraform provider (third-party, maintained in the Snowflake-Labs ecosystem)	Infrastructure as code	Accounts, roles, warehouses, grants

A minimal CI/CD pipeline

```yaml
# .github/workflows/deploy.yml  (GitHub Actions; illustrative)
name: deploy-snowflake
on:
  push:
    branches: [main]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    env:
      SNOWFLAKE_ACCOUNT: ${{ secrets.SNOWFLAKE_ACCOUNT }}
      SNOWFLAKE_USER: ${{ secrets.SNOWFLAKE_DEPLOY_USER }}
      SNOWFLAKE_AUTHENTICATOR: SNOWFLAKE_JWT
      SNOWFLAKE_PRIVATE_KEY_RAW: ${{ secrets.SNOWFLAKE_DEPLOY_KEY }}
      SNOWFLAKE_ROLE: DEPLOY_ANALYTICS
      SNOWFLAKE_WAREHOUSE: DEPLOY_WH
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install "snowflake-snowpark-python[localtest]" snowflake-cli pytest
      - run: pytest -q tests --snowflake-session local        # fast unit tests, no account
needed
      - run: snow sql -x -f deploy/run_all.sql                 # deploy SQL objects
      - run: snow snowpark deploy -x --replace                 # deploy UDFs/procedures
```

Principles: a dedicated deployment **service user** with key-pair (or workload identity federation) authentication and a narrowly privileged deployment role; secrets only in the CI secret store; unit tests before deployment; deploy to dev → test → prod with the same artefacts; record the deployed commit.

11.9 Guided lab

LAB 11.1 — FROM LAPTOP TO SNOWFLAKE

Environment: local Python 3.10+; a Snowflake account with Section 8.1 data.

Step 1. Create a virtual environment and install the packages in Section 11.2.

Step 2. Save the module and tests from Sections 11.3–11.4. Run `python bytemart_snowpark.py --local` and `pytest --snowflake-session local`; compare with the outputs shown.

Step 3. Create a key pair and a service user (Chapter 20 explains the steps), add the `bytemart` connection to `connections.toml`, and run `snow connection test -c bytemart`.

Step 4. Run `python bytemart_snowpark.py` (live). Open Snowsight query history and find the SQL Snowpark generated for `revenue_by_country`. Compare it with the hand-written SQL in Chapter 8.

Step 5. Run `pytest` without `--snowflake-session local` to execute the same tests against Snowflake. If the totals differ, investigate type differences (decimal versus float).

Step 6. Deploy the UDF from Section 11.5 as a permanent function, then call it from SQL.

11.10 Production use case: ByteMart's Python transformations

ByteMart's data-science team maintained feature engineering in pandas notebooks reading full table extracts. Problems: hour-long extracts, out-of-memory errors, copies of customer data on laptops, and no tests.

Migration:

1. Feature logic rewritten as Snowpark functions returning DataFrames (pure, testable).
2. Unit tests in local mode in CI; live tests nightly in a cloned dev schema.
3. Heavy text normalisation moved into a vectorized Python UDF.
4. Training on a Snowpark-optimized warehouse; inference via a model-serving container in SPCS for low-latency recommendations.
5. Deployment through the Snowflake CLI from GitHub Actions under a deployment role.

Result: extracts eliminated; laptops no longer hold customer data; feature pipelines run in minutes and are covered by tests.

11.11 Common mistakes

SNOWPARK AND TOOLING PITFALLS

1. Calling `collect()` or `to_pandas()` on large DataFrames.
2. Python loops over rows instead of DataFrame operations or UDFs.
3. Passwords in code or notebooks.
4. Unpinned package versions in production UDFs.
5. Relying only on local testing — unsupported functions behave differently or raise.
6. Quoted mixed-case column names created from pandas (`"customerId"`).
7. Registering permanent objects from ad-hoc scripts instead of deploying from Git.
8. Compute pools without auto-suspend.

11.12 Troubleshooting exercise

EXERCISE — 'INVALID IDENTIFIER' AFTER WRITING A PANDAS DATAFRAME

An analyst writes a pandas DataFrame with columns `customerId` and `totalSpend` to Snowflake using `session.write_pandas(df, "CUSTOMER_SPEND", auto_create_table=True)`. Afterwards `SELECT customerId FROM customer_spend;` fails with an invalid identifier error. Explain and give two fixes. (Solution at the end of the chapter.)

11.13 Knowledge check

1. What is the difference between a transformation and an action in Snowpark?
2. Where does a Snowpark DataFrame join actually execute?
3. Why prefer `connections.toml` with key-pair authentication over parameters in code?
4. What does local testing mode not support?
5. When would you use a vectorized UDF instead of a DataFrame expression?
6. Name the four core Snowpark Container Services objects.
7. How do CI pipelines authenticate the Snowflake CLI without a configuration file?
8. What does `EXECUTE IMMEDIATE FROM @repo/branches/main/x.sql` do?

11.14 Hands-on assignment

1. Add a function `monthly_channel_revenue(session)` to the module that reproduces the Chapter 8 PIVOT result as a DataFrame (`pivot`), with a unit test.
2. Configure a CI workflow (any CI system) that runs local tests on every pull request.
3. Create a Git repository object for a personal repository and execute a SQL file from it.

11.15 Chapter summary

Snowpark brings Python, Java and Scala to Snowflake: a client DataFrame API whose lazily built plans are translated to SQL and executed in warehouses, and server-side runtimes for UDFs and procedures. Sessions should be created from named connections with key-pair authentication. Transformations should be pure functions that return DataFrames, unit-tested with the local testing framework and validated live. Packages are pinned and declared; your own code is imported from stages. SQL, Snowpark, pandas on Snowflake, UDFs, Snowpark-optimized warehouses and Container Services each fit different needs. The Snowflake CLI, Git repository objects, Workspaces, notebooks, the VS Code extension, CoCo, DCM Projects and CI/CD pipelines form the developer toolchain for building and deploying Snowflake code reliably.

11.16 Key takeaways

- DataFrames are plans; actions execute them in Snowflake.
- Keep data in Snowflake; bring back only results.
- Test locally, verify live, deploy from Git.
- Secrets never in code; deployment through service users and narrow roles.

11.17 Further reading

- Snowpark Developer Guide for Python: https://docs.snowflake.com/en/developer-guide/snowpark/python/index
- Creating a session: https://docs.snowflake.com/en/developer-guide/snowpark/python/creating-session
- Local testing framework: https://docs.snowflake.com/en/developer-guide/snowpark/python/testing-locally
- pandas on Snowflake: https://docs.snowflake.com/en/developer-guide/snowpark/python/pandas-on-snowflake
- Snowpark Container Services overview: https://docs.snowflake.com/en/developer-guide/snowpark-container-services/overview
- Snowflake CLI: https://docs.snowflake.com/en/developer-guide/snowflake-cli/index
- Configuring CLI connections: https://docs.snowflake.com/en/developer-guide/snowflake-cli/connecting/configure-connections
- Git repositories in Snowflake: https://docs.snowflake.com/en/developer-guide/git/git-overview

11.18 Solutions

Troubleshooting — invalid identifier. `write_pandas` with `auto_create_table=True` creates columns using the DataFrame's names **quoted**, preserving case: `"customerId"`. `SELECT customerId` is unquoted, so Snowflake looks for `CUSTOMERID`, which does not exist. Fixes: (1) query with quotes, `SELECT "customerId" FROM customer_spend` (works, but every user must remember); (2) better, normalise names before writing — `df.columns = [c.upper() for c in df.columns]` (or snake_case + upper) — and recreate the table; (3) where supported by your library version, use the option that writes unquoted identifiers. Then document the naming convention (Chapter 6).

Knowledge check answers.

1. Transformations extend the query plan without executing; actions (`collect`, `show`, `count`, `to_pandas`, `save_as_table`) generate SQL and run it.
2. In a Snowflake warehouse, as part of the generated SQL.
3. No secrets in code or notebooks; one place to manage credentials; key pairs avoid passwords and support rotation; the same file serves the CLI and connectors.

4. `session.sql`, UDTFs, vectorized UDFs, built-in table functions, some functions and data types, persistence across sessions, query history and lineage.
5. When logic needs Python libraries (text processing, ML scoring) that cannot be expressed as built-in SQL functions, over many rows — batches make it efficient.
6. Image repository, compute pool, service, job service (plus service functions).
7. A temporary connection (`-x`) built from `SNOWFLAKE_*` environment variables populated from the CI secret store.
8. Executes the SQL script stored at that path of the cloned Git repository on the main branch.

PART IV

Ingestion, Transformation, and Streaming

Roadmap level 3. Getting data in — files, events and change streams — and turning it into tested, layered, analytics-ready tables with tasks, dynamic tables and dbt.

CHAPTER 12

File Formats, Stages, and Bulk Loading

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- choose between Snowflake's loading options;
- define file formats for CSV, JSON, Parquet, Avro, ORC and XML with the options that matter;
- use user, table, named internal and external stages, and query staged files directly;
- configure storage integrations for Amazon S3, Azure and Google Cloud Storage;
- load data with `COPY INTO`, including transformations, metadata columns, validation and error handling;
- explain load metadata and duplicate-file protection;
- infer schemas and enable schema evolution;
- size files and warehouses for efficient loading;
- grant the right privileges for loading;
- build an end-to-end, idempotent, audited bulk ingestion pipeline.

Prerequisites: Chapters 2 (object storage, file formats, IAM), 6 (stages, file formats), 7 (types, VARIANT) and 8 (MERGE).

Where this leads: Chapter 13 (continuous ingestion builds on COPY), Chapter 14 (CDC), Chapter 16 (bronze layer), Chapter 25 (unstructured files and AI).

Execution environment: Snowsight and the Snowflake CLI; `lab_role` / `lab_wh`; examples checked against the documentation, not executed on Snowflake.

12.1 The loading landscape

Method	Latency	Unit	Compute	Typical use	Chapter
`INSERT` / `INSERT … SELECT`	Immediate	Rows	Warehouse	Small inserts, data already in Snowflake	8
`COPY INTO <table>`	Batch (minutes–hours)	Files	Warehouse	Scheduled bulk loads, backfills, migrations	**12**
Snowpipe	~1 minute after file arrival	Files	Serverless	Continuous micro-batch file loading	13
Snowpipe Streaming	Seconds	Rows	Serverless	Event streams, Kafka, application telemetry	13

Method	Latency	Unit	Compute	Typical use	Chapter
Openflow	Batch or streaming	Connector-defined	Openflow runtimes	Database CDC, SaaS sources, files, Kafka	13, 37
External / Iceberg tables	Query in place	Files	Warehouse	Data that stays in the lake	7, 37
Partner tools (Fivetran, Matillion, Informatica...)	Varies	Varies	Vendor + Snowflake	Managed ingestion	37
Data sharing / Marketplace	Live	Tables	Consumer's warehouse	Third-party or partner data without copying	21

This chapter covers bulk loading — the foundation on which Snowpipe and much of Openflow build.

12.2 File formats

A **file format** object tells Snowflake how to parse files. You can define options inline in `COPY` or in a reusable named object (recommended).

CSV and other delimited files

```
CREATE OR REPLACE FILE FORMAT bytemart_lab.raw.ff_products_csv
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  NULL_IF = ('', 'NULL', 'null')
  EMPTY_FIELD_AS_NULL = TRUE
  TRIM_SPACE = TRUE
  ERROR_ON_COLUMN_COUNT_MISMATCH = TRUE
  ENCODING = 'UTF8'
  DATE_FORMAT = 'AUTO'
  TIMESTAMP_FORMAT = 'AUTO'
  COMPRESSION = AUTO;
```

Option	Purpose	Advice
`FIELD_DELIMITER`, `RECORD_DELIMITER`	Separators	Tabs: `'\t'`; pipes: `'|'`
`SKIP_HEADER = n` / `PARSE_HEADER = TRUE`	Skip header rows / read column names from the header	`PARSE_HEADER` enables `MATCH_BY_COLUMN_NAME` and schema detection for CSV
`FIELD_OPTIONALLY_ENCLOSED_BY`	Quote character	Required when values contain delimiters
`ESCAPE`, `ESCAPE_UNENCLOSED_FIELD`	Escape characters	Match the producing system
`NULL_IF`, `EMPTY_FIELD_AS_NULL`	What counts as NULL	Agree with the source team
`ERROR_ON_COLUMN_COUNT_MISMATCH`	Reject rows with wrong column counts	Keep TRUE unless schema evolution needs otherwise
`ENCODING`	Character set (default UTF-8)	Set explicitly for legacy exports (e.g. `'ISO-8859-1'`)

Option	Purpose	Advice
`DATE_FORMAT`, `TIMESTAMP_FORMAT`	Parse formats	Prefer ISO 8601 at source; set explicitly for `DD/MM/YYYY`
`COMPRESSION`	`AUTO` detects gzip, bzip2, brotli, zstd, deflate...	Compress files — less transfer, faster loads

Semi-structured formats

```
CREATE OR REPLACE FILE FORMAT bytemart_lab.raw.ff_json
  TYPE = JSON
  STRIP_OUTER_ARRAY = TRUE        -- a file containing [ {...}, {...} ] becomes one row per
element
  STRIP_NULL_VALUES = FALSE       -- keep "key": null (see Chapter 7)
  COMPRESSION = AUTO;

CREATE OR REPLACE FILE FORMAT bytemart_lab.raw.ff_parquet
  TYPE = PARQUET
  USE_LOGICAL_TYPE = TRUE;        -- interpret Parquet logical types (timestamps, decimals)
```

Format	Loads into	Notes
JSON / NDJSON	One `VARIANT` per document (or per element with `STRIP_OUTER_ARRAY`)	Newline-delimited JSON loads efficiently in parallel; a single huge JSON array is slower
Parquet, ORC, Avro	A `VARIANT` per row by default, or typed columns via `MATCH_BY_COLUMN_NAME`	Self-describing; ideal for schema inference
XML	`VARIANT`	Query with `XMLGET` and `$`/`@` notation

UNSTRUCTURED FILES

PDFs, images and audio are not "loaded" into tables. They stay on a stage (ideally with a **directory table** enabled) and are referenced through the `FILE` type and functions such as `TO_FILE`, `AI_PARSE_DOCUMENT` and `AI_EXTRACT` (Chapter 25).

12.3 Stages

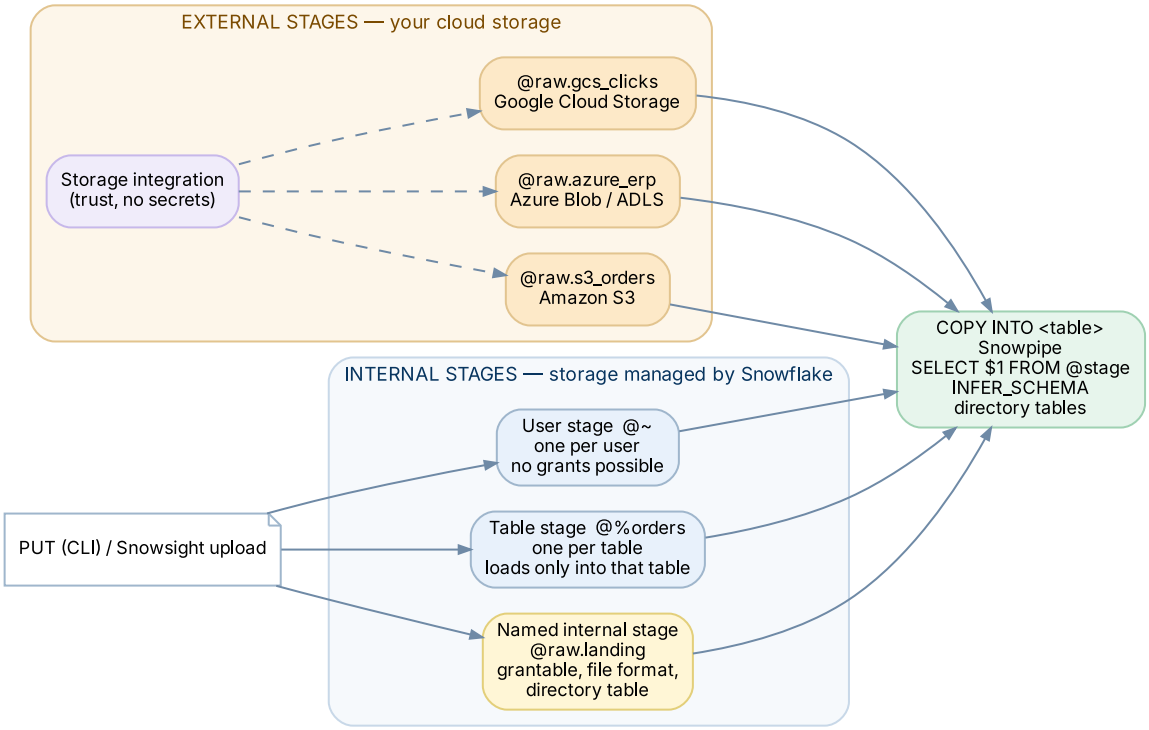


Figure 12.1 — Internal stages store files in Snowflake; external stages point to your cloud storage through a storage integration. All are read by COPY, Snowpipe and direct queries.

Internal stages

Stage	Reference	Characteristics
User stage	`@~`	Every user has one; only that user can access it; cannot be altered or granted
Table stage	`@%orders`	Every table has one; files can be loaded only into that table; no file format object
Named internal stage	`@bytemart_lab.raw.landing`	A schema object: grantable, can carry a default file format, encryption settings and a **directory table**

```
CREATE OR REPLACE STAGE bytemart_lab.raw.landing
  FILE_FORMAT = bytemart_lab.raw.ff_products_csv
  DIRECTORY = (ENABLE = TRUE)
  ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')
  COMMENT = 'Manual uploads and test files';
```

`ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')` uses server-side encryption. The default for internal stages, `SNOWFLAKE_FULL`, adds client-side encryption before upload. Server-side encryption is the recommended setting for stages whose files will be served or processed by Snowflake itself — for example unstructured files accessed through URLs, or documents passed to AI functions (Chapter 25). The encryption type cannot be changed after the stage is created.

Getting files onto an internal stage

```
# Snowflake CLI (from your machine)
snow stage copy ./data/products_20260310.csv @bytemart_lab.raw.landing/products/

# SnowSQL / CLI SQL session
PUT file:///data/products_20260310.csv @bytemart_lab.raw.landing/products/
AUTO_COMPRESS=TRUE;
```

`PUT` uploads (and by default gzip-compresses) local files; `GET` downloads. They run from client tools, not from a Snowsight worksheet — Snowsight offers an upload dialog for stages instead.

```
LIST @bytemart_lab.raw.landing/products/;
REMOVE @bytemart_lab.raw.landing/products/ PATTERN = '.*2025.*';
SELECT relative_path, size, last_modified FROM DIRECTORY(@bytemart_lab.raw.landing);
ALTER STAGE bytemart_lab.raw.landing REFRESH;      -- refresh the directory table
```

External stages and storage integrations

An **external stage** points at a location in your cloud storage. Authentication should go through a **storage integration** (Chapter 2): an account-level object representing a trust relationship, created by an administrator. No keys appear in SQL.

Amazon S3:

```
-- 1. Administrator creates the integration
CREATE OR REPLACE STORAGE INTEGRATION bytemart_s3_int
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::<aws_account_id>:role/snowflake-bytemart-read'
  STORAGE_ALLOWED_LOCATIONS = ('s3://bytemart-landing/orders/', 's3://bytemart-landing/
products/');

-- 2. Read the values Snowflake generated, and put them in the IAM role's trust policy
DESC INTEGRATION bytemart_s3_int;
--   STORAGE_AWS_IAM_USER_ARN  -> the Snowflake principal allowed to assume the role
--   STORAGE_AWS_EXTERNAL_ID   -> the external ID required in the trust policy

-- 3. Allow the loading role to use it
GRANT USAGE ON INTEGRATION bytemart_s3_int TO ROLE lab_role;

-- 4. Create the stage
CREATE OR REPLACE STAGE bytemart_lab.raw.s3_landing
  STORAGE_INTEGRATION = bytemart_s3_int
  URL = 's3://bytemart-landing/'
  FILE_FORMAT = bytemart_lab.raw.ff_products_csv;
```

On the AWS side, the IAM role needs a permissions policy allowing `s3:GetObject`, `s3:GetObjectVersion` and `s3:ListBucket` on the allowed prefixes (plus `s3:PutObject` / `s3:DeleteObject` only if Snowflake must write or purge), and a **trust policy** naming the `STORAGE_AWS_IAM_USER_ARN` principal with the `STORAGE_AWS_EXTERNAL_ID` condition. Bucket and role names here are illustrative.

Azure: `STORAGE_PROVIDER = 'AZURE'`, `AZURE_TENANT_ID = '<tenant_id>'`, allowed locations as `azure://<account>.blob.core.windows.net/<container>/<path>/`. `DESC INTEGRATION` returns a consent URL and an application name; an Azure administrator consents and grants that application a role such as *Storage Blob Data Reader* on the container.

Google Cloud Storage: `STORAGE_PROVIDER = 'GCS'`, allowed locations as `gcs://<bucket>/<path>/`. `DESC INTEGRATION` returns a Snowflake-generated service account, which you grant a role with `storage.objects.get`/`list` on the bucket.

> **NEVER CREATE STAGES WITH EMBEDDED CREDENTIALS IN PRODUCTION**
>
> `CREDENTIALS = (AWS_KEY_ID = … AWS_SECRET_KEY = …)` on a stage stores long-lived keys and spreads them to everyone who can describe or use the stage. Use storage integrations; restrict `STORAGE_ALLOWED_LOCATIONS` to exact prefixes; grant `USAGE` on the integration only to roles that create stages.

Querying staged files directly

Before loading, look at the data:

```
-- CSV: positional columns $1, $2, ...
SELECT $1, $2, $3, $4, $5, METADATA$FILENAME, METADATA$FILE_ROW_NUMBER
FROM @bytemart_lab.raw.s3_landing/products/
     (FILE_FORMAT => 'bytemart_lab.raw.ff_products_csv', PATTERN => '.*products_2026.*[.]csv[
.]gz')
LIMIT 10;

-- JSON: $1 is the VARIANT document
SELECT $1:order_id::NUMBER AS order_id, $1:status::STRING AS status
FROM @bytemart_lab.raw.s3_landing/orders/ (FILE_FORMAT => 'bytemart_lab.raw.ff_json')
LIMIT 10;
```

Schema inference

For self-describing formats (Parquet, Avro, ORC, JSON) and CSV with a header, `INFER_SCHEMA` detects column names and types, and `USING TEMPLATE` creates a table from them:

```
SELECT * FROM TABLE(INFER_SCHEMA(
    LOCATION    => '@bytemart_lab.raw.s3_landing/clicks/',
    FILE_FORMAT => 'bytemart_lab.raw.ff_parquet'));

CREATE OR REPLACE TABLE bytemart_lab.raw.click_events_typed
  USING TEMPLATE (
    SELECT ARRAY_AGG(OBJECT_CONSTRUCT(*))
    FROM TABLE(INFER_SCHEMA(
        LOCATION    => '@bytemart_lab.raw.s3_landing/clicks/',
        FILE_FORMAT => 'bytemart_lab.raw.ff_parquet')));
```

12.4 COPY INTO in depth

Basic syntax

```
COPY INTO bytemart_lab.raw.products_raw
FROM @bytemart_lab.raw.s3_landing/products/
FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_products_csv')
PATTERN = '.*products_[0-9]{8}[.]csv[.]gz'
ON_ERROR = ABORT_STATEMENT;
```

- **What to load:** a stage path, optionally narrowed by `FILES = ('a.csv.gz', 'b.csv.gz')` (explicit list, up to 1,000 files) or `PATTERN = '<regex>'`. Explicit file lists are the fastest way to identify files; patterns scan the stage listing.
- **How to parse:** `FILE_FORMAT`.
- **What to do on error:** `ON_ERROR`.

COPY with transformations

A `SELECT` over the staged files can reorder, cast, compute and add metadata during the load:

```
CREATE OR REPLACE TABLE bytemart_lab.raw.products_raw (
    product_id      VARCHAR,     -- raw layer: keep text, cast later
    sku             VARCHAR,
    product_name    VARCHAR,
    category        VARCHAR,
    unit_price      VARCHAR,
    _source_file    VARCHAR,
    _source_row     NUMBER,
    _file_modified  TIMESTAMP_LTZ,
    _loaded_at      TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
);

COPY INTO bytemart_lab.raw.products_raw
    (product_id, sku, product_name, category, unit_price, _source_file, _source_row, _file_mo
dified)
FROM (
    SELECT $1, $2, $3, $4, $5,
           METADATA$FILENAME,
           METADATA$FILE_ROW_NUMBER,
           METADATA$FILE_LAST_MODIFIED
    FROM @bytemart_lab.raw.s3_landing/products/
)
FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_products_csv')
PATTERN = '.*products_[0-9]{8}[.]csv[.]gz'
ON_ERROR = CONTINUE;
```

Transformations support column reordering, omission, casts, and a subset of functions; they do not support `VALIDATION_MODE`, `MATCH_BY_COLUMN_NAME` or column-count checking. The metadata columns `METADATA$FILENAME`, `METADATA$FILE_ROW_NUMBER`, `METADATA$FILE_LAST_MODIFIED`, `METADATA$START_SCAN_TIME` and `METADATA$FILE_CONTENT_KEY` give every row its **provenance** — essential for debugging and lineage.

Loading by column name

For Parquet, Avro, ORC, JSON and headered CSV, `MATCH_BY_COLUMN_NAME` maps file columns to table columns by name instead of position, and `INCLUDE_METADATA` adds metadata columns in the same mode:

```
COPY INTO bytemart_lab.raw.click_events_typed
FROM @bytemart_lab.raw.s3_landing/clicks/
FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_parquet')
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
INCLUDE_METADATA = (source_file = METADATA$FILENAME);
```

(The table must already have the `source_file` column.)

Error handling options

`ON_ERROR` value	Behaviour	Use when
`ABORT_STATEMENT` **(default for bulk COPY)**	Stop at the first error; load nothing from this statement	Correctness matters more than progress (finance)
`CONTINUE`	Skip bad rows, load the rest	Raw layer with reject tracking
`SKIP_FILE`	Skip any file containing an error (default for Snowpipe)	File-level atomicity
`SKIP_FILE_<n>` / `'SKIP_FILE_<n>%'`	Skip a file if errors exceed a count or percentage	Tolerate occasional noise

Other important options:

Option	Default	Meaning
`VALIDATION_MODE = RETURN_n_ROWS / RETURN_ERRORS / RETURN_ALL_ERRORS`	Not set	**Dry run**: validate and return rows or errors without loading
`FORCE`	`FALSE`	Load files even if already loaded — **can duplicate data**
`LOAD_UNCERTAIN_FILES`	`FALSE`	Load files whose load status is unknown (older than the load-metadata window)
`PURGE`	`FALSE`	Delete files from the stage after successful load
`SIZE_LIMIT`	None	Stop after roughly this many bytes have been loaded (useful for testing)
`RETURN_FAILED_ONLY`	`FALSE`	COPY output lists only files that failed
`ENFORCE_LENGTH` / `TRUNCATECOLUMNS`	`TRUE` / `FALSE`	Error on, or truncate, strings longer than the column

```
-- Dry run before the real load
COPY INTO bytemart_lab.raw.products_raw (product_id, sku, product_name, category, unit_price)
FROM @bytemart_lab.raw.s3_landing/products/
FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_products_csv')
VALIDATION_MODE = RETURN_ERRORS;
```

COPY output and load history

Each `COPY` returns one row per file: `file`, `status` (`LOADED`, `LOAD_FAILED`, `PARTIALLY_LOADED`), `rows_parsed`, `rows_loaded`, `error_limit`, `errors_seen`, `first_error`, `first_error_line`, `first_error_character`, `first_error_column_name`. Capture it:

```
COPY INTO bytemart_lab.raw.products_raw (product_id, sku, product_name, category, unit_price)
FROM @bytemart_lab.raw.s3_landing/products/
FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_products_csv')
ON_ERROR = CONTINUE
->> INSERT INTO bytemart_lab.ops.load_audit
    SELECT CURRENT_TIMESTAMP(), 'PRODUCTS', "file", "status", "rows_parsed", "rows_loaded",
           "errors_seen", "first_error"
    FROM $1;
```

The flow operator passes the COPY result to the `INSERT`; its column names are lowercase and must be quoted. (Create `ops.load_audit` with matching columns first.)

To see rejected rows of a past load:

```
SELECT * FROM TABLE(VALIDATE(bytemart_lab.raw.products_raw, JOB_ID => '_last'));
```

`VALIDATE` returns the errors encountered by a previous COPY (identified by query ID, or `'_last'` for the session's latest). History of loads is available from `INFORMATION_SCHEMA.COPY_HISTORY` (table function, recent history) and `ACCOUNT_USAGE.COPY_HISTORY` (account-wide, longer history).

```
SELECT file_name, status, row_count, row_parsed, first_error_message, last_load_time
FROM TABLE(bytemart_lab.information_schema.copy_history(
       TABLE_NAME => 'BYTEMART_LAB.RAW.PRODUCTS_RAW',
       START_TIME => DATEADD(hour, -24, CURRENT_TIMESTAMP())));
```

Load metadata: why the same file is not loaded twice

Snowflake records, **per target table**, which files have been loaded (by path/name and a content checksum (ETag/MD5)). For bulk COPY this **load metadata is kept for 64 days**. A later COPY skips files already loaded — so re-running a COPY is safe and idempotent. Consequences:

- Re-uploading **a changed file with the same name** within 64 days: by default it is considered changed if its checksum differs and is loaded again; if identical, it is skipped.
- Files **older than 64 days** whose status is unknown are skipped by default; `LOAD_UNCERTAIN_FILES = TRUE` loads them; `FORCE = TRUE` loads everything regardless (risking duplicates).
- `TRUNCATE TABLE` clears the table's load metadata (files can be reloaded); `DELETE FROM` does not.
- Snowpipe keeps its own load history for **14 days** (Chapter 13).

> **DESIGN FILE NAMES FOR IDEMPOTENCY**
>
> Name files uniquely and immutably — `orders_20260310T0100_part0003.json.gz` — and never overwrite a file with different content under the same name. Combined with load metadata and key-based MERGE downstream, this makes the whole pipeline replayable.

12.5 Schema evolution

When sources add columns, **schema evolution** lets `COPY` and Snowpipe add columns (and drop `NOT NULL` constraints) automatically:

```
ALTER TABLE bytemart_lab.raw.click_events_typed SET ENABLE_SCHEMA_EVOLUTION = TRUE;
GRANT EVOLVE SCHEMA ON TABLE bytemart_lab.raw.click_events_typed TO ROLE lab_role;
-- if not owner

COPY INTO bytemart_lab.raw.click_events_typed
FROM @bytemart_lab.raw.s3_landing/clicks/
FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_parquet')
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE;
```

Requirements and limits at the research cutoff: the table has `ENABLE_SCHEMA_EVOLUTION = TRUE`; the load uses `MATCH_BY_COLUMN_NAME`; the role has `EVOLVE SCHEMA` or `OWNERSHIP`; for CSV with `PARSE_HEADER`, `ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE`. It applies to Avro, Parquet, CSV, JSON and ORC; only COPY and Snowpipe loads evolve schemas (not `INSERT`); by default one COPY adds at most 100 columns.

> **SCHEMA EVOLUTION IS NOT SCHEMA GOVERNANCE**
> Automatically adding columns is convenient in the raw layer, but new columns must still be reviewed before they flow into silver and gold models. Pair schema evolution with a change alert (Chapter 19) and data contracts (Chapter 16).

12.6 Performance and cost of loading

- **File size:** aim for roughly **100–250 MB compressed** per file; avoid very large files (tens to hundreds of GB) and very large numbers of tiny files.
- **Parallelism:** a warehouse loads many files in parallel; the achievable parallelism cannot exceed the number of files. One giant file is loaded with limited parallelism; 10,000 tiny files spend most of their time on per-file overhead.
- **Warehouse size:** larger warehouses load more files concurrently. For a few files, an X-Small or Small warehouse is usually sufficient; size up for large backfills with many files, then size down.
- **Formats:** compressed columnar files (Parquet) and compressed delimited files load efficiently; prefer newline-delimited JSON over one huge JSON array.
- **Transformations:** simple transformations in COPY are cheap; complex logic belongs in the transformation layer.
- **Splitting large exports:** have source systems write multiple files (for example, by partition) rather than one file.

Symptom	Likely cause	Fix
Load of one 80 GB file takes hours	No parallelism	Split into ~100–250 MB files
2 million tiny files load slowly and expensively	Per-file overhead	Batch files at the source; compact before loading

Symptom	Likely cause	Fix
Most time spent listing files	Broad `PATTERN` over a huge bucket	Narrow the stage path; use dated prefixes; explicit `FILES`
Load succeeds but rows missing	`ON_ERROR = CONTINUE` hid errors	Audit COPY output; `VALIDATE`

12.7 Permissions for loading

Privilege	On	Needed for
`USAGE`	Database, schema	Everything
`USAGE`	Named file format	Using it in COPY
`READ` (and `WRITE` for PUT/REMOVE)	Named internal stage	Reading/writing files
`USAGE`	External stage	Reading files
`USAGE`	Storage integration	**Creating** external stages that use it
`CREATE STAGE`, `CREATE FILE FORMAT`	Schema	Creating these objects
`INSERT` (and `SELECT` if transforming from the table)	Target table	Loading
`EVOLVE SCHEMA`	Target table	Schema evolution (if not owner)
`USAGE`	Warehouse	Running COPY

Separate the **platform role** that creates integrations and stages from the **loading role** that only reads stages and inserts into raw tables (Chapter 20).

12.8 End-to-end bulk ingestion pipeline

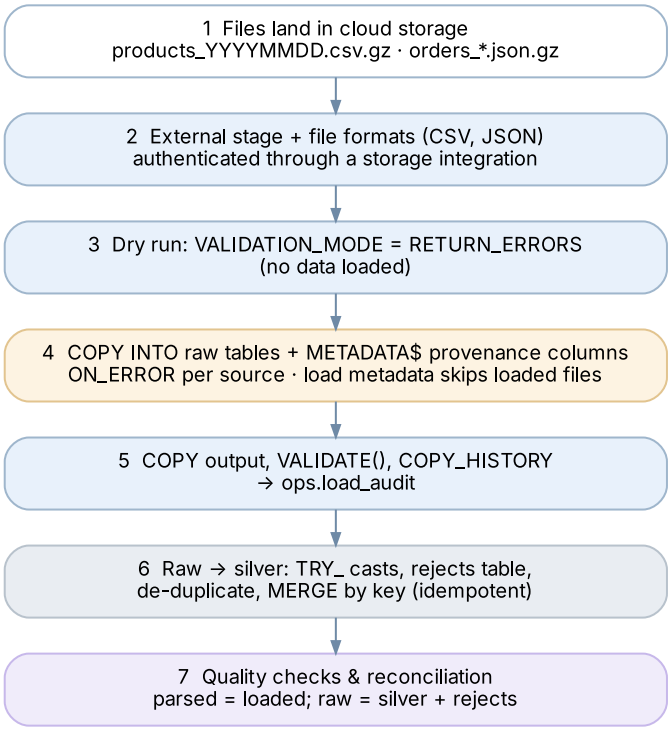


Figure 12.2 — ByteMart's bulk ingestion pipeline, from file arrival to validated silver tables.

Business requirement: the ERP exports the product catalogue as `products_YYYYMMDD.csv.gz` every night; the order service exports changed orders as newline-delimited `orders_YYYYMMDDTHHMM_partNNNN.json.gz`. Load them into raw tables with provenance, reject bad rows into a reviewable place, and publish typed silver tables idempotently.

Step 1 — Raw tables. `products_raw` as in Section 12.4 (all text + metadata). Orders as VARIANT:

```
CREATE OR REPLACE TABLE bytemart_lab.raw.orders_raw (
    doc            VARIANT,
    _source_file   VARCHAR,
    _source_row    NUMBER,
    _loaded_at     TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
);
CREATE TABLE IF NOT EXISTS bytemart_lab.ops.load_audit (
    audited_at TIMESTAMP_LTZ, source VARCHAR, file VARCHAR, status VARCHAR,
    rows_parsed NUMBER, rows_loaded NUMBER, errors_seen NUMBER, first_error VARCHAR);
```

Step 2 — Dry run and load.

```
COPY INTO bytemart_lab.raw.orders_raw (doc, _source_file, _source_row)
FROM (SELECT $1, METADATA$FILENAME, METADATA$FILE_ROW_NUMBER
      FROM @bytemart_lab.raw.s3_landing/orders/)
FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_json')
PATTERN = '.*orders_[0-9]{8}T[0-9]{4}_part[0-9]{4}[.]json[.]gz'
ON_ERROR = SKIP_FILE
->> INSERT INTO bytemart_lab.ops.load_audit
```

```
    SELECT CURRENT_TIMESTAMP(), 'ORDERS', "file", "status", "rows_parsed", "rows_loaded",
           "errors_seen", "first_error" FROM $1;
```

JSON orders use `SKIP_FILE`: a malformed file is set aside whole (a partially loaded order batch is harder to reason about); products use `CONTINUE` with row-level reject tracking.

Step 3 — Raw → silver with validation.

```
CREATE TABLE IF NOT EXISTS bytemart_lab.silver.products (
    product_id NUMBER, sku VARCHAR(30), product_name VARCHAR(100),
    category VARCHAR(40), unit_price NUMBER(10,2), _source_file VARCHAR,
    _updated_at TIMESTAMP_LTZ);

CREATE TABLE IF NOT EXISTS bytemart_lab.silver.products_rejects LIKE bytemart_lab.raw.product
s_raw;
ALTER TABLE bytemart_lab.silver.products_rejects ADD COLUMN reject_reason VARCHAR;

-- Rows that fail typing go to rejects (latest file only for brevity)
INSERT INTO bytemart_lab.silver.products_rejects
SELECT r.*, 'invalid product_id or unit_price'
FROM bytemart_lab.raw.products_raw r
WHERE r._loaded_at >= DATEADD(hour, -24, CURRENT_TIMESTAMP())
  AND (TRY_TO_NUMBER(r.product_id) IS NULL OR TRY_TO_NUMBER(r.unit_price, 10, 2) IS NULL);

-- Valid rows: de-duplicate (latest file wins) and MERGE
MERGE INTO bytemart_lab.silver.products t
USING (
    SELECT TRY_TO_NUMBER(product_id)          AS product_id,
           UPPER(TRIM(sku))                   AS sku,
           TRIM(product_name)                 AS product_name,
           INITCAP(TRIM(category))            AS category,
           TRY_TO_NUMBER(unit_price, 10, 2)   AS unit_price,
           _source_file
    FROM bytemart_lab.raw.products_raw
    WHERE TRY_TO_NUMBER(product_id) IS NOT NULL
      AND TRY_TO_NUMBER(unit_price, 10, 2) IS NOT NULL
    QUALIFY ROW_NUMBER() OVER (PARTITION BY TRY_TO_NUMBER(product_id)
                               ORDER BY _source_file DESC, _source_row DESC) = 1
) s
ON t.product_id = s.product_id
WHEN MATCHED AND NOT (EQUAL_NULL(t.sku, s.sku) AND EQUAL_NULL(t.product_name, s.product_name)
                  AND EQUAL_NULL(t.category, s.category) AND EQUAL_NULL(t.unit_price, s.unit_
price)) THEN
    UPDATE SET sku = s.sku, product_name = s.product_name, category = s.category,
               unit_price = s.unit_price, _source_file = s._source_file,
               _updated_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN
    INSERT (product_id, sku, product_name, category, unit_price, _source_file, _updated_at)
    VALUES (s.product_id, s.sku, s.product_name, s.category, s.unit_price, s._source_file,
            CURRENT_TIMESTAMP());
```

Notes: file names sort chronologically because they embed the date, so "latest file wins" can order by file name. `EQUAL_NULL` treats two NULLs as equal, so the update fires only when something really changed — without it, `t.category <> s.category` would be NULL (not TRUE) when either side is NULL, and changes to or from NULL would be missed.

Step 4 — Orders silver flattens JSON into typed columns (Chapter 8) and MERGEs by `order_id`; lines are flattened into `silver.order_lines` and replaced per order (delete-insert by `order_id`) so that edited orders do not leave stale lines.

Step 5 — Checks and reconciliation.

```
-- Every loaded file accounted for: parsed = loaded + rejected
SELECT a.file, a.rows_parsed, a.rows_loaded,
       (SELECT COUNT(*) FROM bytemart_lab.silver.products_rejects r
        WHERE r._source_file = a.file)                    AS rejected_in_silver
FROM bytemart_lab.ops.load_audit a
WHERE a.source = 'PRODUCTS' AND a.audited_at >= DATEADD(day, -1, CURRENT_TIMESTAMP());
```

Step 6 — Orchestration. Chapter 15 wraps steps 2–5 into a task graph; Chapter 13 replaces scheduled COPY with Snowpipe for orders when lower latency is needed.

12.9 Guided lab: load without a cloud account

LAB 12.1 — INTERNAL STAGE END TO END

Environment: Snowsight + Snowflake CLI on your machine. No cloud storage needed.

Step 1. Create `products_20260310.csv` locally:

```
product_id,sku,product_name,category,unit_price
101,CHG-USBC-65W,USB-C Charger 65W,electronics,899.00
102,HP-NC-700,"Noise-Cancelling Headphones, black",electronics,12999.00
103,KB-MECH-87,Mechanical Keyboard,electronics,4499.00
104,MUG-STEEL,Insulated Steel Mug,home,749.00
X05,LAMP-LED,LED Desk Lamp,home,1899.00
106,CHAIR-ERG,Ergonomic Chair,furniture,15999.00
107,CABLE-HDMI,HDMI Cable 2m,electronics,not-a-price
```

It contains a quoted value with a comma (row 102) and two bad rows (product id `X05`, price `not-a-price`).

Step 2. Create `raw` schema objects: `ff_products_csv`, stage `landing`, table `products_raw` (Sections 12.2–12.4).

Step 3. Upload: `snow stage copy products_20260310.csv @bytemart_lab.raw.landing/products/`. Run `LIST @bytemart_lab.raw.landing/products/;` and note the exact file name (it may carry a `.gz` suffix if the client compressed it); `COMPRESSION = AUTO` reads either form.

Step 4. Preview with `SELECT $1, $2, … FROM @bytemart_lab.raw.landing/products/ (FILE_FORMAT => …)`.

Step 5. Dry run with `VALIDATION_MODE = RETURN_ERRORS`. **Expected:** no errors — every value is text in the raw table, so the CSV itself is valid.

Step 6. Load with the transforming COPY. **Expected COPY output:** one file, `LOADED`, `rows_parsed = 7`, `rows_loaded = 7`.

Step 7. Run the COPY again. **Expected:** the file is skipped (`0` files processed / "Copy executed with 0 files processed") because load metadata records it.

Step 8. Run the rejects insert and the MERGE (Step 3 of Section 12.8). **Expected:** 2 rows in `products_rejects`; 5 rows in `silver.products`, with categories in initial capitals and product 102's name containing the comma.

Step 9. Edit the file (change a price), upload it with the **same name**, run COPY again, and observe whether it reloads; then explain the result using Section 12.4.

12.10 Common mistakes

LOADING PITFALLS

1. Embedded credentials in stages.
2. `ON_ERROR = CONTINUE` without auditing rejected rows.
3. Typed raw tables that reject whole files for one bad value.
4. `FORCE = TRUE` "to make it work" — duplicates.
5. Overwriting files with different content under the same name.
6. One huge file or millions of tiny files.
7. Broad patterns over huge buckets.
8. Forgetting provenance columns.
9. JSON arrays without `STRIP_OUTER_ARRAY`.
10. `PUT` attempted from a Snowsight worksheet.

12.11 Troubleshooting exercise

EXERCISE — 'COPY EXECUTED WITH 0 FILES PROCESSED'

A new team member uploads `orders_20260310T0100_part0001.json.gz` to the stage and runs the COPY. It reports zero files processed, but the file is visible with `LIST`. List the possible causes and how to check each. (Solution at the end of the chapter.)

12.12 Knowledge check

1. When would you use `COPY INTO` rather than Snowpipe?
2. What does `STRIP_OUTER_ARRAY` do?
3. Name the three kinds of internal stage and one limitation of each of the first two.
4. Why is a storage integration better than stage credentials?
5. What metadata columns can you add during COPY, and why?
6. What is the default `ON_ERROR` for bulk COPY, and for Snowpipe?
7. How long is load metadata kept for bulk COPY, and what happens after that?
8. What are the requirements for schema evolution?
9. What file size range does Snowflake recommend for loading?
10. Which privilege allows a role to create stages that use a storage integration?

12.13 Hands-on assignment

1. Complete Lab 12.1 including Step 9.
2. Extend the pipeline to orders JSON: write the flatten + MERGE into `silver.orders` and the delete-insert into `silver.order_lines`.
3. Write an `ops.load_audit` dashboard query showing files loaded per day, rows, rejects and failure rate.
4. (If you have a cloud account) Configure a storage integration and external stage for your own bucket.

12.14 Chapter summary

Bulk loading moves files from stages into tables with `COPY INTO`. File formats describe parsing; stages describe locations — user, table and named internal stages in Snowflake, external stages in your cloud storage accessed through storage integrations. You can query staged files directly, infer schemas, and enable schema evolution. COPY supports transformations, metadata columns for provenance, dry-run validation and several error strategies; its output and `VALIDATE` / `COPY_HISTORY` support auditing. Load metadata (64 days) prevents duplicate loads, making re-runs safe when files are uniquely named and immutable. Efficient loading depends on file sizing (about 100–250 MB compressed), parallelism and appropriate warehouses. A robust pipeline loads raw data with provenance, validates and rejects explicitly, MERGEs into typed silver tables idempotently, and reconciles counts end to end.

12.15 Key takeaways

- Raw layer: permissive types, provenance columns, never lose a row silently.
- Storage integrations, never keys.
- Unique, immutable file names + load metadata + MERGE = replayable pipelines.
- Size files for parallelism.
- Audit every load.

12.16 Further reading

- Overview of data loading: https://docs.snowflake.com/en/guides-overview-loading-data
- Preparing to load data (file sizing): https://docs.snowflake.com/en/user-guide/data-load-considerations-prepare
- CREATE FILE FORMAT: https://docs.snowflake.com/en/sql-reference/sql/create-file-format
- CREATE STAGE: https://docs.snowflake.com/en/sql-reference/sql/create-stage
- COPY INTO <table>: https://docs.snowflake.com/en/sql-reference/sql/copy-into-table
- Querying metadata for staged files: https://docs.snowflake.com/en/user-guide/querying-metadata
- Storage integration for S3: https://docs.snowflake.com/en/user-guide/data-load-s3-config-storage-integration

- Table schema evolution: https://docs.snowflake.com/en/user-guide/data-load-schema-evolution
- INFER_SCHEMA: https://docs.snowflake.com/en/sql-reference/functions/infer_schema

12.17 Solutions

Troubleshooting — 0 files processed. Causes and checks: (1) **Already loaded** — the same file (name and content) was loaded into this table within 64 days; check `COPY_HISTORY` for the file name. (2) **Pattern mismatch** — the regex does not match the full path (`PATTERN` matches against the path relative to the stage location, and `.` must be escaped); test with `LIST @stage PATTERN = '…'`. (3) **Wrong path** — the COPY's `FROM @stage/orders/` differs from where the file was uploaded (case-sensitive in object storage). (4) **Load status uncertain** — the file's last-modified date is older than 64 days and load metadata has expired; `LOAD_UNCERTAIN_FILES = TRUE` would load it. (5) **Different target table** — load metadata is per table; if the team member loaded into a different table, it would have loaded. (6) **Privileges** — normally produce an error, not zero files, but verify `USAGE`/`READ` on the stage. Never "fix" this with `FORCE = TRUE` until you know the file was not already loaded.

Knowledge check answers.

1. For scheduled batch loads, backfills and migrations, especially when files arrive in batches and latency of minutes to hours is acceptable, or when you want explicit control in a pipeline step.
2. Treats a top-level JSON array as many documents, loading one row per array element.
3. User (`@~`; private to the user, cannot be granted), table (`@%t`; loads only into that table, no named file format), named internal (grantable, configurable).
4. No long-lived keys in SQL; trust is managed in cloud IAM and can be revoked; allowed locations are restricted centrally.
5. `METADATA$FILENAME`, `METADATA$FILE_ROW_NUMBER`, `METADATA$FILE_LAST_MODIFIED`, `METADATA$START_SCAN_TIME`, `METADATA$FILE_CONTENT_KEY` — provenance for debugging, lineage, de-duplication and audit.
6. Bulk COPY: `ABORT_STATEMENT`. Snowpipe: `SKIP_FILE`.
7. 64 days; afterwards files with unknown status are skipped unless `LOAD_UNCERTAIN_FILES = TRUE` (or `FORCE = TRUE`).
8. `ENABLE_SCHEMA_EVOLUTION = TRUE` on the table, `MATCH_BY_COLUMN_NAME` in the load, `EVOLVE SCHEMA` or ownership for the role, and for headered CSV `ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE`.
9. Roughly 100–250 MB compressed.
10. `USAGE` on the storage integration.

CHAPTER 13

Snowpipe and Continuous Ingestion

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- explain the difference between batch, micro-batch and streaming ingestion and choose between them;
- build Snowpipe pipes with auto-ingest (S3, Azure, GCS) and the REST API;
- monitor, pause, refresh and safely recreate pipes;
- explain the Snowpipe Streaming high-performance architecture: clients, channels, offset tokens and PIPE objects;
- write a Snowpipe Streaming client in Python and reason about exactly-once delivery;
- describe the Snowflake Connector for Kafka 4.x and Openflow;
- design for late, duplicate and out-of-order events;
- estimate and control the cost of continuous ingestion.

Prerequisites: Chapter 12 (stages, file formats, COPY, load metadata), Chapter 2 (event notifications, IAM).

Where this leads: Chapter 14 (streams consume what pipes load), Chapter 15 (tasks and dynamic tables transform it), Chapter 37 (enterprise integration patterns).

Execution environment: Snowsight; Python 3.10+ for the streaming client. The SQL and Python in this chapter follow the documentation at the research cutoff and **were not executed** against a Snowflake account.

13.1 Batch, micro-batch and streaming

Pattern	Latency	Snowflake mechanism	Unit of work	Compute
Batch	Hours	`COPY INTO` in a task or orchestrator	Many files	Warehouse
Micro-batch	~1 minute	**Snowpipe**	One file at a time	Serverless
Streaming	Seconds	**Snowpipe Streaming** (SDK, Kafka connector, Openflow)	Rows	Serverless

Latency is not free. Ask: *what decision changes if the data arrives in 10 seconds rather than 15 minutes?* For ByteMart, fraud scoring and live stock levels need seconds; finance dashboards need hours. Choose the slowest pattern that meets the requirement — it is usually the simplest and cheapest.

13.2 Snowpipe

Snowpipe loads files continuously as they arrive in a stage, using Snowflake-managed compute. A **pipe** is a named object wrapping a `COPY INTO` statement.

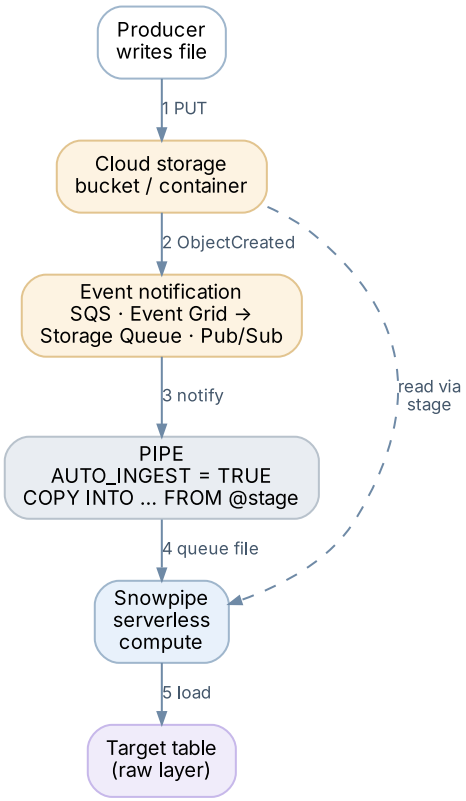


Figure 13.1 — Snowpipe auto-ingest: the cloud provider notifies Snowflake when a file lands, and the pipe loads it with serverless compute.

Auto-ingest from Amazon S3

```
CREATE OR REPLACE PIPE bytemart_lab.raw.pipe_orders
  AUTO_INGEST = TRUE
  COMMENT = 'Order JSON files from the order service'
AS
COPY INTO bytemart_lab.raw.orders_raw (doc, _source_file, _source_row)
FROM (SELECT $1, METADATA$FILENAME, METADATA$FILE_ROW_NUMBER
      FROM @bytemart_lab.raw.s3_landing/orders/)
FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_json');

SHOW PIPES LIKE 'PIPE_ORDERS' IN SCHEMA bytemart_lab.raw;
-- notification_channel column: the ARN of a Snowflake-managed SQS queue
```

Then configure an S3 **event notification** on the bucket (object-created events for the `orders/` prefix) whose destination is that SQS queue ARN. Alternatively, route events through SNS when several consumers need the same events. From now on, each new file triggers a load, typically within a minute.

Auto-ingest from Azure and Google Cloud

Azure and GCS use a **notification integration** that points at a queue Snowflake reads:

```
-- Azure: Event Grid -> Storage Queue
CREATE NOTIFICATION INTEGRATION bytemart_azure_events
  ENABLED = TRUE
  TYPE = QUEUE
  NOTIFICATION_PROVIDER = AZURE_STORAGE_QUEUE
  AZURE_STORAGE_QUEUE_PRIMARY_URI = 'https://<storage_account>.queue.core.windows.net/
<queue>'
  AZURE_TENANT_ID = '<tenant_id>';

-- Google Cloud: Pub/Sub subscription
CREATE NOTIFICATION INTEGRATION bytemart_gcs_events
  ENABLED = TRUE
  TYPE = QUEUE
  NOTIFICATION_PROVIDER = GCP_PUBSUB
  GCP_PUBSUB_SUBSCRIPTION_NAME = 'projects/<project>/subscriptions/<subscription>';

CREATE PIPE bytemart_lab.raw.pipe_orders_az
  AUTO_INGEST = TRUE
  INTEGRATION = 'BYTEMART_AZURE_EVENTS'
AS COPY INTO bytemart_lab.raw.orders_raw (doc) FROM @bytemart_lab.raw.az_landing/orders/
   FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_json');
```

As with storage integrations, `DESC NOTIFICATION INTEGRATION` returns a consent URL (Azure) or service account (GCS) that the cloud administrator must authorise.

Snowpipe REST API

Without auto-ingest, an application calls Snowpipe's REST endpoints (authenticated with key-pair JWT):

Endpoint	Purpose
`insertFiles`	Submit a list of staged file names for loading
`insertReport`	Recent load events for the pipe
`loadHistoryScan`	Load history for a time range

The REST approach suits applications that already know exactly when a file is complete (for example, after a multi-part upload finishes), or internal stages, which do not emit cloud events.

Snowpipe behaviour you must know

Topic	Snowpipe	Bulk COPY (Chapter 12)
Compute	Serverless	Your warehouse
Default `ON_ERROR`	`SKIP_FILE` (`ABORT_STATEMENT` not supported)	`ABORT_STATEMENT`
Load metadata	**Per pipe, 14 days**; a file with the same path and name is skipped **even if its content changed**	Per table, 64 days; changed content (new checksum) is reloaded
`TRUNCATE` target	Does **not** clear pipe load metadata	Clears the table's load metadata

Topic	Snowpipe	Bulk COPY (Chapter 12)
Order of loading	Generally older files first, **no guarantee**	Within one statement, not relevant
Billing	Fixed credits **per GB** loaded (text files by uncompressed size, binary formats by stored size)	Warehouse time
Changing the definition	Not editable — **recreate** the pipe (drops its load metadata)	Edit the statement

RECREATING A PIPE CAN DUPLICATE DATA

Because recreating a pipe drops its load history, any files still in the stage path can be loaded again. Follow the documented recreation procedure: pause the pipe, wait until `SYSTEM$PIPE_STATUS` shows zero pending files, recreate it, then use `ALTER PIPE … REFRESH` with care (it queues files staged in the last 7 days that the new pipe has not loaded) — or move processed files out of the path first.

Operating a pipe

```
-- Health: execution state, pending file count, last errors
SELECT PARSE_JSON(SYSTEM$PIPE_STATUS('bytemart_lab.raw.pipe_orders')) AS status;

-- Pause and resume
ALTER PIPE bytemart_lab.raw.pipe_orders SET PIPE_EXECUTION_PAUSED = TRUE;
ALTER PIPE bytemart_lab.raw.pipe_orders SET PIPE_EXECUTION_PAUSED = FALSE;

-- Queue files that are already in the stage (staged within the last 7 days)
ALTER PIPE bytemart_lab.raw.pipe_orders REFRESH PREFIX = '2026/03/10/';

-- What did the pipe load, and what failed?
SELECT file_name, status, row_count, first_error_message, pipe_received_time, last_load_time
FROM TABLE(bytemart_lab.information_schema.copy_history(
       TABLE_NAME => 'BYTEMART_LAB.RAW.ORDERS_RAW',
       START_TIME => DATEADD(hour, -6, CURRENT_TIMESTAMP())))
ORDER BY last_load_time DESC;

-- Cost
SELECT pipe_name, DATE_TRUNC('day', start_time) AS day,
       SUM(credits_used) AS credits, SUM(bytes_inserted) / POWER(1024, 3) AS gb, SUM(files_in
serted) AS files
FROM snowflake.account_usage.pipe_usage_history
WHERE start_time >= DATEADD(day, -30, CURRENT_TIMESTAMP())
GROUP BY ALL ORDER BY day DESC;
```

Error notifications. A pipe can publish failure events to a cloud messaging service through an **error integration** (`ERROR_INTEGRATION = <notification_integration>` on the pipe), so on-call engineers learn about rejected files without polling. Chapter 15 shows alerts as an alternative.

Privileges. The pipe owner needs `USAGE` on the database, schema, stage (or `READ` for internal stages) and file format, and `INSERT` + `SELECT` on the target table. Grant `MONITOR` on the pipe to operators and `OPERATE` to those who pause/resume it.

13.3 Snowpipe Streaming

Snowpipe Streaming writes **rows** directly into tables — no files, no stage — with seconds of latency. At the research cutoff there are two implementations:

	High-performance architecture (recommended)	Classic architecture
Status	GA; the default for new work	Supported for existing workloads
SDKs	`snowpipe-streaming` (Python), Java SDK, and the Kafka connector 4.x	Java `snowflake-ingest-sdk`, Kafka connector 3.x streaming mode
Server-side object	**PIPE** (default or user-defined) — enables transformations, validation, pre-clustering	Writes straight to the table
Billing	Flat, per GB ingested	Client connection time + serverless compute
Scale	Up to about 10 GB/s per table (documented target)	Lower

The rest of this section describes the high-performance architecture.

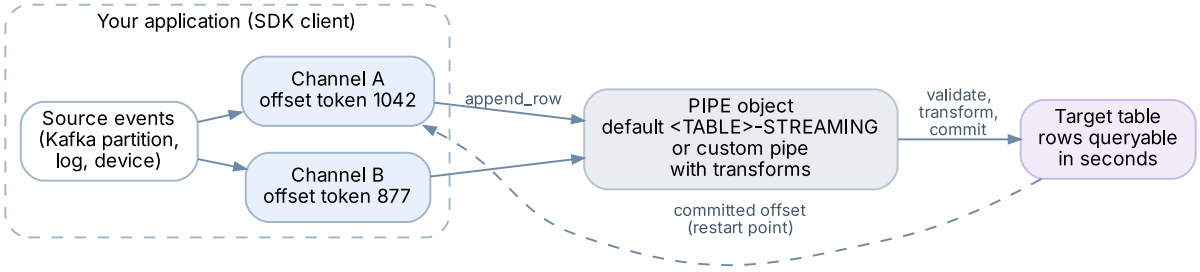


Figure 13.2 — Snowpipe Streaming: a client opens channels, appends rows with offset tokens, and a PIPE validates, transforms and commits them to the table.

Core concepts

- **Client** — your process (or the Kafka connector) authenticated with key-pair auth.
- **Channel** — an ordered, logical stream of rows into one table through one pipe. Rows within a channel are committed in order. A channel typically maps to a source partition (a Kafka partition, a device shard).
- **Offset token** — a string you attach to rows (for example, the Kafka offset). Snowflake records the **latest committed offset token per channel**. After a crash, the client asks for it and resumes from the next position — this is how you get **exactly-once** delivery.
- **PIPE object** — receives the rows. If you do not create one, Snowflake uses a **default pipe** named after the table with a `-STREAMING` suffix that maps fields to columns by name. A custom pipe lets you transform during ingestion.

A custom streaming pipe

```
CREATE OR REPLACE TABLE bytemart_lab.raw.click_events (
    event_id     STRING,
    session_id   STRING,
    customer_id  NUMBER,
```

```
    event_type   STRING,
    event_ts     TIMESTAMP_TZ,
    page         STRING,
    payload      VARIANT,
    ingested_at  TIMESTAMP_LTZ
) CLUSTER BY (TO_DATE(event_ts));

CREATE OR REPLACE PIPE bytemart_lab.raw.click_events_pipe
  CLUSTER_AT_INGEST_TIME = TRUE
AS
COPY INTO bytemart_lab.raw.click_events
FROM (
    SELECT $1:event_id::STRING,
           $1:session_id::STRING,
           $1:customer_id::NUMBER,
           LOWER($1:event_type::STRING),
           $1:event_ts::TIMESTAMP_TZ,
           $1:page::STRING,
           $1:payload,
           CURRENT_TIMESTAMP()
    FROM TABLE(DATA_SOURCE(TYPE => 'STREAMING'))
);
```

- `TABLE(DATA_SOURCE(TYPE => 'STREAMING'))` is the row source for streaming pipes; `$1` is each incoming row as an object.
- `CLUSTER_AT_INGEST_TIME = TRUE` sorts rows by the table's clustering key as they are ingested, reducing later reclustering work.
- Streaming pipes support `ON_ERROR = CONTINUE` only; invalid rows are reported through channel status and error tables rather than failing the stream.

A Python streaming client

Install with `pip install snowpipe-streaming`. Connection settings live in a `profile.json` (never in code) with `user`, `account`, `url`, `private_key_file` and `role`. The example below follows the documented API; it is **not executed** in this book.

```
"""Stream ByteMart click events into Snowflake with exactly-once restarts."""
import json
import time
from snowflake.ingest.streaming import StreamingIngestClient

client = StreamingIngestClient.from_table(          # (1)
    client_name="bytemart-clicks",
    db_name="BYTEMART_LAB",
    schema_name="RAW",
    table_name="CLICK_EVENTS",
    profile_json="profile.json",
)
# To use the custom pipe instead of the default one, create the client for the
# pipe (see the SDK documentation for the pipe-based factory method).

def latest_committed_token(channel):
    """Return the channel's latest committed offset token, or None.

    Implement with the accessor of your SDK version (Java exposes
    getLatestCommittedOffsetToken(); the Python SDK reports the same
    information for a channel -- see the SDK reference)."""
    raise NotImplementedError
```

```
channel, status = client.open_channel("clicks-partition-0", "0")   # (2)
last = latest_committed_token(channel)                              # (3)
start = int(last) + 1 if last else 0

def read_events(start_offset):                          # stand-in for Kafka / a log reader
    for offset in range(start_offset, start_offset + 1000):
        yield offset, {
            "event_id": f"e-{offset}", "session_id": f"s-{offset // 20}",
            "customer_id": 1 + offset % 12, "event_type": "VIEW",
            "event_ts": "2026-03-10T10:00:00+05:30", "page": "/product/102",
            "payload": {"referrer": "search"},
        }

for offset, event in read_events(start):
    channel.append_row(event, str(offset))              # (4)

channel.wait_for_commit(                                # (5)
    lambda token: token is not None and int(token) >= start + 999,
    timeout_seconds=60,
)
print(channel.get_channel_status())                     # (6)
channel.close()
client.close()
```

1. `StreamingIngestClient.from_table` creates a client bound to a table (and its default pipe). Credentials come from `profile.json`.
2. `open_channel(name, offset_token)` opens (or reopens) a named channel. Use a stable name per source partition so that a restart reattaches to the same channel.
3. **Restart point.** Before appending, read the channel's last committed offset token and resume after it. The sample leaves this as a small function to fill in because this book could not confirm the Python accessor's exact name at the research cutoff (the documented Java method is `getLatestCommittedOffsetToken`). If the source cannot replay from an offset, you cannot achieve exactly-once — only at-least-once.
4. `append_row(row, offset_token)` buffers a row; the SDK batches and sends asynchronously.
5. `wait_for_commit` blocks until the committed offset satisfies the predicate. Call it at checkpoints or shutdown — **not after every row** — or throughput collapses.
6. Channel status reports committed offsets and row error counts; export it to your monitoring.

Delivery semantics

Guarantee	How it happens	What you must do
At-most-once	Client fails after reading but before append; no replay	Avoid
At-least-once	Client replays from its own checkpoint, which may be behind Snowflake's commit	Make downstream idempotent (MERGE on event key)
Exactly-once	Client resumes from Snowflake's committed offset token per channel	Monotonic offsets per channel; one writer per channel; replayable source

Even with exactly-once ingestion, the *source* may emit duplicates (a mobile app retrying a request). Keep a natural `event_id` and de-duplicate in silver.

13.4 Kafka, Openflow and other paths

Snowflake Connector for Kafka 4.x

The Kafka connector is a Kafka Connect sink. **Version 4.0 became GA on 20 April 2026** and is rebuilt on the high-performance Snowpipe Streaming architecture: exactly-once, ordered delivery per partition; flat per-GB pricing; server-side validation, transformations and schema evolution through PIPE objects; pre-clustering; error tables for invalid records and a dead-letter queue for converter errors; automatic detection of Iceberg target tables. Version 3.x supported both Snowpipe (file-based) and classic Snowpipe Streaming modes; a documented migration path moves offsets from 3.x streaming channels to 4.x.

By default the connector writes two columns per row — `RECORD_METADATA` (topic, partition, offset, timestamps, key, headers) and `RECORD_CONTENT` (the message) — unless schematization or a custom pipe maps fields to columns. Running two connector instances against the same topics and table can create duplicates; one channel per topic partition is the model.

Openflow

Openflow is Snowflake's managed integration service, based on **Apache NiFi**. It runs connectors and custom data flows on runtimes deployed either in Snowflake (Snowpark Container Services) or in your own cloud account (BYOC). It covers sources that are not files: relational database CDC, SaaS applications, document repositories (useful for AI pipelines in Part VII), and streaming sources. Openflow writes into Snowflake using Snowpipe Streaming among other mechanisms. Connector availability and status vary individually — check the connector catalogue for each source. Chapter 37 covers Openflow architecture and operations in depth.

Choosing an ingestion path

Situation	Recommended path
Daily/hourly exports to a bucket	Scheduled `COPY` (task)
Files arriving continuously, minute latency	Snowpipe auto-ingest
Events already in Kafka	Kafka connector 4.x
Application emits events directly, seconds latency	Snowpipe Streaming SDK
Database CDC (PostgreSQL, MySQL, SQL Server…) or SaaS sources	Openflow connector, or a partner tool
Data that should stay in the lake	Iceberg tables / external tables (Chapter 7)
Partner or third-party data	Data sharing / Marketplace (Chapter 21)

13.5 Designing for real-world event streams

Late and out-of-order events. Store both the **event time** (when it happened) and the **ingestion time** (when Snowflake received it). Aggregate by event time but process by ingestion time: a downstream job picks up rows ingested since its last watermark, then recomputes the affected event-time windows (for example, the last 3 days of daily aggregates). Chapter 15's dynamic tables handle much of this automatically.

Duplicates. Keep a natural event key; de-duplicate in silver with `QUALIFY ROW_NUMBER() OVER (PARTITION BY event_id ORDER BY ingested_at) = 1` or MERGE on `event_id`.

Schema drift. Land a `payload VARIANT` alongside typed core columns, so new fields are never lost, and promote them to columns deliberately.

Ordering. Ordering is guaranteed only within a channel (or Kafka partition). Never assume global ordering across partitions; use event time and sequence numbers.

Retries and recovery.

Failure	What Snowflake does	What you do
Malformed file (Snowpipe)	Skips the file (`SKIP_FILE`), records the error in `COPY_HISTORY`	Fix and re-stage under a **new name**, or load it with a one-off `COPY`
Missed notifications	Nothing — Snowpipe never hears about the file	`ALTER PIPE … REFRESH` (last 7 days); older files via explicit `COPY … FILES = (…)`
Streaming client crash	Uncommitted rows are lost; committed offsets are durable	Restart, reopen the channel, resume after the committed offset token
Transient network/API errors (SDK)	The SDK retries internally	Monitor channel status and error counts; alert on growth
Invalid rows (streaming)	Rejected rows reported (error tables / channel status)	Route to a quarantine process; fix the producer
Kafka connector restart	Resumes from committed offsets per partition (4.x)	Keep one connector instance per topic set; watch the DLQ
Bad data already loaded	—	Repair with Time Travel (`AT/BEFORE`, Chapter 8) or reprocess from bronze; downstream MERGE must be idempotent

Backpressure and bursts. Snowpipe and Snowpipe Streaming scale serverlessly, but downstream tasks and dynamic tables must keep up. Monitor lag (Chapter 15) and source-side buffers.

13.6 Cost and monitoring

Mechanism	Main cost driver	Controls
Snowpipe	GB loaded	Avoid re-staging the same data; compress text files; filter at source
Snowpipe Streaming (HP)	GB ingested	Send only needed fields; avoid duplicate producers
Kafka connector 4.x	GB ingested (+ your Kafka Connect cluster)	Topic selection, SMT filtering
Openflow	Runtime compute + ingestion	Right-size runtimes; schedule batch connectors
Downstream	Warehouses/serverless used to transform	Incremental processing (Chapters 14–15)

Monitor with `SYSTEM$PIPE_STATUS`, `COPY_HISTORY`, `ACCOUNT_USAGE.PIPE_USAGE_HISTORY`, channel status from the SDK, and freshness checks on target tables (`MAX(ingested_at)`) wired to alerts.

13.7 Guided lab

LAB 13.1 — SNOWPIPE ON AN INTERNAL STAGE WITH THE REST-FREE REFRESH PATTERN

Goal: observe pipe behaviour without configuring cloud notifications. **Environment:** Snowsight + Snowflake CLI.

Step 1. Create a pipe without auto-ingest on the internal stage from Lab 12.1:

```
CREATE OR REPLACE PIPE bytemart_lab.raw.pipe_products_lab
AS COPY INTO bytemart_lab.raw.products_raw (product_id, sku, product_name, category, unit_pri
ce)
   FROM @bytemart_lab.raw.landing/pipe_products/
   FILE_FORMAT = (FORMAT_NAME = 'bytemart_lab.raw.ff_products_csv');
```

Step 2. Upload two CSV files (copies of the Lab 12.1 file with different names) to `@bytemart_lab.raw.landing/pipe_products/`.

Step 3. Queue them: `ALTER PIPE bytemart_lab.raw.pipe_products_lab REFRESH;` **Expected:** the statement returns one row per queued file with status `SENT`.

Step 4. Wait a minute, then check `SYSTEM$PIPE_STATUS` (`pendingFileCount` should return to 0) and `COPY_HISTORY` for the table. **Expected:** two files `Loaded`, 7 rows each.

Step 5. Upload a **modified** file with the **same name** as one already loaded and refresh again. **Expected:** it is skipped — Snowpipe tracks path and name, not content. Contrast with Lab 12.1 Step 9.

Step 6. Upload a file with a broken row (an unterminated quote). **Expected:** the whole file is skipped (`SKIP_FILE`) and `COPY_HISTORY` shows the first error.

Step 7. Clean up: `DROP PIPE bytemart_lab.raw.pipe_products_lab;`

13.8 Production use case: ByteMart clickstream

Requirement: 50 million click events per day from the web and mobile apps, queryable within 10 seconds for a live "trending products" panel and for fraud features, retained for 13 months.

Design. Apps publish to Kafka (already used by the platform team). The Kafka connector 4.x writes into `raw.click_events` through the custom pipe above (typed core columns + `payload VARIANT`, ingestion timestamp, clustering by event date at ingest). One channel per partition; exactly-once offsets. A dynamic table (Chapter 15) with a 1-minute target lag de-duplicates by `event_id` and builds sessionized silver data (Chapter 9's sessionization logic). A second dynamic table aggregates trending products over the last 30 minutes. A freshness alert fires if `MAX(ingested_at)` is older than 5 minutes. Data older than 13 months is deleted by a monthly task.

Why not Snowpipe? Minute-level file latency and the cost of producing many small files from the apps. **Why not the SDK directly?** Kafka was already the integration backbone, and the connector removes custom code.

13.9 Common mistakes

CONTINUOUS INGESTION PITFALLS

1. Streaming when hourly batch would meet the requirement.
2. Re-uploading corrected files with the same name to a Snowpipe path (silently skipped).
3. Recreating pipes without the documented procedure (duplicates).
4. Assuming Snowpipe loads files in order.
5. Calling `wait_for_commit` after every row.
6. Two writers on the same channel, or non-monotonic offset tokens.
7. No natural event key, so duplicates cannot be removed.
8. No freshness monitoring — pipes fail quietly when notifications stop.
9. Producing thousands of tiny files per minute for Snowpipe.
10. Hard-coding private keys in client code.

13.10 Troubleshooting exercise

EXERCISE — 'THE PIPE STOPPED LOADING'

Since 02:00 no new orders have appeared in `raw.orders_raw`. Files are present in S3. `SYSTEM$PIPE_STATUS` shows `executionState: RUNNING` and `pendingFileCount: 0`. What do you check, in what order? (Solution at the end of the chapter.)

13.11 Knowledge check

1. What is the latency target of Snowpipe, and how is it billed?
2. How does Snowpipe decide that a file has already been loaded, and for how long?
3. Why might recreating a pipe duplicate data?
4. What do Azure and GCS need that S3 auto-ingest does not?
5. What is a channel in Snowpipe Streaming, and what ordering does it guarantee?
6. How do offset tokens enable exactly-once delivery?
7. What does the default streaming pipe do, and why create a custom one?
8. Name four capabilities introduced by the Kafka connector 4.0.
9. What is Openflow built on, and where can its runtimes run?
10. Why store both event time and ingestion time?

13.12 Hands-on assignment

1. Complete Lab 13.1.

2. Write a monitoring query that returns, for each pipe in `bytemart_lab.raw`, files loaded and failed in the last 24 hours, credits used, and minutes since the last successful load.
3. (Optional, with a trial account and Python) Create `click_events` and run the streaming client against the default pipe; kill it mid-run and restart it; prove no duplicates using `COUNT(*)` vs `COUNT(DISTINCT event_id)`.

13.13 Chapter summary

Snowflake offers batch (`COPY`), micro-batch (Snowpipe) and streaming (Snowpipe Streaming) ingestion. Snowpipe loads files continuously with serverless compute, triggered by cloud event notifications or the REST API; it skips files by path and name for 14 days, defaults to `SKIP_FILE`, does not guarantee order, bills per GB, and must be recreated carefully. Snowpipe Streaming's high-performance architecture ingests rows through channels and PIPE objects with seconds of latency, flat per-GB pricing and exactly-once delivery via committed offset tokens. The Kafka connector 4.x and Openflow build on these mechanisms for Kafka, CDC and SaaS sources. Real event streams need event keys, event and ingestion timestamps, idempotent downstream processing and freshness monitoring.

13.14 Key takeaways

- Pick the slowest ingestion that meets the business need.
- Snowpipe dedup is by file name for 14 days — never reuse names.
- Exactly-once = replayable source + monotonic offsets + resume from committed token.
- Always land an event key, event time and ingestion time.
- Monitor freshness, not just failures.

13.15 Further reading

- Snowpipe introduction: https://docs.snowflake.com/en/user-guide/data-load-snowpipe-intro
- Automating Snowpipe for Amazon S3: https://docs.snowflake.com/en/user-guide/data-load-snowpipe-auto-s3
- Snowpipe REST API: https://docs.snowflake.com/en/user-guide/data-load-snowpipe-rest-apis
- Snowpipe Streaming high-performance — getting started: https://docs.snowflake.com/en/user-guide/snowpipe-streaming/snowpipe-streaming-high-performance-getting-started
- Kafka connector overview: https://docs.snowflake.com/en/user-guide/kafka-connector-overview
- Kafka connector 4.0 GA release note: https://docs.snowflake.com/release-notes/2026/other/2026-04-20-kafka-connector-v4-ga
- Openflow: https://docs.snowflake.com/en/user-guide/data-integration/openflow/about

13.16 Solutions

Troubleshooting — the pipe stopped loading. `RUNNING` with zero pending files means Snowflake is not receiving notifications. Check, in order: (1) `COPY_HISTORY` — any loads or failures after 02:00? Failures point to data, absence points to notifications. (2) `SYSTEM$PIPE_STATUS` fields such as `lastReceivedMessageTimestamp` and any `error` — when did the last notification arrive? (3) The S3 event notification configuration — was it changed or overwritten (S3 allows only one configuration per prefix/event combination; a new configuration for another consumer can replace it)? (4) Whether the file path still matches the pipe's stage path (a producer change from `orders/` to `Orders/`). (5) The IAM role and storage integration (a revoked trust policy surfaces as errors in pipe status). Recovery: fix the notification, then `ALTER PIPE … REFRESH` (files from the last 7 days) and verify counts; for older files use a one-off `COPY` with an explicit file list.

Knowledge check answers.

1. Typically within a minute of being notified; fixed credits per GB loaded.
2. By file path and name, in per-pipe load metadata kept for 14 days; content changes are not considered.
3. Recreation drops the pipe's load metadata, so files still in the path can be loaded again (for example via `REFRESH`).
4. A notification integration (Azure Storage Queue via Event Grid, or a GCS Pub/Sub subscription) referenced by `INTEGRATION` in the pipe.
5. An ordered, named stream of rows from a client into one table via one pipe; rows within a channel are committed in order.
6. Snowflake stores the latest committed offset token per channel; after failure the client resumes after that token, so nothing is skipped or written twice.
7. It maps incoming fields to columns by name; a custom pipe adds transformations, casts, computed columns and ingest-time clustering.
8. Any four of: high-performance Snowpipe Streaming, exactly-once ordered delivery, per-GB pricing, server-side validation/transformations/schema evolution via pipes, pre-clustering, error tables, DLQ, Iceberg auto-detection, offset migration from 3.x.
9. Apache NiFi; in Snowflake (Snowpark Container Services) or in your cloud account (BYOC).
10. Event time is needed for correct business aggregation; ingestion time for incremental processing, latency monitoring and late-arrival handling.

CHAPTER 14

Change Data Capture and Streams

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- explain change data capture (CDC) patterns and when each applies;
- create standard, append-only and insert-only streams and read their metadata columns;
- explain exactly when a stream's offset advances, and why one stream per consumer;
- apply stream changes to a target with the canonical MERGE — and explain why the naive MERGE fails;
- use the `CHANGES` clause for ad-hoc change queries;
- prevent, detect and recover from stale streams;
- apply CDC events from external databases idempotently, including out-of-order and replayed events;
- design incremental processing that is safe to re-run.

Prerequisites: Chapter 8 (MERGE, Time Travel), Chapter 9 (watermarks, SCD2), Chapter 13 (ingestion).

Where this leads: Chapter 15 (tasks consume streams; dynamic tables automate incremental refresh), Chapter 16 (incremental dbt models), Chapter 37 (CDC from enterprise databases).

Execution environment. Snowflake stream objects cannot run outside Snowflake. In this chapter, the **stream statements are documentation-verified, not executed**. The **MERGE and CDC logic was executed on DuckDB 1.5.6**, using a table that holds exactly the rows a Snowflake stream would return for the given changes (labelled "simulated"). Because DuckDB accepts `METADATA$…` as column names, the MERGE statements shown are the same text you run in Snowflake.

14.1 Change data capture patterns

Pattern	How changes are found	Strengths	Weaknesses
Full reload	Reload everything each run	Simple, always correct	Expensive; no history
Timestamp / watermark	`WHERE updated_at > :last_watermark` (Chapter 9)	Simple, portable	Misses deletes; depends on reliable timestamps; late updates
Snapshot diff	Compare today's snapshot with yesterday's	Catches deletes	Expensive for large tables
Log-based CDC (source DB)	Read the database transaction log (Openflow CDC connectors, Debezium, partner tools)	Complete, low impact on source, ordered	Infrastructure; event semantics to handle

Pattern	How changes are found	Strengths	Weaknesses
Snowflake streams	Snowflake tracks changes to a Snowflake table	Built-in, transactional, includes deletes	Only for changes made *inside* Snowflake
Dynamic tables	Snowflake computes incremental refresh for you	Declarative	Less control (Chapter 15)

In practice, log-based CDC brings changes *into* Snowflake (Section 14.6), and streams, tasks and dynamic tables propagate them *through* Snowflake's layers.

14.2 Streams

A **stream** is a schema object that records an **offset** — a point in the transactional history of a source object — and, when queried, returns the rows that changed between that offset and now, plus metadata columns. A stream stores no table data; it uses the source's version history (the same machinery as Time Travel).

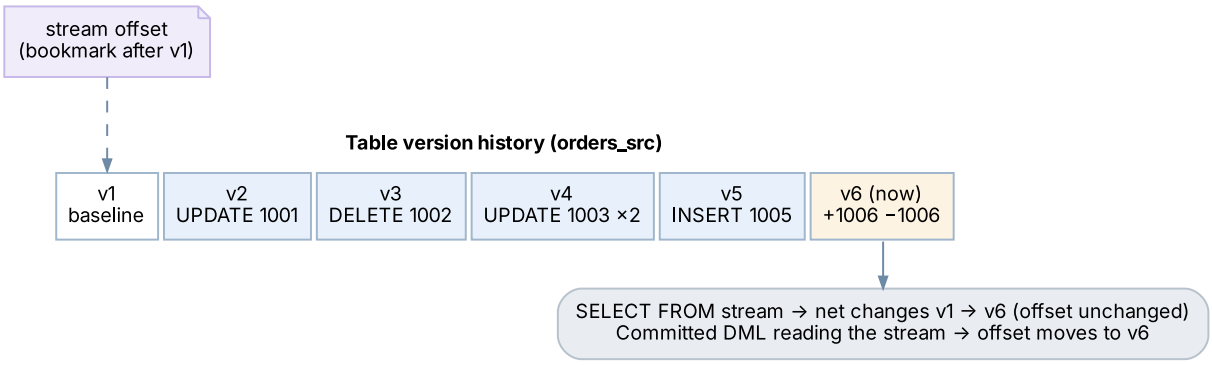


Figure 14.1 — A stream is a bookmark in a table's change history. Querying it returns the net changes since the bookmark; consuming it in a committed DML statement moves the bookmark forward.

Stream types

Type	Syntax	Tracks	Source objects (at the research cutoff)
Standard (delta)	default	Inserts, updates, deletes — **net** changes	Standard tables, dynamic tables, Snowflake-managed and externally managed V3 Iceberg tables, directory tables, views
Append-only	`APPEND_ONLY = TRUE`	Inserts only	Standard tables, dynamic tables, Snowflake-managed Iceberg tables, views
Insert-only	`INSERT_ONLY = TRUE`	Inserts only	External tables, externally managed Iceberg tables, Delta Direct tables without partition columns

```
-- Standard stream: full change tracking
CREATE OR REPLACE STREAM bytemart_lab.raw.orders_stream
  ON TABLE bytemart_lab.raw.orders_src
  COMMENT = 'Changes to orders for the silver replica';

-- Append-only: cheaper and simpler for insert-only sources (event logs)
CREATE OR REPLACE STREAM bytemart_lab.raw.click_events_new
  ON TABLE bytemart_lab.raw.click_events
  APPEND_ONLY = TRUE;
```

```
-- Insert-only on an external table
CREATE OR REPLACE STREAM bytemart_lab.raw.lake_orders_new
  ON EXTERNAL TABLE bytemart_lab.raw.lake_orders
  INSERT_ONLY = TRUE;

-- Start the stream with the table's existing rows included
CREATE OR REPLACE STREAM bytemart_lab.raw.orders_stream_init
  ON TABLE bytemart_lab.raw.orders_src
  SHOW_INITIAL_ROWS = TRUE;

-- Start from a point in the past (within Time Travel)
CREATE OR REPLACE STREAM bytemart_lab.raw.orders_stream_since
  ON TABLE bytemart_lab.raw.orders_src
  AT (TIMESTAMP => '2026-03-10 09:00:00'::TIMESTAMP_LTZ);
```

Creating the first stream on a table enables **change tracking** on it (adding hidden change-tracking columns). Streams on **views** require change tracking on every underlying table (`ALTER TABLE … SET CHANGE_TRACKING = TRUE`) and support only simple views: projections, filters, inner or cross joins and `UNION ALL` — not `GROUP BY`, `QUALIFY`, `DISTINCT`, `LIMIT` or correlated subqueries.

Metadata columns

Column	Meaning
`METADATA$ACTION`	`INSERT` or `DELETE`
`METADATA$ISUPDATE`	`TRUE` when the row is one half of an update: an update appears as a `DELETE` of the old values and an `INSERT` of the new values
`METADATA$ROW_ID`	A unique, immutable identifier for the source row, stable across its changes

Net changes. A standard stream returns the *net* effect between the offset and now. If a row was updated twice, you see one DELETE (original values) and one INSERT (latest values). If a row was inserted and then deleted within the interval, you see nothing.

Worked example: what a stream returns

ByteMart replicates operational orders into a silver table. The baseline has already been replicated:

```
CREATE TABLE orders_src (
    order_id     INTEGER PRIMARY KEY,
    customer_id  INTEGER,
    status       VARCHAR,
    amount       DECIMAL(10,2),
    updated_at   TIMESTAMP
);
INSERT INTO orders_src VALUES
    (1001, 1, 'PLACED',  1299.00, TIMESTAMP '2026-03-10 09:00:00'),
    (1002, 2, 'PLACED',   899.00, TIMESTAMP '2026-03-10 09:05:00'),
    (1003, 3, 'PLACED',  4499.00, TIMESTAMP '2026-03-10 09:10:00'),
    (1004, 4, 'SHIPPED',  749.00, TIMESTAMP '2026-03-10 08:00:00');

CREATE TABLE orders_replica AS SELECT * FROM orders_src;   -- already in sync
```

The stream is created now (in Snowflake: `CREATE STREAM orders_stream ON TABLE orders_src;`), and then these changes happen on the source:

```
UPDATE orders_src SET status = 'SHIPPED', updated_at = TIMESTAMP '2026-03-10 10:00:00'
WHERE order_id = 1001;
DELETE FROM orders_src WHERE order_id = 1002;
UPDATE orders_src SET status = 'PAID',    updated_at = TIMESTAMP '2026-03-10 10:05:00'
WHERE order_id = 1003;
UPDATE orders_src SET status = 'SHIPPED', updated_at = TIMESTAMP '2026-03-10 10:20:00'
WHERE order_id = 1003;
INSERT INTO orders_src VALUES (1005, 5, 'PLACED', 15999.00, TIMESTAMP '2026-03-10 10:30:00');
INSERT INTO orders_src VALUES (1006, 6, 'PLACED',   499.00, TIMESTAMP '2026-03-10 10:40:00');
DELETE FROM orders_src WHERE order_id = 1006;
```

Querying the stream returns:

```
SELECT order_id, status, amount, METADATA$ACTION, METADATA$ISUPDATE, METADATA$ROW_ID
FROM orders_stream
ORDER BY order_id, METADATA$ACTION;
```

Result (simulated on DuckDB from Snowflake's documented stream semantics):

order_id	status	amount	METADATA$ACTION	METADATA$ISUPDATE	METADATA$ROW_ID
1001	PLACED	1299.00	DELETE	TRUE	rid-1001
1001	SHIPPED	1299.00	INSERT	TRUE	rid-1001
1002	PLACED	899.00	DELETE	FALSE	rid-1002
1003	PLACED	4499.00	DELETE	TRUE	rid-1003
1003	SHIPPED	4499.00	INSERT	TRUE	rid-1003
1005	PLACED	15999.00	INSERT	FALSE	rid-1005

6 rows.

Read it carefully:

- **1001** (one update) → a DELETE/INSERT pair with `ISUPDATE = TRUE`.
- **1002** (deleted) → a single DELETE with `ISUPDATE = FALSE`.
- **1003** (updated twice) → still one pair: original values out, latest values in. The intermediate `PAID` state is invisible — streams give net changes, not a full event history.
- **1005** (inserted) → a single INSERT.
- **1006** (inserted then deleted) → nothing.
- Real `METADATA$ROW_ID` values are opaque strings; `rid-…` stands in for them here.

When does the offset advance?

This is the most important rule about streams:

1. **Querying** a stream (`SELECT`) never advances the offset. You can look as often as you like.
2. The offset advances when the stream is read by a **DML statement** (`INSERT`, `MERGE`, `UPDATE`, `DELETE`, CTAS, `COPY INTO <location>`) **and that transaction commits**. If the transaction rolls back, the offset does not move — the changes are offered again next time.
3. Within an explicit transaction, every statement sees the **same** stream contents (repeatable read), so you can read one stream into several targets atomically:

```
BEGIN;
  INSERT INTO bytemart_lab.silver.orders_change_log
      SELECT *, CURRENT_TIMESTAMP() FROM bytemart_lab.raw.orders_stream;
  MERGE INTO bytemart_lab.silver.orders_replica t USING (...stream...) s ON ...;
COMMIT;   -- both statements saw the same changes; the offset advances once
```

1. A stream has a **single offset**. If two consumers read the same stream, whichever commits first "takes" the changes and the other never sees them. **Create one stream per consumer.**

14.3 Applying stream changes with MERGE

The naive MERGE — and why it is wrong

A first attempt maps DELETE rows to deletes and INSERT rows to upserts:

```
MERGE INTO orders_replica t
USING orders_stream s
ON t.order_id = s.order_id
WHEN MATCHED AND s.METADATA$ACTION = 'DELETE' THEN DELETE
WHEN MATCHED AND s.METADATA$ACTION = 'INSERT' THEN
    UPDATE SET status = s.status, amount = s.amount, updated_at = s.updated_at
WHEN NOT MATCHED AND s.METADATA$ACTION = 'INSERT' THEN
    INSERT (order_id, customer_id, status, amount, updated_at)
    VALUES (s.order_id, s.customer_id, s.status, s.amount, s.updated_at);
```

```
SELECT * FROM orders_replica ORDER BY order_id;
```

Result (executed on DuckDB):

order_id	customer_id	status	amount	updated_at
1004	4	SHIPPED	749.00	2026-03-10 08:00:00
1005	5	PLACED	15999.00	2026-03-10 10:30:00

2 rows.

Orders **1001 and 1003 have disappeared** although they still exist in the source. Each update contributes *two* source rows that match the same target row: the DELETE half fires `DELETE`, the INSERT half fires `UPDATE`. DuckDB applied both and the row vanished. **Snowflake, by default, refuses such a statement** with a "duplicate row detected during DML action" error, because one target row is matched by several source rows with conflicting actions (`ERROR_ON_NONDETERMINISTIC_MERGE = TRUE`). Either way the pipeline is broken: silently wrong on one engine, failing every run on the other.

The canonical MERGE

The fix is to **drop the DELETE half of updates** before merging. What remains is exactly one row per changed key: an INSERT (new or updated row) or a DELETE (genuinely deleted row).

```
MERGE INTO orders_replica t
USING (
    SELECT *
    FROM orders_stream
    WHERE NOT (METADATA$ACTION = 'DELETE' AND METADATA$ISUPDATE)   -- (1)
```

```
) s
ON t.order_id = s.order_id                                          -- (2)
WHEN MATCHED AND s.METADATA$ACTION = 'DELETE' THEN                  -- (3)
    DELETE
WHEN MATCHED AND s.METADATA$ACTION = 'INSERT' THEN                  -- (4)
    UPDATE SET customer_id = s.customer_id, status = s.status,
               amount = s.amount, updated_at = s.updated_at
WHEN NOT MATCHED AND s.METADATA$ACTION = 'INSERT' THEN              -- (5)
    INSERT (order_id, customer_id, status, amount, updated_at)
    VALUES (s.order_id, s.customer_id, s.status, s.amount, s.updated_at);
```

1. Remove the "before image" of updates. `METADATA$ISUPDATE` is a BOOLEAN, so it can be used directly as a condition.
2. Match on the **business key**. (Matching on `METADATA$ROW_ID` is possible if you store it in the target, but the business key is clearer and survives re-creation of the source.)
3. A real delete removes the replica row.
4. An insert for an existing key is an update.
5. An insert for a new key is an insert. A DELETE for a key the target does not have is ignored (no `WHEN NOT MATCHED … DELETE` exists).

```
SELECT * FROM orders_replica ORDER BY order_id;
```

Result (executed on DuckDB):

order_id	customer_id	status	amount	updated_at
1001	1	SHIPPED	1299.00	2026-03-10 10:00:00
1003	3	SHIPPED	4499.00	2026-03-10 10:20:00
1004	4	SHIPPED	749.00	2026-03-10 08:00:00
1005	5	PLACED	15999.00	2026-03-10 10:30:00

4 rows.

Prove that the replica equals the source, in both directions:

```
(SELECT 'missing in replica' AS issue, * FROM (SELECT * FROM orders_src EXCEPT SELECT * FROM
orders_replica))
UNION ALL
(SELECT 'extra in replica', * FROM (SELECT * FROM orders_replica EXCEPT SELECT * FROM orders_
src));
```

Result (executed on DuckDB):

issue	order_id	customer_id	status	amount	updated_at

0 rows.

Zero rows: the replica is identical to the source. Make this reconciliation a scheduled data-quality check (Chapter 19).

Keeping a change log in the same transaction

Many teams also keep every change for audit. In Snowflake, run both statements inside one transaction so they consume the same stream contents:

```
CREATE TABLE orders_change_log AS
SELECT order_id, status, amount, updated_at,
       METADATA$ACTION AS change_action, METADATA$ISUPDATE AS is_update,
       CASE WHEN METADATA$ACTION = 'INSERT' AND METADATA$ISUPDATE THEN 'UPDATE'
            WHEN METADATA$ACTION = 'INSERT' THEN 'INSERT'
            WHEN NOT METADATA$ISUPDATE THEN 'DELETE'
       END AS change_type,
       TIMESTAMP '2026-03-10 11:00:00' AS captured_at
FROM orders_stream;
```

```
SELECT order_id, change_type, status, captured_at
FROM orders_change_log
WHERE change_type IS NOT NULL       -- the before-images of updates have no change_type
ORDER BY order_id;
```

Result (executed on DuckDB):

order_id	change_type	status	captured_at
1001	UPDATE	SHIPPED	2026-03-10 11:00:00
1002	DELETE	PLACED	2026-03-10 11:00:00
1003	UPDATE	SHIPPED	2026-03-10 11:00:00
1005	INSERT	PLACED	2026-03-10 11:00:00

4 rows.

14.4 The CHANGES clause

For ad-hoc questions ("what changed in this table since 9 o'clock?") you do not need a stream. If change tracking is enabled on the table, the `CHANGES` clause returns the same shape of result for any interval within Time Travel, without an offset to manage:

```
ALTER TABLE bytemart_lab.raw.orders_src SET CHANGE_TRACKING = TRUE;

SELECT order_id, status, METADATA$ACTION, METADATA$ISUPDATE
FROM bytemart_lab.raw.orders_src
  CHANGES (INFORMATION => DEFAULT)                -- or APPEND_ONLY
  AT (TIMESTAMP => '2026-03-10 09:30:00'::TIMESTAMP_LTZ)
  END (TIMESTAMP => '2026-03-10 11:00:00'::TIMESTAMP_LTZ);
```

`CHANGES` is read-only and stateless: perfect for investigations and backfills; streams are for repeated, exactly-once consumption.

14.5 Staleness and recovery

A stream's offset must stay within the source table's data retention period. If the stream is not consumed for too long, the history it points to is gone and the stream becomes **stale** — it can no longer return changes.

- To protect unconsumed streams, Snowflake **temporarily extends** the source's retention (when it is under 14 days) up to `MAX_DATA_EXTENSION_TIME_IN_DAYS` (default **14** days). This extension costs storage.

- `SHOW STREAMS` and `DESCRIBE STREAM` return `STALE` and `STALE_AFTER`. Monitor `STALE_AFTER` and alert well before it arrives.
- `SYSTEM$STREAM_HAS_DATA('<stream>')` returns whether the stream currently has changes (used in task `WHEN` conditions, Chapter 15). Consuming regularly — even when there is little data — keeps streams fresh.

```
SHOW STREAMS IN SCHEMA bytemart_lab.raw;
SELECT "name", "stale", "stale_after", "mode", "table_name"
FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))
WHERE "stale_after" < DATEADD(day, 3, CURRENT_TIMESTAMP());
```

Recovering from a stale stream.

1. Recreate the stream (`CREATE OR REPLACE STREAM …`). It starts from *now*; changes between the old offset and now are not available from it.
2. Repair the target for the missed interval: if Time Travel still covers it, use `CHANGES` for that interval; otherwise do a **full comparison** (the EXCEPT reconciliation above) or a full rebuild of the target, then resume incremental processing.
3. Fix the cause — a suspended task, a failing consumer, or retention set too short.

> **THINGS THAT AFFECT STREAMS**
>
> Recreating the source table with `CREATE OR REPLACE` drops its change history, which makes streams on it stale. Lowering the source's data retention can make streams stale sooner. Treat any DDL on a stream's source — replacing, swapping, cloning over it — as a change to the pipeline and test its effect on streams in a development account first.

14.6 Applying CDC events from external databases

When changes come from an operational database (via an Openflow CDC connector, Debezium on Kafka, or a partner tool), they typically arrive as **events** with an operation code and a log sequence number (LSN):

```
CREATE TABLE cdc_orders_events (
    order_id INTEGER, op CHAR(1), lsn BIGINT,
    status VARCHAR, amount DECIMAL(10,2), event_ts TIMESTAMP);
INSERT INTO cdc_orders_events VALUES
    (1001, 'U', 101, 'SHIPPED', 1299.00, TIMESTAMP '2026-03-10 10:00:00'),
    (1002, 'D', 102, NULL,         NULL, TIMESTAMP '2026-03-10 10:01:00'),
    (1003, 'U', 103, 'PAID',    4499.00, TIMESTAMP '2026-03-10 10:05:00'),
    (1003, 'U', 107, 'SHIPPED', 4499.00, TIMESTAMP '2026-03-10 10:20:00'),
    (1005, 'I', 108, 'PLACED', 15999.00, TIMESTAMP '2026-03-10 10:30:00'),
    (1006, 'I', 109, 'PLACED',   499.00, TIMESTAMP '2026-03-10 10:40:00'),
    (1006, 'D', 110, NULL,         NULL, TIMESTAMP '2026-03-10 10:41:00'),
    (1003, 'U', 103, 'PAID',    4499.00, TIMESTAMP '2026-03-10 10:05:00');  -- replayed
duplicate

CREATE TABLE cdc_orders_replica (
    order_id INTEGER PRIMARY KEY, status VARCHAR, amount DECIMAL(10,2), _lsn BIGINT);
INSERT INTO cdc_orders_replica VALUES
    (1001, 'PLACED', 1299.00, 50), (1002, 'PLACED', 899.00, 50),
    (1003, 'PLACED', 4499.00, 50), (1004, 'SHIPPED', 749.00, 50);
```

The batch contains several events per key, a replayed duplicate, and an insert-then-delete. Three rules make the apply step correct and idempotent:

1. **Collapse to the latest event per key** within the batch (highest LSN).
2. **Never go backwards**: update only if the event's LSN is newer than what the target has.
3. **Deletes of unknown keys are no-ops; inserts of deleted keys are skipped.**

```
MERGE INTO cdc_orders_replica t
USING (
    SELECT *
    FROM cdc_orders_events
    QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY lsn DESC) = 1   -- rule 1
) s
ON t.order_id = s.order_id
WHEN MATCHED AND s.op = 'D' AND s.lsn > t._lsn THEN DELETE                    -- rules 2, 3
WHEN MATCHED AND s.op IN ('I', 'U') AND s.lsn > t._lsn THEN
    UPDATE SET status = s.status, amount = s.amount, _lsn = s.lsn
WHEN NOT MATCHED AND s.op IN ('I', 'U') THEN
    INSERT (order_id, status, amount, _lsn) VALUES (s.order_id, s.status, s.amount, s.lsn);
```

```
SELECT * FROM cdc_orders_replica ORDER BY order_id;
```

Result (executed on DuckDB):

order_id	status	amount	_lsn
1001	SHIPPED	1299.00	101
1003	SHIPPED	4499.00	107
1004	SHIPPED	749.00	50
1005	PLACED	15999.00	108

4 rows.

Now a late event arrives for 1001 with an *older* LSN (a retry from the source), together with a full replay of the earlier batch:

```
INSERT INTO cdc_orders_events VALUES
    (1001, 'U', 95, 'PLACED', 1299.00, TIMESTAMP '2026-03-10 09:59:00');
```

Run the identical MERGE again:

```
SELECT * FROM cdc_orders_replica ORDER BY order_id;
```

Result (executed on DuckDB):

order_id	status	amount	_lsn
1001	SHIPPED	1299.00	101
1003	SHIPPED	4499.00	107
1004	SHIPPED	749.00	50
1005	PLACED	15999.00	108

4 rows.

Unchanged. The replay changed nothing and the stale event did not roll 1001 back to `PLACED`. That is what **idempotent** means: applying the same input twice yields the same state.

TWO REMAINING EDGE CASES

Deleted then re-inserted keys across batches: with rule 3, a delete removes the row and its `_lsn`; a much older replayed insert for that key could re-create it. If replays of old batches are possible, keep **tombstones** (a `_deleted` flag with the LSN) instead of physically deleting, and filter them out in a view. *Primary key changes* in the source appear as delete + insert of different keys and are handled naturally.

In Snowflake, `cdc_orders_events` is usually a raw table loaded by Openflow or Kafka; an **append-only stream** on it feeds this MERGE (filtered by the stream, so each event is processed once), and a task runs it when `SYSTEM$STREAM_HAS_DATA` is true (Chapter 15).

14.7 Designing incremental processing

Principle	How
One stream per consumer	Name streams after the consumer: `orders_stream__silver_replica`
Consume in one transaction	All statements that need the changes run inside `BEGIN … COMMIT`
Idempotent apply	MERGE on keys; guard on version/LSN; never blind `INSERT` of updates
Handle deletes explicitly	Hard delete, soft delete flag, or SCD2 end-dating (Chapter 9)
Reconcile	Periodic full compare or count/sum checks between source and target
Plan for rebuilds	Every incremental target must have a documented full-rebuild path
Watch staleness	Alert on `STALE_AFTER`; consume regularly
Prefer declarative when possible	Dynamic tables (Chapter 15) remove offset management entirely

Streams + SCD2. To maintain history instead of a replica, feed the filtered stream (INSERT rows with `ISUPDATE`, plain INSERTs, plain DELETEs) into Chapter 9's SCD2 MERGE: updates close the current version and insert a new one; deletes close the current version (and optionally set `is_deleted`).

14.8 Guided lab

LAB 14.1 — STREAMS IN SNOWFLAKE

Environment: Snowsight, `lab_role`, `lab_wh`. Use the tables from Section 14.2 (create them in `bytemart_lab.raw` and `bytemart_lab.silver`).

Step 1. Create `raw.orders_src` with the four baseline rows and `silver.orders_replica` as a copy. Create `raw.orders_stream ON TABLE raw.orders_src`.

Step 2. `SELECT * FROM raw.orders_stream;` **Expected:** no rows (no changes since creation).

Step 3. Apply the changes from Section 14.2. Query the stream twice. **Expected:** the six rows shown in the simulated result (with real `METADATA$ROW_ID` values), identical both times — querying does not consume.

Step 4. Run the naive MERGE. **Expected:** an error about duplicate/nondeterministic rows. Nothing changes, and the stream still has data (the failed statement did not commit).

Step 5. Run the canonical MERGE. **Expected:** "number of rows inserted 1, updated 2, deleted 1". Query the stream: **no rows** — the committed MERGE advanced the offset.

Step 6. Run the EXCEPT reconciliation. **Expected:** 0 rows.

Step 7. Create a second stream for an audit consumer, make a change, and show that consuming one stream does not affect the other.

Step 8. `SHOW STREAMS` and read `stale_after`.

14.9 Production use case: ByteMart inventory replica

Requirement: inventory levels change in the warehouse-management system (WMS, PostgreSQL) about 2,000 times per minute. Analysts and the product-availability dashboard need a replica within 2 minutes; finance needs daily snapshots of stock value; audit needs every change for 7 years.

Design.

1. An Openflow PostgreSQL CDC connector writes change events into `raw.inventory_events` (append-only).
2. An append-only stream `raw.inventory_events__replica` feeds a serverless task every minute (only when the stream has data) that runs the collapse-and-guard MERGE into `silver.inventory`.
3. A second append-only stream `raw.inventory_events__audit` feeds an audit task that copies events into `audit.inventory_changes` (retained 7 years, cheaper storage settings, restricted role).
4. A dynamic table computes daily stock value from `silver.inventory` and product costs.
5. Nightly reconciliation compares row counts and `SUM(quantity)` per warehouse between Snowflake and a WMS extract; differences raise an alert.
6. Alerts monitor stream staleness and event freshness.

Why two streams? Two consumers with different schedules and failure modes; a single shared stream would let whichever runs first "steal" the changes.

14.10 Common mistakes

CDC PITFALLS

1. The naive MERGE without filtering the DELETE half of updates.
2. Sharing one stream between consumers.
3. Reading a stream with `SELECT` and expecting it to be consumed.
4. Consuming a stream outside a transaction when several statements need the changes.
5. Assuming streams show every intermediate state (they show net changes).
6. Letting streams go stale because a task was suspended.
7. `CREATE OR REPLACE` on a source table that has streams.
8. Applying external CDC events without collapsing per key and guarding on LSN.
9. No reconciliation and no full-rebuild path.
10. Using streams where a dynamic table would be simpler.

14.11 Troubleshooting exercise

EXERCISE — THE REPLICA KEEPS LOSING ROWS

A replica built from a stream has fewer rows than its source. The MERGE runs every 5 minutes without errors. A colleague mentions that "to be safe" they also wrote a small task that runs `INSERT INTO ops.stream_audit SELECT COUNT(*) FROM raw.orders_stream` every minute. Explain what is happening and how to fix it.

14.12 Knowledge check

1. What does a stream store, and what does it return when queried?
2. Name the three stream types and one source each supports.
3. How does an update appear in a standard stream?
4. What happens in a stream if a row is inserted and deleted between two consumptions?
5. Exactly when does a stream's offset advance?
6. Why must the canonical MERGE filter out some rows, and which?
7. What is the difference between a stream and the `CHANGES` clause?
8. What makes a stream stale, and how do you detect it in advance?
9. What are the three rules for applying external CDC events idempotently?
10. Why create one stream per consumer?

14.13 Hands-on assignments

1. Complete Lab 14.1.
2. Write the SCD2 version: maintain `silver.orders_history` from the stream, closing versions on update and delete.
3. Extend Section 14.6 with tombstones (`_deleted BOOLEAN`) and a view that hides deleted rows; prove that replaying an old insert for a deleted key no longer resurrects it.

14.14 Chapter summary

Change data capture moves only what changed. Inside Snowflake, streams track changes to tables, views and other objects: standard streams return net inserts, updates (as DELETE/INSERT pairs with `METADATA$ISUPDATE`) and deletes; append-only and insert-only streams return inserts. A stream's offset advances only when it is consumed by committed DML, so a failed run is retried safely, and each consumer needs its own stream. The canonical MERGE removes the before-images of updates and applies one row per key. The `CHANGES` clause offers stateless change queries. Streams become stale when their offset falls outside retention; monitor `STALE_AFTER` and keep a full-rebuild path. CDC events from external databases are applied idempotently by collapsing to the latest event per key, guarding on sequence numbers and treating deletes carefully.

14.15 Key takeaways

- Filter `METADATA$ACTION = 'DELETE' AND METADATA$ISUPDATE` before MERGE.
- Offsets move only on committed DML.
- One consumer, one stream.
- Collapse per key, guard on LSN, reconcile always.
- Prefer dynamic tables when you do not need hand-written change logic.

14.16 Further reading

- Introduction to streams: https://docs.snowflake.com/en/user-guide/streams-intro
- CREATE STREAM: https://docs.snowflake.com/en/sql-reference/sql/create-stream
- CHANGES clause: https://docs.snowflake.com/en/sql-reference/constructs/changes
- MERGE (nondeterministic results): https://docs.snowflake.com/en/sql-reference/sql/merge
- Streams and tasks examples: https://docs.snowflake.com/en/user-guide/data-pipelines-examples

14.17 Solutions

Troubleshooting — the replica keeps losing rows. `INSERT INTO … SELECT COUNT(*) FROM raw.orders_stream` is a DML statement that reads the stream, so when it commits it **advances the stream's offset**. Every minute the audit task consumes the changes, and the MERGE (every 5 minutes) only sees the changes made since the last audit run. Fix: give the audit its own stream (or count with a plain `SELECT`, which does not consume), then repair the replica with a full comparison or rebuild, and add the EXCEPT reconciliation as a scheduled check.

Knowledge check answers.

1. An offset into the source's change history (no data copy); the net changed rows since the offset with metadata columns.
2. Standard (tables, views, dynamic tables…), append-only (tables, views…), insert-only (external tables, externally managed Iceberg tables).
3. As a DELETE row with old values and an INSERT row with new values, both with `METADATA$ISUPDATE = TRUE`.
4. Nothing appears.
5. When the stream is used in a DML statement (including CTAS and `COPY INTO <location>`) whose transaction commits.
6. Each update has two rows matching the same target row; the DELETE half of updates (`ACTION = 'DELETE' AND ISUPDATE`) must be removed so each key has one row.
7. A stream keeps an offset and supports repeated exactly-once consumption; `CHANGES` is a stateless query over an explicit interval within Time Travel (requires change tracking).

8. Its offset falls outside the source's retention (including any extension); `SHOW STREAMS` / `DESCRIBE STREAM` `STALE_AFTER`.
9. Collapse to the latest event per key; apply only events newer than the target's version; treat deletes of unknown keys as no-ops and skip inserts of deleted keys (or use tombstones).
10. Consumption advances the single offset; consumers sharing a stream would each see only part of the changes.

CHAPTER 15

Tasks, Dynamic Tables, and Orchestration

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- create scheduled, serverless and triggered tasks, and the privileges they need;
- build task graphs with dependencies, configuration, return values and finalizers;
- control overlap, retries, timeouts and failure suspension;
- build declarative pipelines with dynamic tables, choose target lags and refresh modes, and understand incremental refresh;
- use frozen regions, backfill and custom incremental dynamic tables;
- know when materialized views are the right tool;
- choose between tasks + streams, dynamic tables, materialized views and external orchestrators;
- monitor pipelines and react to failures with alerts;
- run backfills and reprocessing safely.

Prerequisites: Chapter 10 (stored procedures), Chapter 12 (COPY), Chapter 14 (streams, MERGE).

Where this leads: Chapter 16 (dbt and the medallion architecture), Chapter 17–18 (cost and performance of pipelines), Chapter 19 (data quality and observability), Chapter 29 (AI pipelines).

Execution environment: Snowsight; `lab_role` with `EXECUTE TASK` and `EXECUTE MANAGED TASK` granted (see 15.1). The SQL in this chapter is checked against the documentation at the research cutoff and **was not executed** against a Snowflake account. Business logic reused here (MERGE patterns) was executed in Chapters 9 and 14.

15.1 Tasks

A **task** runs one SQL statement — often `CALL` of a stored procedure, or a Snowflake Scripting block — on a schedule, after other tasks, or when a condition is true.

Serverless or warehouse?

	Serverless task	Warehouse task
Declared by	Omitting `WAREHOUSE`	`WAREHOUSE = <name>`

	Serverless task	Warehouse task
Compute	Snowflake-managed, auto-sized from run history	Your warehouse
Sizing controls	`USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE`, `SERVERLESS_TASK_MIN_STATEMENT_SIZE` / `MAX_STATEMENT_SIZE`, `TARGET_COMPLETION_INTERVAL`	Warehouse size
Billing	Serverless task credits for actual compute used	Warehouse credits (including idle time until auto-suspend, and the 60-second minimum)
Privilege	`EXECUTE MANAGED TASK` on account	`USAGE` on the warehouse
Good for	Short, frequent, predictable jobs; jobs that should finish by a deadline	Long jobs sharing a warehouse already running; precise control

Both kinds require the task owner's role to have `EXECUTE TASK` on the account:

```
USE ROLE ACCOUNTADMIN;
GRANT EXECUTE TASK, EXECUTE MANAGED TASK ON ACCOUNT TO ROLE lab_role;
```

Creating tasks

```
USE ROLE lab_role;

-- Serverless, every 60 minutes, sized to finish within 20 minutes
CREATE OR REPLACE TASK bytemart_lab.ops.t_refresh_inventory
  SCHEDULE = '60 MINUTES'
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
  TARGET_COMPLETION_INTERVAL = '20 MINUTES'
  SERVERLESS_TASK_MAX_STATEMENT_SIZE = 'MEDIUM'          -- cost guardrail
  COMMENT = 'Hourly inventory replica refresh'
AS
  CALL bytemart_lab.ops.refresh_inventory();

-- Warehouse task with a CRON schedule (02:00 India time, every day)
CREATE OR REPLACE TASK bytemart_lab.ops.t_daily_sales
  WAREHOUSE = lab_wh
  SCHEDULE = 'USING CRON 0 2 * * * Asia/Kolkata'
  USER_TASK_TIMEOUT_MS = 1800000                         -- 30 minutes (default 1 hour)
  SUSPEND_TASK_AFTER_NUM_FAILURES = 3                    -- default 10
AS
  CALL bytemart_lab.ops.load_daily_sales(DATEADD(day, -1, CURRENT_DATE()));

-- Tasks are created SUSPENDED
ALTER TASK bytemart_lab.ops.t_refresh_inventory RESUME;
ALTER TASK bytemart_lab.ops.t_daily_sales RESUME;

-- Run once now, outside the schedule (task must exist; useful for testing)
EXECUTE TASK bytemart_lab.ops.t_daily_sales;
```

Line notes:

- `SCHEDULE` accepts an interval (`'<n> MINUTES'`, `HOURS`, `SECONDS`) or `'USING CRON <min> <hour> <day-of-month> <month> <day-of-week> <time zone>'`. Always give CRON schedules an explicit time zone; remember daylight-saving transitions in zones that have them.
- `TARGET_COMPLETION_INTERVAL` (serverless only) tells Snowflake how long a run may take; it scales compute to meet it, within the min/max statement size bounds.
- A task executes **one** statement. Put multi-step logic in a stored procedure (Chapter 10) or a scripting block, so it is testable and its transaction handling is explicit.
- Tasks run with the privileges of the **task owner role** (not of whoever resumed it). `EXECUTE AS USER <user>` runs the task on behalf of a specific user, which requires `IMPERSONATE` on that user.

Triggered tasks

A **triggered task** has no `SCHEDULE`; it runs when its stream has data:

```
CREATE OR REPLACE TASK bytemart_lab.ops.t_apply_order_changes
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
  WHEN SYSTEM$STREAM_HAS_DATA('bytemart_lab.raw.orders_stream')
AS
  CALL bytemart_lab.ops.apply_order_changes();      -- the canonical MERGE of Chapter 14
ALTER TASK bytemart_lab.ops.t_apply_order_changes RESUME;
```

Triggered tasks react within seconds of new data, but not more often than `USER_TASK_MINIMUM_TRIGGER_INTERVAL_IN_SECONDS` (default 30). A scheduled task can also have a `WHEN` clause: the condition is evaluated at each scheduled time, and if it is false the run is skipped. Evaluating `SYSTEM$STREAM_HAS_DATA` does not consume warehouse credits, so "every minute, when there is data" is cheap.

15.2 Task graphs

A **task graph** (DAG) is a root task with a schedule and child tasks that run `AFTER` their predecessors.

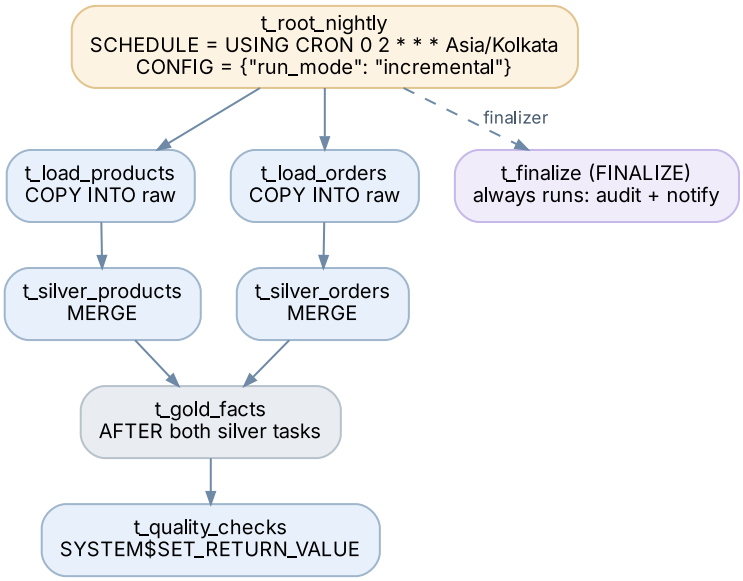


Figure 15.1 — ByteMart's nightly task graph: a scheduled root, parallel load and silver branches, a gold task waiting for both, checks, and a finalizer that always runs.

```
CREATE OR REPLACE TASK bytemart_lab.ops.t_root_nightly
  SCHEDULE = 'USING CRON 0 2 * * * Asia/Kolkata'
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'SMALL'
  CONFIG = $${"run_mode": "incremental", "lookback_days": 3}$$
  OVERLAP_POLICY = NO_OVERLAP
  TASK_AUTO_RETRY_ATTEMPTS = 2
AS
  SELECT 'start';                                          -- root can be a no-op

CREATE OR REPLACE TASK bytemart_lab.ops.t_load_products
  AFTER bytemart_lab.ops.t_root_nightly
AS CALL bytemart_lab.ops.load_products();

CREATE OR REPLACE TASK bytemart_lab.ops.t_load_orders
  AFTER bytemart_lab.ops.t_root_nightly
AS CALL bytemart_lab.ops.load_orders();

CREATE OR REPLACE TASK bytemart_lab.ops.t_silver_products
  AFTER bytemart_lab.ops.t_load_products
AS CALL bytemart_lab.ops.merge_silver_products();

CREATE OR REPLACE TASK bytemart_lab.ops.t_silver_orders
  AFTER bytemart_lab.ops.t_load_orders
AS CALL bytemart_lab.ops.merge_silver_orders(
     SYSTEM$GET_TASK_GRAPH_CONFIG('lookback_days')::NUMBER);

CREATE OR REPLACE TASK bytemart_lab.ops.t_gold_facts
  AFTER bytemart_lab.ops.t_silver_products, bytemart_lab.ops.t_silver_orders
AS CALL bytemart_lab.ops.build_gold_facts();

CREATE OR REPLACE TASK bytemart_lab.ops.t_quality_checks
  AFTER bytemart_lab.ops.t_gold_facts
AS
DECLARE
  failures NUMBER;
  dq_failed EXCEPTION (-20001, 'Data quality checks failed');
BEGIN
  SELECT COUNT(*) INTO :failures
```

```
  FROM bytemart_lab.ops.dq_results
  WHERE run_date = CURRENT_DATE() AND passed = FALSE;
  CALL SYSTEM$SET_RETURN_VALUE(TO_VARCHAR(:failures));
  IF (failures > 0) THEN
    RAISE dq_failed;              -- fail the task so the graph run is marked failed
  END IF;
END;

CREATE OR REPLACE TASK bytemart_lab.ops.t_finalize
  FINALIZE = bytemart_lab.ops.t_root_nightly
AS CALL bytemart_lab.ops.record_graph_run_and_notify();

-- Resume children first, root last — or all at once:
SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('bytemart_lab.ops.t_root_nightly');
```

Key rules:

Concept	Rule (at the research cutoff)
Root	Only the root has a `SCHEDULE`; children use `AFTER`
Size	Up to 1,000 tasks per graph; a task can have up to 100 predecessors and 100 children
Fan-in	A task with several predecessors runs only after **all** succeed
Changing a graph	Suspend the root before modifying tasks; resume afterwards
`CONFIG`	JSON on the root; any task reads values with `SYSTEM$GET_TASK_GRAPH_CONFIG('<key>')`
Return values	A task sets a value with `SYSTEM$SET_RETURN_VALUE`; a direct child reads it with `SYSTEM$GET_PREDECESSOR_RETURN_VALUE`
Finalizer	One per root (`FINALIZE = <root>`); runs after the graph completes **whether it succeeded or failed** — use it for audit, cleanup and notifications
Overlap	`OVERLAP_POLICY` on the root: `NO_OVERLAP` (default — a new run is skipped while one is running), `ALLOW_CHILD_OVERLAP`, `ALLOW_ALL_OVERLAP`
Retries	`TASK_AUTO_RETRY_ATTEMPTS` (0–30, set on the root) retries a failed graph run from the failed tasks; manually, `EXECUTE TASK <root> RETRY LAST`
Failure suspension	After `SUSPEND_TASK_AFTER_NUM_FAILURES` consecutive failures (default 10) the task is suspended automatically

RAISE ERRORS DELIBERATELY

A procedure that catches an exception and returns a string like `'FAILED'` makes the task **succeed** — the graph continues, and nobody is alerted. Either let the exception propagate (re-raise after logging, as in Chapter 10's `load_daily_sales`) or `RAISE` explicitly. Task state should tell the truth.

Monitoring tasks

```
-- Recent runs of tasks in a schema (INFORMATION_SCHEMA: recent history, fast)
SELECT name, state, scheduled_time, completed_time, error_message, return_value
FROM TABLE(bytemart_lab.information_schema.task_history(
       SCHEDULED_TIME_RANGE_START => DATEADD(hour, -24, CURRENT_TIMESTAMP())))
ORDER BY scheduled_time DESC;

-- Graph runs (status per run of a root task)
SELECT root_task_name, state, scheduled_time, completed_time, first_error_task_name, first_er
```

```
ror_message
FROM TABLE(bytemart_lab.information_schema.complete_task_graphs())
ORDER BY scheduled_time DESC;

-- Serverless task cost (ACCOUNT_USAGE: longer history, latency up to a few hours)
SELECT task_name, DATE_TRUNC('day', start_time) AS day, SUM(credits_used) AS credits
FROM snowflake.account_usage.serverless_task_history
WHERE start_time >= DATEADD(day, -30, CURRENT_TIMESTAMP())
GROUP BY ALL ORDER BY day DESC, credits DESC;
```

Snowsight's task graph view shows each run, task durations and errors visually — the fastest way to debug a failing graph.

15.3 Dynamic tables

A **dynamic table** is defined by a query. Snowflake keeps its contents up to date automatically within a **target lag**, refreshing **incrementally** where it can. You declare *what* the result is; Snowflake handles *when* and *how* — no streams, tasks or MERGE statements to write.

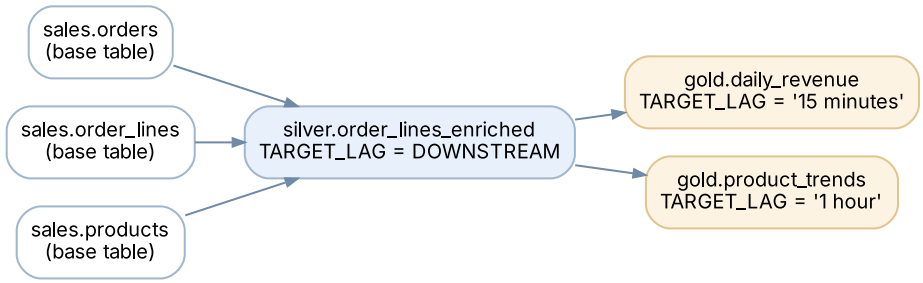


Figure 15.2 — A dynamic table pipeline. Gold tables declare how fresh they must be; the intermediate table uses DOWNSTREAM and refreshes only as often as its consumers need.

```
CREATE OR REPLACE DYNAMIC TABLE bytemart_lab.silver.order_lines_enriched
  TARGET_LAG = DOWNSTREAM
  WAREHOUSE = transform_wh
  REFRESH_MODE = AUTO
  INITIALIZE = ON_CREATE
AS
SELECT o.order_id, o.order_ts, CAST(o.order_ts AS DATE) AS order_date,
       o.customer_id, o.channel, o.status,
       l.product_id, p.category, l.quantity, l.unit_price,
       l.quantity * l.unit_price AS line_revenue
FROM bytemart_lab.sales.orders o
JOIN bytemart_lab.sales.order_lines l ON l.order_id = o.order_id
JOIN bytemart_lab.sales.products p    ON p.product_id = l.product_id
WHERE o.status <> 'CANCELLED';

CREATE OR REPLACE DYNAMIC TABLE bytemart_lab.gold.daily_revenue
  TARGET_LAG = '15 minutes'
  WAREHOUSE = transform_wh
AS
SELECT order_date, channel, category,
       COUNT(DISTINCT order_id) AS orders,
       SUM(line_revenue)        AS revenue
FROM bytemart_lab.silver.order_lines_enriched
GROUP BY ALL;
```

Target lag

- `TARGET_LAG = '<n> { seconds | minutes | hours | days }'` — maximum staleness relative to the base tables; **minimum 60 seconds**. Snowflake schedules refreshes to stay within it; it is a target, not a hard guarantee.
- `TARGET_LAG = DOWNSTREAM` — refresh only when a downstream dynamic table needs fresh data. Use it for intermediate tables so the most demanding consumer sets the pace.
- A dynamic table's lag cannot be shorter than that of dynamic tables it depends on.
- Shorter lag = more refreshes = more cost. Set lag from the business need, not from habit.

Refresh modes

`REFRESH MODE`	Behaviour
`AUTO` (default)	Incremental when the query supports it and it is efficient; otherwise full. The decision is made at creation — check the result (`SHOW DYNAMIC TABLES`, `refresh_mode` and `refresh_mode_reason`)
`INCREMENTAL`	Always incremental; creation fails if the query cannot be incrementalized
`FULL`	Recompute everything on each refresh (for small tables or unsupported constructs)
`ADAPTIVE` (GA 30 Jul 2026)	Incremental, but reinitializes when heuristics show a full recompute would be significantly cheaper
`CUSTOM_INCREMENTAL` (GA 27 Jul 2026)	Your own `MERGE INTO SELF` / `INSERT INTO SELF` logic via `REFRESH USING` (Section 15.4)

The refresh mode cannot be changed with `ALTER`; use `CREATE OR REPLACE` or `CREATE OR ALTER`. **Incremental refresh** processes only changes in the base tables since the last refresh (Snowflake uses change tracking — the same machinery as streams). Most joins, filters, projections, `UNION ALL`, many aggregates and window functions are supported, and 2026 releases widened coverage further (see "What's New"). Non-deterministic functions, some outer-join forms and certain constructs force a full refresh. `EXPLAIN CHANGES` (GA September 2026) predicts how a dynamic table's query will be refreshed — use it while designing.

Other important options

Option	Purpose
`INITIALIZE = ON_CREATE` (default) / `ON_SCHEDULE`	Populate synchronously at creation (creation fails if the refresh fails) or at the first scheduled refresh
`INITIALIZATION_WAREHOUSE`	Separate (often larger) warehouse for the initial build and reinitializations
`SCHEDULER = DISABLE`	No automatic refresh — refresh only with `ALTER DYNAMIC TABLE … REFRESH` (useful when an external orchestrator controls timing); `TARGET_LAG` cannot be set
`FROZEN WHERE (<predicate>)`	Declares a **frozen region** — rows matching it are not recomputed (for example, closed accounting periods); formerly `IMMUTABLE WHERE`
`BACKFILL FROM <table>`	Seed a frozen region (or a custom incremental table) from existing data instead of recomputing history

Option	Purpose
`CLUSTER BY`, `DATA_RETENTION_TIME_IN_DAYS`, policies, tags	As for tables

```
-- Keep history before the current month frozen (not recomputed)
ALTER DYNAMIC TABLE bytemart_lab.gold.daily_revenue
  SET FROZEN WHERE (order_date < DATE_TRUNC('month', CURRENT_DATE()));

-- Operate
ALTER DYNAMIC TABLE bytemart_lab.gold.daily_revenue REFRESH;      -- manual refresh
ALTER DYNAMIC TABLE bytemart_lab.gold.daily_revenue SUSPEND;
ALTER DYNAMIC TABLE bytemart_lab.gold.daily_revenue RESUME;
ALTER DYNAMIC TABLE bytemart_lab.gold.daily_revenue SET TARGET_LAG = '30 minutes';
```

Monitoring dynamic tables

```
SHOW DYNAMIC TABLES IN SCHEMA bytemart_lab.gold;

SELECT name, state, refresh_action, refresh_trigger,
       data_timestamp, refresh_start_time, refresh_end_time, state_message
FROM TABLE(bytemart_lab.information_schema.dynamic_table_refresh_history(
       NAME_PREFIX => 'BYTEMART_LAB.GOLD.'))
ORDER BY refresh_start_time DESC
LIMIT 50;
```

`refresh_action` shows `INCREMENTAL`, `FULL`, `REINITIALIZE` or `NO_DATA`. A table that was expected to refresh incrementally but shows `FULL` is a cost problem. Snowsight shows the dynamic table graph with lag over time. `ACCOUNT_USAGE` views provide longer history for cost and SLA reporting.

15.4 Custom incremental dynamic tables

When the desired result cannot be expressed as a single `SELECT` — accumulators, audit trails, stream–static joins with conditional logic — a **custom incremental** dynamic table lets you write the refresh DML yourself while Snowflake still schedules it, tracks changes and handles lag:

```
CREATE OR REPLACE DYNAMIC TABLE bytemart_lab.gold.customer_lifetime_value (
    customer_id     NUMBER,
    lifetime_orders NUMBER,
    lifetime_spend  NUMBER(14,2),
    last_order_ts   TIMESTAMP_NTZ
)
  TARGET_LAG = '10 minutes'
  WAREHOUSE = transform_wh
  REFRESH USING (
    MERGE INTO SELF AS tgt
    USING (
        SELECT customer_id,
               COUNT(*)          AS new_orders,
               SUM(order_total)  AS new_spend,
               MAX(order_ts)     AS max_ts
        FROM bytemart_lab.raw.order_events CHANGES (INFORMATION => APPEND_ONLY)
        GROUP BY customer_id
    ) AS src
```

```
    ON tgt.customer_id = src.customer_id
    WHEN MATCHED THEN UPDATE SET
        tgt.lifetime_orders = tgt.lifetime_orders + src.new_orders,
        tgt.lifetime_spend  = tgt.lifetime_spend  + src.new_spend,
        tgt.last_order_ts   = GREATEST(tgt.last_order_ts, src.max_ts)
    WHEN NOT MATCHED THEN INSERT (customer_id, lifetime_orders, lifetime_spend,
last_order_ts)
        VALUES (src.customer_id, src.new_orders, src.new_spend, src.max_ts)
  );
```

Rules worth remembering: an explicit column list is required; refer to the table as `SELF`; one DML statement; `CHANGES` without time bounds (Snowflake binds the interval to refresh boundaries); base tables need change tracking; deduplicate the source so one target row is matched once; each refresh is one transaction; change the definition only with `CREATE OR ALTER`; not combinable with `FROZEN WHERE`. Because the logic is yours, so is its correctness — test it like any MERGE (Chapter 14).

15.5 Materialized views

A **materialized view** (Enterprise Edition and higher) stores the result of a query over a **single table** and is maintained automatically by a serverless background service. Queries against it are always current (Snowflake combines the stored result with any not-yet-maintained changes), and the optimizer can **rewrite** queries on the base table to use it.

```
CREATE OR REPLACE MATERIALIZED VIEW bytemart_lab.gold.mv_clicks_by_day_page AS
SELECT TO_DATE(event_ts) AS event_date, page, COUNT(*) AS views
FROM bytemart_lab.raw.click_events
GROUP BY TO_DATE(event_ts), page;
```

Restrictions (summary): one base table — no joins; a limited set of aggregate functions; no window functions, `HAVING`, `ORDER BY`, `LIMIT`, UDFs or non-deterministic functions; cannot query another view. They shine for **simple, hot aggregations or projections over a large table**, for **re-clustering a table a second way**, and for **external tables** (materializing lake data for speed). For anything with joins or multi-step logic, use dynamic tables.

15.6 Choosing the right tool

Need	Best fit	Why
Declarative transformation with a freshness target	**Dynamic table**	No orchestration code; incremental refresh; lineage graph
Logic that is not one `SELECT` (procedural steps, external calls, files)	**Task (+ procedure)**	Full control
Exactly-once processing of changes with custom apply rules	**Stream + task**, or **custom incremental DT**	Explicit change handling
Fast single-table aggregate kept always current, query rewrite	**Materialized view**	Automatic, transparent
Loading files on a schedule	**Task running COPY**; or Snowpipe	

Need	Best fit	Why
Cross-system workflows (Snowflake + APIs + other platforms), complex SLAs, human approvals	**External orchestrator** (Airflow, Dagster, Prefect, cloud schedulers) calling Snowflake	One control plane across systems
dbt projects	**dbt Projects on Snowflake** scheduled by tasks, or dbt via an external orchestrator (Chapter 16)	
"Tell me when something is wrong"	**Alert**	Condition + action

Many production platforms combine them: Snowpipe → raw; dynamic tables → silver/gold; a nightly task graph for procedural jobs (snapshots, exports, ML scoring); alerts for SLAs; an external orchestrator only where workflows leave Snowflake.

15.7 Alerts

An **alert** evaluates a condition and runs an action when it is true.

```
-- 1. One-time setup by an administrator: an email notification integration
CREATE NOTIFICATION INTEGRATION bytemart_email_int
  TYPE = EMAIL
  ENABLED = TRUE
  ALLOWED_RECIPIENTS = ('data-oncall@bytemart.example');

GRANT USAGE ON INTEGRATION bytemart_email_int TO ROLE lab_role;
GRANT EXECUTE ALERT, EXECUTE MANAGED ALERT ON ACCOUNT TO ROLE lab_role;

-- 2. A serverless, scheduled freshness alert
CREATE OR REPLACE ALERT bytemart_lab.ops.alert_orders_stale
  SCHEDULE = '15 MINUTES'
  IF (EXISTS (
      SELECT 1
      FROM bytemart_lab.raw.orders_raw
      HAVING MAX(_loaded_at) < DATEADD(minute, -60, CURRENT_TIMESTAMP())
  ))
  THEN
    CALL SYSTEM$SEND_EMAIL(
        'bytemart_email_int',
        'data-oncall@bytemart.example',
        'ByteMart: orders are more than 60 minutes stale',
        'No new rows in raw.orders_raw for 60 minutes. Check pipe status and COPY_HISTORY.');

ALTER ALERT bytemart_lab.ops.alert_orders_stale RESUME;      -- alerts are created suspended
```

- Omitting `WAREHOUSE` makes the alert **serverless** (requires `EXECUTE MANAGED ALERT`); with `WAREHOUSE` it uses that warehouse.
- **Alerts on new data** omit `SCHEDULE`: the condition is evaluated when new rows arrive in the queried table — useful for "alert on every error row logged".
- `SYSTEM$SEND_SNOWFLAKE_NOTIFICATION` can send to email, webhooks (Slack, Microsoft Teams, PagerDuty via webhook integrations) and cloud queues.
- Email recipients must be verified users in the account and listed in `ALLOWED_RECIPIENTS`.
- `EXECUTE ALERT <name>` evaluates an alert immediately — useful for testing.

- History: `INFORMATION_SCHEMA.ALERT_HISTORY()` and `ACCOUNT_USAGE.ALERT_HISTORY`.

Typical ByteMart alerts: data freshness per source, failed task graph runs (query `COMPLETE_TASK_GRAPHS` for `FAILED`), dynamic tables in `FAILED` / `SUSPENDED` state or lagging, stream `STALE_AFTER` approaching, data-quality failures, and daily credit spikes (Chapter 18).

15.8 External orchestration

External orchestrators run Snowflake work through the Python connector, Snowpark or provider packages (for example, Airflow's Snowflake provider):

```
# Airflow DAG fragment (illustrative): run the Snowflake task graph and wait for it
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator

run_root = SQLExecuteQueryOperator(
    task_id="run_bytemart_nightly",
    conn_id="snowflake_bytemart",              # credentials live in the Airflow connection
    sql="EXECUTE TASK bytemart_lab.ops.t_root_nightly",
)
```

Guidance:

- Keep transformation logic **in Snowflake** (procedures, dynamic tables, dbt); let the orchestrator decide **when** and **in what order across systems**.
- Use key-pair or OAuth service users with minimal roles; never passwords in DAG code.
- Set `QUERY_TAG` per job so cost can be attributed (`ALTER SESSION SET QUERY_TAG = 'airflow:bytemart_nightly'`).
- Avoid double scheduling: if Airflow triggers a task graph, do not also give its root a schedule (or keep the schedule and let Airflow only monitor).

15.9 Backfills and reprocessing

Situation	Approach
Re-run one day of a task-based pipeline	Parameterize procedures by date; call them for the date (Chapter 10's `load_daily_sales(p_run_date)` is idempotent via delete-insert)
Re-run a task graph with different settings	Pass a different configuration to the run (the root's `CONFIG`; `EXECUTE TASK` can supply run-specific configuration — see the EXECUTE TASK reference)
Dynamic table logic changed	`CREATE OR REPLACE` / `CREATE OR ALTER` triggers reinitialization; use `INITIALIZATION_WAREHOUSE` for a larger warehouse; keep history with `FROZEN WHERE` + `BACKFILL FROM` where recomputation is impossible or expensive
Months of history to rebuild	Rebuild into a new table (CTAS or new dynamic table), validate, then `SWAP WITH` (Chapter 8)
Late-arriving data	Incremental windows that recompute the last *n* days (`lookback_days` in the task graph config); dynamic tables handle late data automatically

Every backfill should be: **idempotent** (safe to repeat), **bounded** (explicit date range), **observable** (logged in a run table), and **reconciled** (counts and sums compared after the run).

15.10 Guided lab

> **LAB 15.1 — SAME PIPELINE TWO WAYS**
>
> **Goal:** build ByteMart's daily revenue pipeline as a task graph and as dynamic tables, then compare. **Environment:** Snowsight; `lab_role` with `EXECUTE TASK` and `EXECUTE MANAGED TASK`; the Chapter 8 tables in `bytemart_lab.sales` (`orders`, `order_lines`, `products`).

Step 1 — Dynamic tables. Create `silver.order_lines_enriched` (`TARGET_LAG = DOWNSTREAM`) and `gold.daily_revenue` (`TARGET_LAG = '1 minute'`) as in Section 15.3. Run `SHOW DYNAMIC TABLES` and note `refresh_mode` and `refresh_mode_reason`. **Expected:** `INCREMENTAL` for the silver table (inner joins, filters and projections are incrementalizable). For the gold aggregate, read `refresh_mode` and `refresh_mode_reason`: if a construct such as `COUNT(DISTINCT …)` prevents incremental refresh in your account, the reason says so — try `EXPLAIN CHANGES` and, if needed, replace it with a supported formulation.

Step 2. Insert a new order with two lines into `sales.orders` and `sales.order_lines`. Within about a minute, query `gold.daily_revenue`. **Expected:** the day's revenue includes the new order. Inspect `DYNAMIC_TABLE_REFRESH_HISTORY` for `refresh_action = INCREMENTAL`.

Step 3 — Task version. Create a procedure `ops.build_daily_revenue()` that rebuilds a table `gold.daily_revenue_t` with `INSERT OVERWRITE` from the same query. Create a serverless task every 5 minutes calling it, resume it, and wait for one run.

Step 4 — Compare. Run `SELECT * FROM gold.daily_revenue MINUS SELECT * FROM gold.daily_revenue_t` (and the reverse). **Expected:** no rows once both have refreshed.

Step 5 — Failure handling. Make the procedure fail (for example, reference a missing table), run `EXECUTE TASK`, and find the error in `TASK_HISTORY`. Fix it.

Step 6 — Alert. Create the freshness alert from Section 15.7 (pointed at `gold.daily_revenue_t` with a 10-minute threshold), suspend the task, and wait for the email. Resume everything, then **suspend all lab tasks, alerts and dynamic tables** to stop spending credits.

15.11 Production use case: ByteMart's platform schedule

Layer	Mechanism	Freshness
Orders, inventory (CDC)	Openflow/Kafka → raw; triggered tasks apply MERGE	~1–2 min
Product catalogue	Nightly task graph (COPY + MERGE)	Daily 02:00 IST
Silver and gold analytics	Dynamic tables; intermediate `DOWNSTREAM`, gold 15 min–1 h	15 min–1 h
Finance month-end	Dynamic table with `FROZEN WHERE` for closed periods	Closed months never recomputed

Layer	Mechanism	Freshness
Snapshots, exports, ML scoring	Nightly task graph with finalizer	Daily
SLAs and failures	Alerts → on-call email/webhook	15 min checks
Cross-system release workflow	Airflow triggers dbt deploys and task graphs	On demand

Results after migrating from a single 4-hour nightly batch: analytics freshness from 24 h to 15 min; fewer custom MERGE scripts; costs held flat by setting lags from business needs and moving intermediate tables to `DOWNSTREAM`.

15.12 Common mistakes

ORCHESTRATION PITFALLS

1. Forgetting that tasks, alerts are created suspended (or resuming children but not the root).
2. Procedures that swallow errors so tasks "succeed".
3. Modifying a graph without suspending the root.
4. CRON without a time zone.
5. One-minute target lags on everything.
6. Expecting incremental refresh without checking `refresh_mode`.
7. Leaving lab tasks and dynamic tables running.
8. Double scheduling (task schedule + external orchestrator).
9. No finalizer, so failures are never reported.
10. Materialized views where dynamic tables are needed (joins).

15.13 Troubleshooting exercise

EXERCISE — 'THE DASHBOARD IS 6 HOURS STALE'

`gold.daily_revenue` has `TARGET_LAG = '15 minutes'`, but the dashboard shows data from 6 hours ago. List what you would check, in order, and the likely fixes. (Solution at the end of the chapter.)

15.14 Knowledge check

1. How do you make a task serverless, and which privileges are needed?
2. What does `TARGET_COMPLETION_INTERVAL` do?
3. How does a triggered task differ from a scheduled task with `WHEN`?
4. What runs when a task graph fails, and how is it declared?
5. How do tasks in a graph share configuration and pass values?
6. What does `NO_OVERLAP` mean?

7. What is `TARGET_LAG = DOWNSTREAM` for?
8. Name the dynamic table refresh modes.
9. What is a frozen region?
10. When is a materialized view a better choice than a dynamic table?
11. What makes an alert serverless? An alert on new data?
12. Which four properties should every backfill have?

15.15 Hands-on assignment

1. Complete Lab 15.1.
2. Build the nightly task graph of Figure 15.1 with stub procedures; make one task fail and verify that the finalizer records the failure.
3. Write a monitoring query that lists, for every dynamic table in `bytemart_lab`, its target lag, the maximum actual lag in the last 24 hours, and the count of `FULL` refreshes.

15.16 Chapter summary

Tasks run single statements on schedules, after predecessors, or when streams have data; serverless tasks remove warehouse management and can target completion intervals. Task graphs add dependencies, shared configuration, return values, overlap control, retries and finalizers. Dynamic tables turn pipelines into declarations: a query plus a target lag, refreshed incrementally where possible, with refresh modes (including GA adaptive and custom incremental modes), frozen regions and backfill for history. Materialized views serve single-table, always-current aggregations. Alerts close the loop by detecting stale data and failures. External orchestrators coordinate across systems while logic stays in Snowflake. Backfills must be idempotent, bounded, observable and reconciled.

15.17 Key takeaways

- Prefer dynamic tables for declarative transformations; tasks for procedures.
- Set lag from business need; use `DOWNSTREAM` for intermediates.
- Check refresh mode — incremental is where the savings are.
- Failures must fail loudly: re-raise, finalizers, alerts.
- Suspend lab objects.

15.18 Further reading

- Introduction to tasks: https://docs.snowflake.com/en/user-guide/tasks-intro
- CREATE TASK: https://docs.snowflake.com/en/sql-reference/sql/create-task
- Task graphs: https://docs.snowflake.com/en/user-guide/tasks-graphs
- Dynamic tables: https://docs.snowflake.com/en/user-guide/dynamic-tables-about

- CREATE DYNAMIC TABLE: https://docs.snowflake.com/en/sql-reference/sql/create-dynamic-table
- Custom incrementalization: https://docs.snowflake.com/en/user-guide/dynamic-tables/custom-incrementalization
- Materialized views: https://docs.snowflake.com/en/user-guide/views-materialized
- Alerts: https://docs.snowflake.com/en/user-guide/alerts
- Adaptive refresh mode GA: https://docs.snowflake.com/release-notes/2026/other/2026-07-30-dynamic-tables-adaptive-refresh-mode-ga

15.19 Solutions

Troubleshooting — 6 hours stale. (1) `SHOW DYNAMIC TABLES` — is the table or an upstream table **suspended** or **failed** (`scheduling_state`)? Dynamic tables are suspended automatically after repeated refresh failures. (2) `DYNAMIC_TABLE_REFRESH_HISTORY` — recent refreshes failing (`state_message`), or taking longer than the lag (full refreshes on a growing table)? (3) Is the **warehouse** suspended by a resource monitor, or does the owner role lack `USAGE`? (4) Are the **base tables** themselves stale — did ingestion stop (Chapter 13)? A dynamic table can be perfectly fresh relative to stale inputs. (5) Is an upstream table `SCHEDULER = DISABLE` or has a longer lag? Fixes: resume, fix the failing query or privileges, fix ingestion, use a larger warehouse or restructure for incremental refresh, and add an alert on lag and state.

Knowledge check answers.

1. Omit `WAREHOUSE`; the owner role needs `EXECUTE TASK` and `EXECUTE MANAGED TASK` on the account.
2. For serverless tasks, it sets how long a run should take; Snowflake scales compute (within statement-size bounds) to meet it.
3. A triggered task has no schedule and runs when its stream has data (no more often than the minimum trigger interval); a scheduled task with `WHEN` checks the condition at each scheduled time.
4. The finalizer task, declared with `FINALIZE = <root task>`; it runs whether the graph succeeded or failed.
5. `CONFIG` on the root read with `SYSTEM$GET_TASK_GRAPH_CONFIG`; `SYSTEM$SET_RETURN_VALUE` / `SYSTEM$GET_PREDECESSOR_RETURN_VALUE`.
6. A new graph run is skipped while a previous run of the same graph is still executing.
7. Intermediate tables that refresh only when downstream dynamic tables need them.
8. `AUTO`, `INCREMENTAL`, `FULL`, `ADAPTIVE`, `CUSTOM_INCREMENTAL`.
9. A region of a dynamic table (defined by `FROZEN WHERE`) that refreshes do not recompute — for example, closed periods.
10. Single-table aggregates or projections that must always be current, benefit from automatic query rewrite, or materialize external tables.
11. Omitting `WAREHOUSE`; omitting `SCHEDULE`.
12. Idempotent, bounded, observable, reconciled.

CHAPTER 16

Modern Data Engineering and ELT

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- design a layered (medallion) data platform on Snowflake with environments and naming conventions;
- define data contracts and place data-quality checks at the right layers;
- build a dbt project: sources, staging, intermediate and mart models, materializations, Jinja and macros, seeds, tests, unit tests, snapshots and packages;
- choose and reason about incremental strategies — including their failure modes;
- use Snowflake-specific dbt features (transient tables, dynamic tables, grants, query tags);
- deploy, run and schedule dbt with **dbt Projects on Snowflake**;
- set up CI/CD for dbt with zero-copy clones and state-based selection;
- compare dbt with other transformation tools and Snowflake-native options.

Prerequisites: Chapter 9 (modelling, SCD), Chapters 12–15 (ingestion, CDC, orchestration), Chapter 11 (Git, CLI, CI/CD basics).

Where this leads: Chapter 19 (monitoring and observability at scale), Chapter 21 (governance, lineage and sharing), Chapter 27 (semantic views on top of marts), Chapter 40 (guided capstone projects).

Execution environment. The complete ByteMart dbt project in this chapter **was executed** with dbt Core 1.12.5 and the dbt-duckdb 1.11.0 adapter on DuckDB 1.5.6 (`--target local`); every output shown is captured from those runs by `code/ch16/demo.py`. The Snowflake targets, dbt Projects on Snowflake SQL and CI pipeline are **documentation-verified, not executed**. Where Snowflake behaviour differs from DuckDB, the text says so.

16.1 Platform architecture

Layers

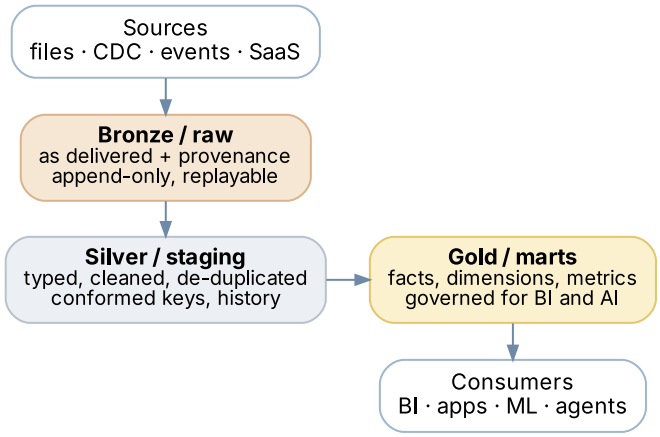


Figure 16.1 — The medallion architecture: each layer has one job and one owner, and data only flows forward.

Layer	Also called	Contents	Rules	Built by
Bronze	raw, landing	Data as delivered + provenance columns (file, row, load time, offsets)	Append-only; never edited by hand; replayable; permissive types or `VARIANT`	Ingestion (Ch 12–14)
Silver	staging, integration, core	Typed, cleaned, de-duplicated, conformed keys; history (SCD2, snapshots)	One model per source entity in staging; business logic minimal; tested	dbt / dynamic tables
Gold	marts, presentation	Facts, dimensions, aggregates, metrics, semantic views	Business definitions; contracts; documented owners; governed	dbt / dynamic tables

Additional zones that real platforms need: a **sandbox** per analyst or team (cheap, transient, auto-dropped), a **quarantine** for rejected rows, an **ops/audit** schema for run logs and quality results, and **shares/exports** for outbound data.

Databases, schemas and environments

A common, scalable layout uses **one database per environment** and **one schema per layer or domain**:

```
BYTEMART_DEV        (per-developer schemas: DBT_AARAV_STAGING, DBT_AARAV_MARTS, ...)
BYTEMART_CI         (one schema set per pull request, created from zero-copy clones)
BYTEMART            (production)
  ├── RAW           bronze: loaded by pipelines
  ├── STAGING       silver: dbt views
  ├── MARTS         gold: facts, dimensions
  ├── SNAPSHOTS     SCD2 history
  ├── SEMANTIC      semantic views (Chapter 27)
  └── OPS           run logs, audit, data-quality results
```

Principles: production is written **only** by service users through deployment pipelines; developers read production (often through clones) and write to dev; access is granted to **database roles** per schema (Chapter 20); naming is consistent (`stg_`, `int_`, `fct_`, `dim_`, `agg_`).

16.2 Data contracts

A **data contract** is an explicit agreement about a dataset: its schema (columns, types, nullability), semantics (what a row means, units, keys), quality expectations, freshness, ownership and how changes are made.

Where	Mechanism
Source → bronze	Agreed file/event schema; schema evolution policy (Chapter 12); rejects and alerts
Bronze → silver	dbt source definitions with freshness; staging tests (keys, accepted values, relationships)
Gold	**dbt model contracts** (`contract: {enforced: true}` with declared types and constraints) — dbt fails the build if the model's output does not match
Physical	Snowflake `NOT NULL` (enforced) and `CHECK` constraints (enforced on standard tables at the research cutoff); primary/foreign/unique keys are informational (`RELY` helps the optimizer)
Consumers	Versioned models (dbt model versions), deprecation dates, semantic views with documented metrics (Chapter 27)

Change policy. Additive changes (a new nullable column) can be released freely; breaking changes (renames, type changes, removed columns, changed grain) require a new model version, a migration window and consumer sign-off.

16.3 Data quality

Quality checks belong at every layer, with different purposes:

Layer	Check	Action on failure
Ingestion	File parsed? Row counts? Schema drift?	Reject file/rows to quarantine; alert
Silver	Keys unique and not null; accepted values; referential integrity; freshness	Fail the build (`error`) or warn (`warn`) depending on severity
Gold	Reconciliation with source totals; business rules (revenue ≥ 0); contracts	Fail the build; do not publish
Ongoing	Volume, freshness and distribution anomalies	Data metric functions and alerts (Chapter 19)

Severity matters: a duplicated primary key in `fct_order_lines` must stop the pipeline (revenue would be overstated); a missing city on a customer should warn. dbt supports `severity: warn` and thresholds (`warn_if`, `error_if`) per test.

16.4 dbt fundamentals

dbt (data build tool) turns `SELECT` statements into tables and views, in dependency order, with testing, documentation and lineage. You write models as SQL files; dbt compiles Jinja, builds a DAG from `ref()` and `source()`, and runs the SQL in your warehouse. dbt does not move data between systems — it transforms data already in Snowflake.

Concept	What it is
Model	A `.sql` file containing one `SELECT`; materialized as view, table, incremental table, ephemeral CTE, or (on Snowflake) dynamic table
Source	A declared raw table (`source('bytemart_raw','orders')`), with optional freshness rules
`ref()`	Reference to another model; builds the DAG and resolves the environment-specific name
Seed	A small CSV checked into the project and loaded as a table (lookup tables, test fixtures)
Data test	A query that returns failing rows (generic: `unique`, `not_null`, `accepted_values`, `relationships`, custom; or singular SQL files)
Unit test	Fixed input rows + expected output for one model's logic (dbt ≥ 1.8)
Snapshot	SCD2 history of a mutable source
Macro	A reusable Jinja function
Package	Reusable macros/models from the dbt package hub or Git (`packages.yml`, `dbt deps`)
Profile/target	Connection settings per environment (`profiles.yml`)

The ByteMart project

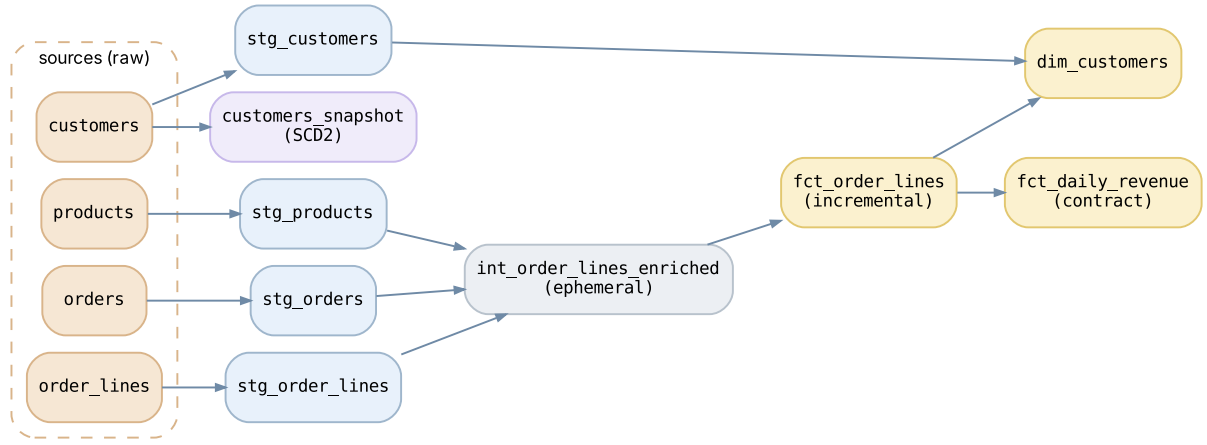


Figure 16.2 — Lineage of the ByteMart dbt project, as dbt builds it from ref() and source().

```
bytemart_dbt/
├── dbt_project.yml
├── profiles.yml                (local DuckDB + Snowflake dev/prod, secrets from env vars)
├── seeds/raw/                  customers, products, orders, order_lines (local stand-ins
for raw)
├── models/
```

```
│   ├── staging/                  _sources.yml, _staging.yml, stg_*.sql   (views)
│   ├── intermediate/             int_order_lines_enriched.sql            (ephemeral)
│   └── marts/                    _marts.yml, fct_order_lines.sql (incremental),
│                                 fct_daily_revenue.sql (contract), dim_customers.sql
├── snapshots/customers_snapshot.yml
├── macros/                       generate_schema_name.sql, money.sql
└── tests/                        assert_revenue_reconciles.sql, generic/*.sql
```

The raw seed data is ByteMart's familiar dataset from Chapter 8, with one realistic twist: the order service sends `status` in mixed case (`delivered`, `Delivered`, `DELIVERED`). On Snowflake, the raw tables would be loaded by the pipelines of Chapters 12–14; locally, `dbt seed` loads them into a `raw` schema.

Project configuration sets materializations and schemas per folder:

```yaml
name: bytemart
version: "1.0.0"
config-version: 2
profile: bytemart

model-paths: ["models"]
seed-paths: ["seeds"]
snapshot-paths: ["snapshots"]
macro-paths: ["macros"]
test-paths: ["tests"]
clean-targets: ["target", "dbt_packages"]

vars:
  revenue_statuses: ["DELIVERED", "SHIPPED", "PENDING"]   # statuses that count as revenue

seeds:
  bytemart:
    raw:                      # local stand-in for tables loaded by ingestion pipelines
      +schema: raw
      +column_types:
        unit_price: decimal(10,2)
        order_ts: timestamp

models:
  bytemart:
    staging:
      +materialized: view
      +schema: staging
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
      +schema: marts
      +snowflake_warehouse: transform_wh     # used on Snowflake, ignored by DuckDB

snapshots:
  bytemart:
    +target_schema: snapshots
```

Schema naming macro. By default dbt names custom schemas `<target_schema>_<custom>`. This override keeps clean names (`staging`, `marts`) in production and prefixes them with the developer's schema in `dev`, so developers cannot overwrite each other's objects:

```
{#-
  Use the custom schema name as-is (raw, staging, marts) in prod and local,
```

```
  and prefix it with the developer's schema in dev, so developers never
  overwrite each other's objects.
-#}
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- if custom_schema_name is none -%}
        {{ target.schema }}
    {%- elif target.name == 'dev' -%}
        {{ target.schema }}_{{ custom_schema_name | trim }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}
{%- endmacro %}
```

A staging model does one job: rename, cast, clean. No joins, no business logic:

```
select
    cast(order_id as integer)          as order_id,
    cast(customer_id as integer)       as customer_id,
    cast(order_ts as timestamp)        as order_ts,
    cast(order_ts as date)             as order_date,
    upper(trim(status))                as status,       -- source sends mixed case
    lower(trim(channel))               as channel
from {{ source('bytemart_raw', 'orders') }}
```

A reusable macro keeps monetary rounding consistent everywhere:

```
{# Round a monetary expression consistently and cast it to a fixed-scale decimal. #}
{% macro money(expr, precision=12) -%}
    cast(round({{ expr }}, 2) as decimal({{ precision }}, 2))
{%- endmacro %}
```

An intermediate model joins staging models; it is `ephemeral`, so dbt inlines it as a CTE into the models that use it:

```
-- One row per order line, enriched with order and product attributes.
select
    l.order_id,
    l.line_no,
    o.order_ts,
    o.order_date,
    o.customer_id,
    o.channel,
    o.status,
    l.product_id,
    p.category,
    l.quantity,
    l.unit_price,
    p.list_price,
    {{ money('l.quantity * l.unit_price') }}                    as line_revenue,
    {{ money('l.quantity * (p.list_price - l.unit_price)') }}   as line_discount
from {{ ref('stg_order_lines') }} l
join {{ ref('stg_orders') }}   o on o.order_id   = l.order_id
join {{ ref('stg_products') }} p on p.product_id = l.product_id
```

Tests on staging models — generic tests declared in YAML (dbt ≥ 1.10 places test parameters under `arguments`):

```yaml
version: 2

models:
  - name: stg_customers
    description: One row per customer, cleaned.
    columns:
      - name: customer_id
        data_tests: [unique, not_null]
      - name: email
        data_tests: [unique, not_null]
      - name: loyalty_tier
        data_tests:
          - accepted_values:
              arguments:
                values: ["GOLD", "SILVER", "BRONZE", "NONE"]
      - name: referred_by_customer_id
        data_tests:
          - relationships:
              arguments:
                to: ref('stg_customers')
                field: customer_id

  - name: stg_products
    columns:
      - name: product_id
        data_tests: [unique, not_null]
      - name: list_price
        data_tests: [not_null, non_negative]

  - name: stg_orders
    columns:
      - name: order_id
        data_tests: [unique, not_null]
      - name: status
        data_tests:
          - accepted_values:
              arguments:
                values: ["PENDING", "SHIPPED", "DELIVERED", "CANCELLED"]
      - name: customer_id
        data_tests:
          - relationships:
              arguments:
                to: ref('stg_customers')
                field: customer_id

  - name: stg_order_lines
    data_tests:
      - unique_combination:
          arguments:
            columns: ["order_id", "line_no"]
    columns:
      - name: quantity
        data_tests: [not_null, non_negative]
      - name: product_id
        data_tests:
          - relationships:
              arguments:
                to: ref('stg_products')
                field: product_id
```

`non_negative` and `unique_combination` are **custom generic tests** written in the project (no package needed):

```
{# Generic test: fails for every row where the column is negative. #}
{% test non_negative(model, column_name) %}
select {{ column_name }}
from {{ model }}
where {{ column_name }} < 0
{% endtest %}
```

Marts. A daily revenue mart that uses a project variable for the statuses that count as revenue:

```
select
    order_date,
    channel,
    category,
    cast(count(distinct order_id) as bigint) as orders,
    cast(sum(quantity) as bigint)      as units,
    {{ money('sum(line_revenue)', 14) }} as revenue
from {{ ref('fct_order_lines') }}
where status in (
    {%- for s in var('revenue_statuses') %}'{{ s }}'{{ "," if not loop.last }}{% endfor -%}
)
group by order_date, channel, category
```

Its contract, the other marts' tests, and a **unit test** of the customer segmentation logic:

```
version: 2

models:
  - name: fct_order_lines
    description: Order-line grain fact table (incremental).
    data_tests:
      - unique_combination:
          arguments:
            columns: ["order_id", "line_no"]
    columns:
      - name: line_revenue
        data_tests: [not_null, non_negative]

  - name: fct_daily_revenue
    description: Revenue per day, channel and category for revenue-bearing statuses.
    config:
      contract:
        enforced: true
    columns:
      - name: order_date
        data_type: date
        constraints: [{type: not_null}]
      - name: channel
        data_type: varchar
      - name: category
        data_type: varchar
      - name: orders
        data_type: bigint
      - name: units
        data_type: bigint
      - name: revenue
        data_type: decimal(14,2)
```

```yaml
  - name: dim_customers
    description: One row per customer with lifetime metrics.
    columns:
      - name: customer_id
        data_tests: [unique, not_null]
      - name: value_segment
        data_tests:
          - accepted_values:
              arguments:
                values: ["HIGH", "MEDIUM", "LOW", "NO_ORDERS"]

unit_tests:
  - name: test_value_segment_boundaries
    description: Segment thresholds are inclusive at 20,000 and 5,000.
    model: dim_customers
    given:
      - input: ref('stg_customers')
        rows:
          - {customer_id: 1, customer_name: A, loyalty_tier: GOLD}
          - {customer_id: 2, customer_name: B, loyalty_tier: NONE}
          - {customer_id: 3, customer_name: C, loyalty_tier: NONE}
          - {customer_id: 4, customer_name: D, loyalty_tier: NONE}
      - input: ref('fct_order_lines')
        rows:
          - {customer_id: 1, order_id: 10, status: DELIVERED, line_revenue: 20000.00,
order_date: 2026-01-01}
          - {customer_id: 2, order_id: 11, status: DELIVERED, line_revenue: 4999.99,
order_date: 2026-01-01}
          - {customer_id: 3, order_id: 12, status: CANCELLED, line_revenue: 9000.00,
order_date: 2026-01-01}
    expect:
      rows:
        - {customer_id: 1, value_segment: HIGH,      lifetime_orders: 1}
        - {customer_id: 2, value_segment: LOW,       lifetime_orders: 1}
        - {customer_id: 3, value_segment: NO_ORDERS, lifetime_orders: 0}
        - {customer_id: 4, value_segment: NO_ORDERS, lifetime_orders: 0}
```

The unit test feeds four customers and three order lines into `dim_customers` and checks the segment boundaries — including that a cancelled order does not count. It tests logic, not data, and runs before the model is built.

A singular test reconciles the mart against a direct computation:

```sql
-- Singular test: revenue in the daily mart must equal revenue computed directly
-- from staged order lines for revenue-bearing orders. Returns rows only on mismatch.
with mart as (
    select sum(revenue) as revenue from {{ ref('fct_daily_revenue') }}
),
direct as (
    select sum(l.quantity * l.unit_price) as revenue
    from {{ ref('stg_order_lines') }} l
    join {{ ref('stg_orders') }} o on o.order_id = l.order_id
    where o.status in (
        {%- for s in var('revenue_statuses') %}'{{ s }}'{{ "," if not loop.last }}{% endfor -
%}
    )
)
select mart.revenue as mart_revenue, direct.revenue as direct_revenue
```

```
from mart cross join direct
where mart.revenue <> direct.revenue
```

Running the project

`dbt seed` loads the raw stand-ins, then `dbt build` runs models, snapshots, unit tests and data tests in DAG order (a model's tests run before anything downstream of it is built):

```
$ dbt seed --target local
1 of 4 OK loaded seed file raw.customers ..................................... [INSERT 8 in
0.23s]
2 of 4 OK loaded seed file raw.order_lines ................................... [INSERT 17
in 0.26s]
3 of 4 OK loaded seed file raw.orders ........................................ [INSERT 12
in 0.25s]
4 of 4 OK loaded seed file raw.products ...................................... [INSERT 7 in
0.21s]
Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=4

$ dbt build --target local --exclude resource_type:seed      (excerpt)
4 of 34 OK created sql view model staging.stg_products ....................... [OK in
0.27s]
2 of 34 OK created sql view model staging.stg_order_lines .................... [OK in
0.27s]
1 of 34 OK created sql view model staging.stg_customers ...................... [OK in
0.31s]
3 of 34 OK created sql view model staging.stg_orders ......................... [OK in
0.30s]
5 of 34 OK snapshotted snapshots.customers_snapshot .......................... [OK in
0.35s]
24 of 34 OK created sql incremental model marts.fct_order_lines ............... [OK in
0.10s]
28 of 34 OK created sql table model marts.fct_daily_revenue ................... [OK in
0.27s]
30 of 34 PASS assert_revenue_reconciles ...................................... [PASS in
0.06s]
29 of 34 PASS dim_customers::test_value_segment_boundaries .................... [PASS in
0.33s]
31 of 34 OK created sql table model marts.dim_customers ....................... [OK in
0.04s]
Finished running 1 incremental model, 1 snapshot, 2 table models, 25 data tests, 1 unit test,
4 view models in 0 hours 0 minutes and 2.20 seconds (2.20s).
Done. PASS=34 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=34
```

The daily revenue mart:

```
order_date | channel | category    | orders | units | revenue
-----------+---------+-------------+--------+-------+----------
2026-01-05 | web     | Electronics | 1      | 2     | 1798.00
2026-01-05 | web     | Home        | 1      | 1     | 749.00
2026-01-07 | app     | Electronics | 1      | 1     | 12999.00
2026-01-20 | app     | Electronics | 1      | 2     | 5398.00
2026-02-03 | web     | Home        | 1      | 4     | 5296.00
2026-02-11 | web     | Electronics | 1      | 2     | 13898.00
2026-02-14 | app     | Home        | 1      | 4     | 2996.00
2026-02-20 | app     | Furniture   | 1      | 1     | 15999.00
2026-03-01 | web     | Home        | 1      | 1     | 1899.00
2026-03-03 | app     | Electronics | 1      | 1     | 4499.00
2026-03-08 | app     | Electronics | 1      | 1     | 12499.00
```

```
2026-03-09 | web     | Electronics | 1      | 3     | 2697.00
2026-03-09 | web     | Home        | 1      | 1     | 1899.00
```

```
total_revenue | order_days
--------------+------------
82626.00      | 13
```

Total revenue 82,626.00 is Chapter 9's 98,625.00 for all 17 lines minus the cancelled order 1004 (15,999.00), and the mixed-case statuses were all normalized — otherwise `delivered` rows would have silently fallen out of the `IN (...)` filter. The customer dimension:

```
customer_id | customer_name | lifetime_orders | lifetime_revenue | value_segment
------------+---------------+-----------------+------------------+---------------
1           | Aarav Sharma  | 3               | 9844.00          | MEDIUM
2           | Priya Nair    | 2               | 15995.00         | MEDIUM
3           | Oliver Smith  | 1               | 4596.00          | LOW
4           | Amelia Jones  | 2               | 17795.00         | MEDIUM
5           | Ethan Brown   | 1               | 13898.00         | MEDIUM
6           | Diya Patel    | 1               | 15999.00         | MEDIUM
7           | Noah Wilson   | 1               | 4499.00          | LOW
8           | Kavya Reddy   | 0               | 0.00             | NO_ORDERS
```

Materializations on Snowflake

Materialization	dbt-snowflake behaviour	Use for
`view`	`CREATE OR REPLACE VIEW`	Staging; light logic
`table`	`CREATE OR REPLACE TABLE … AS SELECT` — **transient by default** in dbt-snowflake (no Fail-safe); set `transient: false` for tables that need Fail-safe	Marts rebuilt cheaply
`incremental`	Inserts/merges only new or changed rows (Section 16.5)	Large facts
`ephemeral`	Inlined CTE, no object	Intermediate steps used once
`dynamic_table`	`CREATE DYNAMIC TABLE` with `target_lag`, `snowflake_warehouse`, `refresh_mode`, `initialize` configs	Freshness-driven marts refreshed by Snowflake rather than by dbt runs
`snapshot`	SCD2 table maintained by `dbt snapshot`	History of mutable sources

```sql
-- A dbt model materialized as a Snowflake dynamic table
{{ config(
    materialized = 'dynamic_table',
    snowflake_warehouse = 'transform_wh',
    target_lag = '15 minutes',
    refresh_mode = 'INCREMENTAL'
) }}
select order_date, channel, sum(line_revenue) as revenue
from {{ ref('fct_order_lines') }}
group by all
```

Other useful dbt-snowflake configs: `copy_grants: true` (keep grants across rebuilds), `grants: {select: ['analyst_ro']}`, `query_tag` (cost attribution), `cluster_by`, `secure: true` for views, `snowflake_warehouse` per model (route heavy models to a larger warehouse).

16.5 Incremental models

An **incremental** model processes only new or changed source rows on each run and merges them into the existing table. The model SQL contains a filter that applies only when `is_incremental()` is true (the table exists and no full refresh was requested):

```
{{
    config(
        materialized = 'incremental',
        unique_key = ['order_id', 'line_no'],
        incremental_strategy = 'delete+insert',
        on_schema_change = 'append_new_columns'
    )
}}

select *
from {{ ref('int_order_lines_enriched') }}

{% if is_incremental() %}
-- Reprocess a 3-day window behind the latest loaded order to catch late updates.
where order_ts >= (
    select {{ dbt.dateadd('day', -3, 'max(order_ts)') }} from {{ this }}
)
{% endif %}
```

Strategies

`incremental_strategy`	Behaviour on Snowflake	Needs `unique_key`	Notes
`merge` (default on Snowflake)	`MERGE` on the unique key	Yes (otherwise behaves like append)	Duplicate keys in the new batch cause a nondeterministic-merge error
`delete+insert`	Delete target rows with keys in the batch, then insert	Yes	Robust to duplicates within a key group; used in the ByteMart project because it behaves the same on DuckDB
`append`	Insert only	No	Event logs that never change
`insert_overwrite`	Replace the whole table contents with the batch on Snowflake	No	Rarely what you want on Snowflake — read the adapter docs first
`microbatch`	dbt splits the work into time batches by `event_time`, each run independently and retryable	No (uses `event_time`, `batch_size`, `begin`)	Large time-series facts and backfills

`on_schema_change` (`ignore` default, `append_new_columns`, `sync_all_columns`, `fail`) controls what happens when the model's columns change.

The late-update trap (executed)

The model above reprocesses a 3-day window behind the latest loaded `order_ts`. Watch what happens with real changes. The source receives a **new order 1013** (10 March) and a **status change on order 1010** (placed 3 March, now cancelled). Then the incremental model and its descendants run:

```
$ dbt run --select fct_order_lines+ --target local
1 of 3 OK created sql incremental model marts.fct_order_lines .................. [OK in
0.24s]
2 of 3 OK created sql table model marts.dim_customers ......................... [OK in
0.21s]
3 of 3 OK created sql table model marts.fct_daily_revenue ..................... [OK in
0.24s]
Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=3
```

```
order_id | line_no | order_date | status  | line_revenue
----------+---------+------------+---------+--------------
1010     | 1       | 2026-03-03 | PENDING | 4499.00
1013     | 1       | 2026-03-10 | PENDING | 798.00
```

The new order arrived — but order **1010 is still `PENDING`**. The window started three days before the latest `order_ts` (9 March), so a change to an order placed on 3 March was never read. No error, no test failure: the fact table is silently wrong, and revenue still includes a cancelled order.

The causes are structural: the filter uses the **business timestamp** (`order_ts`) rather than a **change timestamp**, and the lookback window is shorter than how late updates can arrive. A full refresh repairs the data:

```
$ dbt run --select fct_order_lines+ --full-refresh --target local
1 of 3 OK created sql incremental model marts.fct_order_lines .................. [OK in
0.21s]
2 of 3 OK created sql table model marts.dim_customers ......................... [OK in
0.18s]
3 of 3 OK created sql table model marts.fct_daily_revenue ..................... [OK in
0.26s]
Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=3
```

```
order_id | line_no | order_date | status    | line_revenue
----------+---------+------------+-----------+--------------
1010     | 1       | 2026-03-03 | CANCELLED | 4499.00
1013     | 1       | 2026-03-10 | PENDING   | 798.00
```

Production fixes, in order of preference:

1. **Filter on a change timestamp** — `updated_at` from the source, or the raw table's `_loaded_at` (Chapter 12) — not on the business date. Every changed row is then picked up, however old the order.
2. If you must use a business date, make the lookback window **longer than the maximum lateness** you observe, and measure that lateness.
3. Drive the model from a **stream** or a **dynamic table** (Chapters 14–15), which see every change by construction.

4. Schedule periodic `--full-refresh` runs (for example weekly) and reconcile counts and sums against the source.

16.6 Snapshots

A **snapshot** records how a mutable source changes over time — dbt's implementation of SCD Type 2 (Chapter 9). Since dbt 1.9, snapshots are configured in YAML:

```
snapshots:
  - name: customers_snapshot
    relation: source('bytemart_raw', 'customers')
    description: SCD Type 2 history of customer city and loyalty tier.
    config:
      unique_key: customer_id
      strategy: check
      check_cols: ["city", "loyalty_tier"]
      hard_deletes: invalidate
```

The `check` strategy compares the listed columns on each run (use the `timestamp` strategy with an `updated_at` column when the source has a reliable one — it is cheaper). `hard_deletes: invalidate` closes the current version when a row disappears from the source.

Customer 2 (Priya) moves from Kochi to Bengaluru and is upgraded to GOLD; then the snapshot runs again:

```
$ dbt snapshot --target local
1 of 1 OK snapshotted snapshots.customers_snapshot ............................ [OK in
0.30s]
Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=1
```

```
customer_id | city      | loyalty_tier | is_current
------------+-----------+--------------+-----------
2           | Kochi     | SILVER       | False
2           | Bengaluru | GOLD         | True
```

dbt closed the old version (`dbt_valid_to` set) and inserted a new current one. Snapshots capture only the state seen **at each run**: if a value changes twice between runs, the intermediate value is never recorded. For complete history, snapshot more often or build history from CDC events (Chapter 14).

16.7 When a test fails

A new status value, `Returned`, appears in the source (the returns feature launched without telling the data team). The next test run:

```
$ dbt test --select stg_orders --target local
1 of 5 FAIL 1 accepted_values_stg_orders_status__PENDING__SHIPPED__DELIVERED__CANCELLED
[FAIL 1 in 0.24s]
Got 1 result, configured to fail if != 0
Done. PASS=4 WARN=0 ERROR=1 SKIP=0 NO-OP=0 REUSED=0 TOTAL=5
```

The `accepted_values` test caught the contract breach before any mart was rebuilt (`dbt build` would have skipped everything downstream of `stg_orders`). The right response is a conversation, not a code tweak: is `RETURNED` revenue? Does it need a refund amount? The answer updates the accepted values, `revenue_statuses`, and the documentation — in one reviewed change.

To investigate failures, `dbt test --store-failures` writes failing rows to tables in the warehouse; `dbt show --select <model>` previews model output; the compiled SQL is in `target/compiled/`.

16.8 Packages and documentation

Packages add tested macros and models. A typical `packages.yml`:

```
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.3.0", "<2.0.0"]
```

`dbt deps` installs them into `dbt_packages/`. Pin version ranges; review package code like your own (packages run SQL with your role). The ByteMart project deliberately uses no packages so that it runs offline; in a real project `dbt_utils.unique_combination_of_columns` would replace the hand-written `unique_combination` test.

Documentation. Descriptions in YAML plus `dbt docs generate` produce a browsable site with lineage. On Snowflake, `persist_docs: {relation: true, columns: true}` writes descriptions into object and column comments, where Snowsight, Horizon Catalog and AI features (Chapters 23–27) can read them.

16.9 Schema evolution, lineage and observability

Schema evolution across layers. New source columns arrive first in bronze (automatically with schema evolution, Chapter 12, or inside a `VARIANT` payload). They should *not* flow automatically into gold. In dbt, staging models select explicit columns, so a new source column is ignored until someone adds it; incremental models choose `on_schema_change` deliberately (`append_new_columns` in silver, `fail` for contracted gold models); contracts make breaking changes fail loudly. Removed or renamed source columns break staging models immediately — which is what you want, because the failure is close to the cause.

Lineage answers "where does this number come from?" and "what breaks if I change this?". Three complementary sources:

Lineage source	Granularity	Where
dbt DAG (`ref`/`source`)	Model to model, and column-level in some tools	`dbt docs`, dbt Projects on Snowflake views in Snowsight
Snowflake object lineage (Horizon)	Table/view/column, from executed SQL including non-dbt jobs	Snowsight lineage graph; `ACCOUNT_USAGE.ACCESS_HISTORY` (Chapter 21)
Ingestion provenance	Row to file/offset	Provenance columns in bronze (Chapter 12)

Observability means knowing that pipelines are healthy *without* waiting for a user to complain: run status (task history, dbt run results), freshness (source freshness, `MAX(_loaded_at)` alerts), volume (row counts against history), quality (test results stored over time, data metric functions), and cost (credits per pipeline via query tags). Chapter 19 builds an observability layer on these signals; at minimum, store dbt `run_results.json` outcomes and test failures in an `OPS` table and alert on failures and freshness.

16.10 dbt Projects on Snowflake

dbt Projects on Snowflake run dbt natively inside Snowflake: no external dbt runtime, scheduler or credentials to manage. Snowflake provides managed dbt Core (and dbt Fusion) runtimes; you develop in **Workspaces** in Snowsight or locally, deploy a **DBT PROJECT** object, and execute it with SQL.

```
-- 1. Deploy from a Git repository stage (Chapter 11 shows the GIT REPOSITORY object)
CREATE OR REPLACE DBT PROJECT bytemart.ops.bytemart_dbt
  FROM '@bytemart.ops.bytemart_repo/branches/main/bytemart_dbt'
  DBT_VERSION = '1.12.3'
  DEFAULT_TARGET = 'prod'
  EXTERNAL_ACCESS_INTEGRATIONS = (dbt_hub_access)        -- only if packages must be fetched
  COMMENT = 'ByteMart transformations';

-- 2. Run it
EXECUTE DBT PROJECT bytemart.ops.bytemart_dbt ARGS = 'build --target prod';

-- 3. Schedule it: run, then test, every morning
CREATE OR REPLACE TASK bytemart.ops.t_dbt_build
  WAREHOUSE = transform_wh
  SCHEDULE = 'USING CRON 30 2 * * * Asia/Kolkata'
AS
  EXECUTE DBT PROJECT bytemart.ops.bytemart_dbt ARGS = 'build --target prod';
ALTER TASK bytemart.ops.t_dbt_build RESUME;
```

Topic	Detail (at the research cutoff)
Sources for `FROM`	Named internal stage, Git repository stage branch path, a workspace, or another dbt project object; the directory must contain one `dbt_project.yml` at its root
dbt version	`DBT_VERSION` pins it; otherwise the account's `DEFAULT_DBT_VERSION` applies (an execution without a pinned version falls back to 1.9.4)
Profiles	`profiles.yml` (or `dbt_projects_profiles.yml`, which takes precedence) defines targets; dbt runs with the target's role, limited to the calling user's privileges
Compilation	`AUTO_COMPILE = TRUE` (default) runs `dbt deps` (when an external access integration is set) and `dbt compile` at deployment
Results	`EXECUTE DBT PROJECT` returns success, exception, stdout and an `OUTPUT_ARCHIVE_URL` with logs and `target/` artifacts; Snowsight shows run history and lineage
Run options	`ARGS` (any dbt command and flags), `DBT_VERSION` override, `ENVIRONMENT` / `ENV_VARS` (keys must start with `DBT_`), `WRITEBACK`
Slim CI	Import the artifacts of the last successful production run (`IMPORTS` with `SYSTEM$DBT_GET_LAST_SUCCESSFUL_RUN_TARGET`) and run `state:modified+` with `--state` (GA 10 Sep 2026; some features need the 2026_06 behavior change bundle)
Privileges	`CREATE DBT PROJECT` on the schema to deploy; `USAGE` on the project to execute
CLI	`snow dbt deploy` and `snow dbt execute` wrap the same operations for CI pipelines

Topic	Detail (at the research cutoff)
Cost	Runs on your warehouse; no separate licence

Choose dbt Projects on Snowflake when your transformations live entirely in Snowflake and you want one platform for run, schedule, observe and govern. Choose dbt Cloud / the dbt platform or self-hosted dbt with an external orchestrator when you need its IDE and features, cross-warehouse projects, or an existing orchestration estate.

16.11 CI/CD for dbt

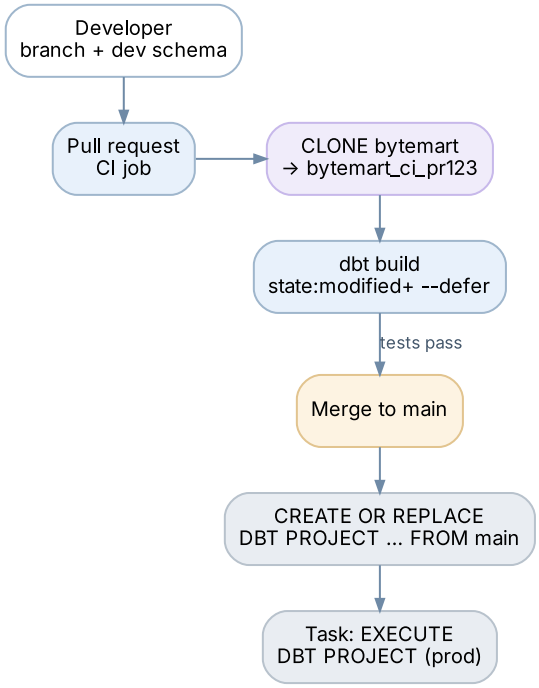


Figure 16.3 — dbt CI/CD: pull requests build only what changed, against a zero-copy clone of production; merges deploy to production.

On every pull request:

1. Create an isolated CI schema or database — ideally a **zero-copy clone** of production (`CREATE DATABASE bytemart_ci_pr123 CLONE bytemart;`), so tests run against production-shaped data at no storage cost until changed.
2. Build only modified models and their descendants, deferring unmodified upstream references to production: `dbt build --select state:modified+ --defer --state ./prod-artifacts`.
3. Fail the PR on test failures; post a summary.
4. Drop the CI database when the PR closes.

On merge to main: deploy (`CREATE OR REPLACE DBT PROJECT … FROM` the main branch, or `snow dbt deploy`), then run in production through the scheduled task.

An illustrative GitHub Actions job (not executed in this book):

```yaml
name: dbt-ci
on: pull_request
jobs:
  build:
    runs-on: ubuntu-latest
    env:
      SNOWFLAKE_ACCOUNT: ${{ secrets.SNOWFLAKE_ACCOUNT }}
      SNOWFLAKE_USER: ${{ secrets.SNOWFLAKE_CI_USER }}
      SNOWFLAKE_PRIVATE_KEY_PATH: ${{ runner.temp }}/ci_key.p8
      PRIVATE_KEY_PASSPHRASE: ${{ secrets.SNOWFLAKE_CI_KEY_PASSPHRASE }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install "dbt-snowflake~=1.10"
      - name: Write private key
        run: printf '%s' "${{ secrets.SNOWFLAKE_CI_PRIVATE_KEY }}" >
"$SNOWFLAKE_PRIVATE_KEY_PATH"
      - name: Fetch production artifacts
        run: ./scripts/fetch_prod_manifest.sh prod-artifacts      # e.g. from a stage or
artifact store
      - name: Build changed models
        working-directory: bytemart_dbt
        run: dbt build --target ci --select state:modified+ --defer --state ../prod-artifacts
```

Notes: secrets come from the CI secret store, never from the repository; the CI service user has a role that can create and drop only CI databases; the `ci` target in `profiles.yml` points at the PR's clone; pin the adapter version range used in production.

16.12 Other transformation approaches

Approach	Strengths	Consider when
dbt (Core, dbt platform, or dbt Projects on Snowflake)	SQL-first, testing, lineage, huge community	Most analytics engineering
Dynamic tables (Chapter 15)	Declarative freshness, incremental refresh, no scheduler	Pipelines defined by "keep this result fresh"; can be generated by dbt
Stored procedures + tasks (Chapters 10, 15)	Full procedural control	Complex operational logic, file and API interactions
Snowpark (Chapter 11)	Python DataFrames, ML integration	Python-heavy teams, feature engineering
Matillion	Visual, low-code ELT with orchestration; pushes SQL down to Snowflake	Mixed-skill teams wanting a GUI
Coalesce	Snowflake-focused, metadata-driven transformation with templates	Standardized, column-aware patterns at scale

These are complementary: a common pattern is dbt for silver/gold logic, dynamic tables as a dbt materialization for low-latency marts, procedures for operational jobs, and tasks or an orchestrator for scheduling.

16.13 End-to-end: the ByteMart platform so far

Putting Parts III and IV together:

1. **Ingest** — product CSVs by scheduled COPY (Chapter 12); orders by Snowpipe auto-ingest; clickstream by Kafka connector 4.x (Chapter 13); inventory CDC by Openflow (Chapter 14).
2. **Bronze** — `RAW` schema with provenance columns, audit table, rejects.
3. **Silver/Gold** — dbt project (this chapter) deployed as a dbt Project on Snowflake; high-freshness marts as dynamic tables.
4. **Orchestrate** — a nightly task graph (Chapter 15) whose steps include `EXECUTE DBT PROJECT … build`; triggered tasks for CDC apply; finalizer for audit.
5. **Quality** — dbt tests and contracts in every build; reconciliation tests; freshness alerts.
6. **Deliver** — CI/CD with zero-copy clones; production written only by the deployment pipeline.

Part V turns to making this platform fast and cost-efficient; Part VI to securing and governing it.

16.14 Guided lab

> **LAB 16.1 — RUN, BREAK AND FIX THE BYTEMART DBT PROJECT LOCALLY**
> **Environment:** any machine with Python 3.10+. No Snowflake account needed. Files: `code/ch16/` in the book's source package.

Step 1. `pip install dbt-duckdb` (this book used dbt Core 1.12.5 and dbt-duckdb 1.11.0).

Step 2. In `code/ch16/bytemart_dbt`: `export DBT_PROFILES_DIR=$PWD`, then `dbt seed --target local` and `dbt build --target local --exclude resource_type:seed`. **Expected:** `PASS=34 … ERROR=0`.

Step 3. `dbt show --select fct_daily_revenue --target local --limit 20`. **Expected:** the 13 rows shown in Section 16.4; total revenue 82,626.00.

Step 4. Reproduce the late-update trap: run `python3 ../demo.py` (it rebuilds everything and applies the changes), or apply the two changes by hand with the DuckDB CLI and run `dbt run --select fct_order_lines+`. Confirm order 1010 stays `PENDING`.

Step 5. Fix the model properly: add an `_updated_at` column to the raw orders seed, carry it through staging, and filter the incremental model on it. Change 1010 again and show that an ordinary incremental run now picks it up.

Step 6. Break the contract: change `fct_daily_revenue` to output `revenue` as `cast(sum(line_revenue) as double)` and run `dbt run -s fct_daily_revenue`. **Expected** (verified with the book's environment): a compilation error, "This model has an enforced contract that failed", with a table row `revenue | DOUBLE | DECIMAL(14,2) | data type mismatch`. The previous version of the table is left untouched.

Step 7 (with a Snowflake account). Add your account details to the `dev` target via environment variables, install `dbt-snowflake`, and run `dbt build --target dev`; then deploy the project with `CREATE DBT PROJECT` from a Git repository stage or a workspace and run it with `EXECUTE DBT PROJECT`.

16.15 Production use case: from scripts to a governed dbt platform

Before. ByteMart's analysts maintained 140 SQL scripts run by cron on a VM: no tests, no lineage, a shared service password, and every Monday someone fixed numbers by hand.

After (12 weeks).

- Week 1–2: medallion layout and naming; raw schemas owned by ingestion; database roles per schema.
- Week 3–6: scripts migrated into dbt staging and marts, starting from the 20 most-used reports; tests on every primary key and status column; reconciliation tests on revenue.
- Week 7–8: contracts on finance-facing marts; dynamic table materialization for the two dashboards needing 15-minute freshness.
- Week 9–10: dbt Projects on Snowflake with a scheduled task graph; CI on pull requests against zero-copy clones.
- Week 11–12: docs persisted to Snowflake comments; freshness alerts; the VM retired.

Outcome. Monday corrections stopped; the late-update bug class was found by reconciliation tests in week 4 (the same failure mode as Section 16.5) and fixed by switching incremental filters to `_loaded_at`; credit use fell because staging became views and marts incremental.

16.16 Common mistakes

ARCHITECTURE AND DBT PITFALLS

1. Business logic in staging models; or staging skipped entirely.
2. Incremental filters on business dates instead of change timestamps.
3. `merge` incremental models with duplicate keys in the batch.
4. No tests on primary keys of facts.
5. Forgetting that dbt-snowflake tables are transient by default.
6. Hard-coded database/schema names instead of `ref()` / `source()`.
7. Developers sharing one dev schema.
8. Packages without version pins.
9. Credentials in `profiles.yml` committed to Git.
10. Contracts declared but never enforced (`enforced: true` missing).

16.17 Troubleshooting exercise

> **EXERCISE — 'REVENUE DOUBLED OVERNIGHT'**
> After a release, `fct_daily_revenue` shows twice the expected revenue for the last three days only. `fct_order_lines` is incremental with `incremental_strategy = 'append'` and the 3-day lookback filter from Section 16.5. What happened, how do you prove it, and how do you fix both the data and the model?

16.18 Knowledge check

1. What belongs in bronze, silver and gold?
2. Why one database per environment and schemas per layer?
3. Name three places to enforce a data contract.
4. What is the difference between a data test and a unit test in dbt?
5. What does an ephemeral model become?
6. Which incremental strategy is the default on Snowflake, and what does it require?
7. Why did order 1010 stay `PENDING` after the incremental run?
8. What does the `check` snapshot strategy compare, and what can snapshots miss?
9. What does `EXECUTE DBT PROJECT` return?
10. What does `state:modified+ --defer` achieve in CI?
11. Why use zero-copy clones for CI?
12. What is the default table type created by dbt-snowflake, and why does it matter?

16.19 Hands-on assignment

1. Complete Lab 16.1 Steps 1–6.
2. Add `fct_orders` (order grain) with a contract and tests, and `dim_products`; add an `agg_customer_monthly` model materialized as `dynamic_table` (configs only — it will not run on DuckDB; use `enabled: "{{ target.type == 'snowflake' }}"`).
3. Write `stg_orders` freshness rules and a `dbt source freshness` run step for the nightly task graph.
4. Write a one-page data contract for `fct_daily_revenue`: grain, columns, types, owners, freshness, quality rules, change policy.

16.20 Chapter summary

A sound Snowflake data platform separates bronze, silver and gold layers, environments and responsibilities, with contracts and quality checks placed where they protect consumers. dbt turns SQL `SELECT`s into a tested, documented DAG: sources, staging views, ephemeral intermediates, incremental facts, contracted marts, snapshots, macros and tests. Incremental models are powerful and dangerous: filter on change timestamps, size

lookbacks from measured lateness, reconcile and full-refresh periodically — the executed example showed a silently wrong fact table otherwise. On Snowflake, dbt adds transient tables, dynamic table materializations, grants and query tags, and dbt Projects on Snowflake run, schedule and observe dbt natively. CI/CD with zero-copy clones and state-based selection keeps changes safe. dbt, dynamic tables, procedures, Snowpark and tools such as Matillion and Coalesce complement each other.

16.21 Key takeaways

- One job per layer; data flows forward only.
- Contracts at the edges, tests everywhere, reconciliation always.
- Incremental = filter on change time + reconciliation + periodic full refresh.
- dbt Projects on Snowflake: deploy with SQL, run with SQL, schedule with tasks.
- CI on clones, deploy only from main.

16.22 Further reading

- dbt documentation: https://docs.getdbt.com/docs/introduction
- dbt incremental models: https://docs.getdbt.com/docs/build/incremental-models
- dbt unit tests: https://docs.getdbt.com/docs/build/unit-tests
- dbt model contracts: https://docs.getdbt.com/docs/mesh/govern/model-contracts
- dbt Snowflake configurations: https://docs.getdbt.com/reference/resource-configs/snowflake-configs
- dbt Projects on Snowflake: https://docs.snowflake.com/en/user-guide/data-engineering/dbt-projects-on-snowflake
- CREATE DBT PROJECT: https://docs.snowflake.com/en/sql-reference/sql/create-dbt-project
- EXECUTE DBT PROJECT: https://docs.snowflake.com/en/sql-reference/sql/execute-dbt-project
- dbt-duckdb adapter: https://github.com/duckdb/dbt-duckdb

16.23 Solutions

Troubleshooting — revenue doubled. With `append`, every run inserts all rows in the 3-day lookback window again — rows already in the table are duplicated. Proof: `SELECT order_id, line_no, COUNT(*) FROM fct_order_lines GROUP BY 1, 2 HAVING COUNT(*) > 1` returns the last three days' lines; the `unique_combination` test on the fact would have failed (was it removed or set to `warn`?). Fix the data with `dbt run --select fct_order_lines+ --full-refresh`; fix the model by using `merge` or `delete+insert` with `unique_key = ['order_id','line_no']`; restore the uniqueness test at `error` severity; add the reconciliation test so this class of error cannot reach dashboards again.

Knowledge check answers.

1. Bronze: data as delivered with provenance; silver: typed, cleaned, de-duplicated, conformed, historized; gold: business-ready facts, dimensions, metrics.

2. Isolation, simple access control (grant per database/schema), safe cloning, and identical code across environments with only the database changing.
3. Any three of: ingestion schema checks, source tests/freshness, dbt model contracts, Snowflake NOT NULL/CHECK constraints, versioned models/semantic views.
4. A data test queries real data for violations; a unit test runs a model's logic on fixed inputs and compares with expected outputs.
5. A CTE inlined into the models that reference it; no database object.
6. `merge`; a `unique_key` (and unique keys within each batch).
7. The incremental filter used `order_ts` with a 3-day lookback from the latest order; the change was to an older order, so it was never read.
8. The listed columns' current values against the latest snapshot version; intermediate changes between runs are missed.
9. Success flag, exception, dbt stdout, and an output archive URL with logs and artifacts.
10. Builds and tests only changed models and their descendants, reading unchanged upstream models from production.
11. Production-shaped data instantly, without copying storage, isolated per PR.
12. Transient tables — no Fail-safe; set `transient: false` for tables that need Fail-safe protection.